



National University of Computer and Emerging Sciences, Lahore



InSync-Bringing Intelligence and Humans into Sync

Anum Aamir 23L-0523 BS(CS)

Areej Hafeez 23L-0983 BS(CS)

Haadiyah Zafar 23L-0744 BS(CS)

Supervisor: Mr. Junaid Hussain Muzamal

Final Year Project

October 5, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: InSync-Bringing Intelligence and Humans into Sync

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: October 7, 2026

Name: Anum Aamir

Signature:

Name: Areej Hafeez

Signature:

Name: Haadiyah Zafar

Signature:

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Teaching each student based on their own individual needs is a challenge that persists within education systems. It becomes even more challenging as class size increases and students join the class with diverse prior knowledge. While most AI tools are currently used to tutor, assess, track progress and collaborate, none communicate with each other and share their knowledge regarding the student's performance. This project presents InSync – a web-based learning platform featuring five AI agents working together as a team, sharing one common perspective of each student. The difficulty experienced by the learner during one particular activity influences the next explanation, the level of difficulty of the quiz, the creation of discussion groups, and the advice provided to the teacher. The teacher remains in charge of the whole process and evaluates all the proposals made by the system prior to making any actions. InSync provides greater assistance to the learners and gives the teacher an evidence-based idea of how to assist each particular student and why.

Executive Summary

Today, there are numerous AI-based learning tools available to both students and teachers, but they function independently from one another. For example, a tutoring tool can explain some concepts to a learner, the quiz tool tests the user's knowledge, and the dashboard provides performance metrics. However, none of them are sharing their insights regarding each particular learner. Thus, if a person has problems with a certain concept during the test, the next tutoring will not adjust itself and the teacher will not know about the problem until it is too late. In big classes, the teacher cannot track every student's progress and connect the dots. Thus, in such a way, help is provided too late and is not personalized. That is why we decided to create InSync.

InSync is an educational learning platform that combines tutoring, adaptive testing, monitoring progress and collaboration with other learners. It consists of five AI agents with different responsibilities, all of whom have access to the same record of each learner, known as the shared learner context. The Tutor Agent gives answers to student's questions according to the teacher's learning materials, tailoring its explanation according to the level of the learner. The Quiz Agent sets the level of difficulty of the questions according to previous answers from the learner and identifies the concept that causes difficulty to the learner. The Progress Agent monitors past performance to uncover learning patterns and frequent errors. The Teacher Assistant Agent uses this information to give clear evidence-based recommendations for the teacher. Finally, the Discussion Agent divides learners into groups according to the learner's strengths and weaknesses and provides hints during group discussions without revealing the correct answer. Because all five agents share the learner information, all the insights gained by one of the agents positively affect the way other agents provide help to the learner.

The system is implemented in the form of a web application. The backend is built with FastAPI, and the agents are coordinated using LangGraph. Learners' and activities' information is stored in the same Supabase PostgreSQL database. Explanations, questions and hints are generated by large language models, and it is possible to substitute an AI provider in case of increased costs or reduced quality. Teacher-in-the-loop guarantees that the recommendations and AI-generated groups will only be applied after the teacher's approval. In order to ensure student privacy, only the minimum necessary information about learners is provided to AI services.

InSync makes learning easier for both students and educators. First of all, it provides students with personalized learning experience in terms of explanations and tasks and enables them to study alongside other students who form groups where all participants are able to support each other. Secondly, it gives educators an overview of the entire class right away. Thus, they have the possibility to see where the

problems are in order to act appropriately with all of the reasons presented in front of them. As regards the first prototype, the developers pay special attention to only the Programming Fundamentals subject in order to find an answer to the key question of the project: can coordinated AI agents be more relevant and timely than standalone AI tools?

In general, it can be said that InSync is an appropriate solution to creating a connection between various aspects of learning process. In other words, it enables specialists to merge different AI agents with an understanding of every single learner and keep the educator informed about everything that happens.

Table of Contents

List of Figures	xii
List of Tables	xiii
1 Introduction	1
1.1 Purpose of this Document	1
1.2 Intended Audience	2
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 Conclusion	3
2 Project Vision	4
2.1 Problem Domain Overview	4
2.2 Problem Statement	4
2.3 Problem Elaboration	5
2.4 Goals and Objectives	6
2.5 Project Scope	6
2.6 Sustainable Development Goal (SDG)	7
2.7 Constraints	7
2.8 Business Opportunity	9
2.9 Stakeholders Description/ User Characteristics	9
2.9.1 Stakeholders Summary	9
2.9.2 Key High-Level Goals and Problems of Stakeholders	10
3 Literature Review / Related Work	11
3.1 Definitions, Acronyms, and Abbreviations	11
3.2 Detailed Literature Review	11
3.2.1 MultiTutor: Collaborative LLM Agents for Multimodal Student Support	11
3.2.2 FACET: Multi-Agent AI Supporting Teachers in Scaling Differentiated Learning for Diverse Students	12

3.2.3	Mapping Student-AI Interaction Dynamics in Multi-Agent Learning Environments: Supporting Personalized Learning and Reducing Performance Gaps . . .	13
3.2.4	Artificial Intelligence Agents for Personalized Adaptive Learning	15
3.2.5	A Generative Artificial Intelligence Enhanced Multiagent Approach to Empowering Collaborative Problem Solving Across Different Learning Domains	16
3.2.6	Multiple Agent Designs in Conversational Intelligent Tutoring Systems	17
3.2.7	Multi Agent Configurations for Social Learning with LLMs	18
3.2.8	A Multi Agent Ecosystem for Programming Instruction	19
3.2.9	Khanmigo, an Existing AI Tutoring Product from Khan Academy	20
3.2.10	AgentTutor: Empowering Personalized Learning with Multi-Turn Interactive Teaching	21
3.2.11	EducationQ: Evaluating LLMs' Teaching Capabilities Through Multi-Agent Dialogue	22
3.2.12	LLM Agents for Education: Advances and Applications	23
3.2.13	ALIGNAgent: Adaptive Learner Intelligence for Gap Identification and Next-Step Guidance	24
3.2.14	LecturaAgents: A Multi-Agent Framework for Adaptive Personalized AI-Assisted Learning and Embodied Teaching	25
3.2.15	Impact of AI-Agent-Supported Collaborative Learning on University Programming Courses	26
3.3	Literature Review Summary Table	27
3.4	Conclusion	30
4	Software Requirement Specifications	31
4.1	List of Features	31
4.2	Functional Requirements	32
4.2.1	Student Functional Requirements	32
4.2.2	Teacher Functional Requirements	32
4.3	Quality Attributes	33
4.3.1	Usability	33
4.3.2	Reliability	33
4.3.3	Performance Efficiency	33
4.3.4	Maintainability	34
4.3.5	Flexibility	34
4.4	Non-Functional Requirements	34

4.4.1	Security	34
4.4.2	Data Privacy	34
4.4.3	Availability	35
4.4.4	Compatibility	35
4.4.5	Accessibility	35
4.4.6	AI Transparency and Safety	35
4.5	Assumptions	36
4.6	Use Cases	36
4.6.1	Log In to the System	37
4.6.2	Attend an AI Tutoring Session	38
4.6.3	Attempt an Adaptive Quiz	39
4.6.4	View Personal Progress	40
4.6.5	View Feedback and Study Recommendations	41
4.6.6	Join a Collaborative Workspace	42
4.6.7	Discussion in a Group Chat	43
4.6.8	Receive AI Guided Hints in a Group	44
4.6.9	View Class Dashboard	45
4.6.10	Review an AI Recommendation	46
4.6.11	Upload Learning Material	47
4.6.12	Create and Assign a Quiz	48
4.6.13	Create and Manage Study Groups	49
4.6.14	Create and Manage Class	50
4.6.15	Start a Learning Session	51
4.7	Hardware and Software Requirements	51
4.7.1	Hardware Requirements	51
4.7.2	Software Requirements	51
4.8	Graphical User Interface	53
4.8.1	Authentication Screens	53
4.8.2	Student Interface	55
4.8.3	Teacher Interface	58
4.9	Database Design	65
4.9.1	ER Diagram	65
4.9.2	Data Dictionary	66
4.10	Risk Analysis	71

4.10.1	AI gives incorrect or misleading explanations	71
4.10.2	Dependence on a third-party language model service	71
4.10.3	Leak or misuse of student data	72
4.10.4	Unsafe or inappropriate AI-generated content	72
4.10.5	Agents fail to coordinate or share learner context correctly	72
4.10.6	Scope growth and schedule delays	73
4.10.7	Students rely on AI and interact less with peers	73
4.10.8	Difficulty finding real users for testing	73
4.10.9	Slow responses under heavy use	74
4.10.10	Inaccurate learner profiles lead to poor personalization	74
4.10.11	Team members unavailable or workload imbalance	74
4.10.12	Loss of code or data due to missing backups	75
5	Proposed Approach and Methodology	76
5.1	Overall Multi-Agent Learning Approach	76
5.2	Learner Context Management	77
5.3	Tutor Agent Explanation Procedure	78
5.4	Adaptive Quiz Difficulty Heuristic	79
5.5	Progress Tracking and Performance Analysis	81
5.5.1	Performance Data Collection	81
5.5.2	Performance Pattern Analysis	82
5.5.3	Recurring Mistake Detection	84
5.6	Teacher Recommendation Methodology	85
5.6.1	Teacher Review	86
5.7	Score-Based Collaborative Group Formation	87
5.7.1	Learner Performance Representation	87
5.7.2	Balanced Complementarity	88
5.7.3	Strength-Based Complementarity	88
5.7.4	Group Formation Procedure	89
5.8	Discussion Hint Generation	90
5.9	Agent Coordination and Information Handoff	91
5.10	End-to-End Learning Workflow	93
5.11	Conclusion	94
6	High-Level and Low-Level Design	96

6.1	System Overview	96
6.1.1	Functional Overview	96
6.1.2	Basic Design Approach	97
6.2	Design Considerations	98
6.2.1	Assumptions and Dependencies	98
6.2.2	General Constraints	99
6.2.3	Goals and Guidelines	99
6.2.4	Development Methods	100
6.3	System Architecture	100
6.3.1	Subsystem Architecture	101
6.3.2	Material Processing Pipeline	104
6.4	Architectural Strategies	104
6.4.1	Shared-State Agent Coordination with LangGraph	105
6.4.2	Single PostgreSQL Database with pgvector	106
6.4.3	Separation of the Web Backend and the Agent Service	106
6.4.4	Teacher-in-the-Loop Control	107
6.4.5	Grounded Generation through Retrieval	108
6.4.6	Error Detection and Recovery for External AI Services	109
6.4.7	Real-Time Communication in the Group Workspace	109
6.4.8	Asynchronous Processing of Long AI Tasks	110
6.4.9	Privacy and Responsible Use of Learner Data	110
6.5	Detailed System Interaction Sequence Diagram	111
6.6	Domain Model/Class Diagram	112
6.6.1	Domain Model	112
6.6.2	Agent Layer Class Diagram	113
6.7	Policies and Tactics	116
6.7.1	Coding Guidelines and Conventions	116
6.7.2	Source Code Organization	116
6.7.3	Version Control and Continuous Integration	116
6.7.4	Database Management	117
6.7.5	Prompt Design and AI Output Validation	117
6.7.6	User Interface	117
6.7.7	Extensibility	118
6.7.8	Engineering Trade-offs	118

6.8 Conclusion	118
--------------------------	-----

List of Figures

2.1 This figure represents the targeted SDG of our FYP (Quality Education)	8
4.1 Login Screen	53
4.2 Forgot Password Screen	54
4.3 Sign Up Screen	54
4.4 Student Dashboard	55
4.5 Student Views his Classes	55
4.6 Student Views Data Structure Class	56
4.7 Adaptive Quiz Screen	56
4.8 Student Views his Progress	57
4.9 Student Portal Settings Page	58
4.10 Teacher Dashboard	59
4.11 Create Class Dialog	59
4.12 Teacher Views Classes	60
4.13 Data Structure Class Overview	61
4.14 Teacher uploads Learning Material	62
4.15 Students Performance Overview	63
4.16 Group Monitoring Screen	64
4.17 AI Insights and Recommendations	64
4.18 ER Diagram	65
5.1 Agent Coordination and Information Handoff	91
6.1 Overall architecture of the InSync system	102
6.2 This figure represents the material processing pipeline of the system	105
6.3 This figure represents the retrieval process of the system	108
6.4 This figure represents the system sequence diagram of the system	111
6.5 This figure represents the Agent Layer Class diagram of the system	114

List of Tables

3.1	Summary of Literature Review	27
4.1	Log In to the System Use Case	37
4.2	Attend an AI Tutoring Session Use Case	38
4.3	Attempt an Adaptive Quiz Use Case	39
4.4	View Personal Progress Use Case	40
4.5	View Feedback and Study Recommendations Use Case	41
4.6	Join a Collaborative Workspace Use Case	42
4.7	Discussion in a Group Chat Use Case	43
4.8	Receive AI Guided Hints in a Group Use Case	44
4.9	View Class Dashboard Use Case	45
4.10	Review an AI Recommendation Use Case	46
4.11	Upload Learning Material Use Case	47
4.12	Create and Assign a Quiz Use Case	48
4.13	Create and Manage Study Groups Use Case	49
4.14	Create and Manage Class Use Case	50
4.15	Start a Learning Session Use Case	51
4.16	Software Technologies Used in InSync	52
4.17	Data dictionary of the InSync database	66
5.1	Teacher Assistant Agent Recommendations	85
6.1	Domain Model Class Responsibilities	112
6.2	Agent Layer Class Responsibilities	114

Chapter 1 Introduction

The application of artificial intelligence in the educational process is getting increasingly widespread and students currently have access to a variety of systems that are capable of tutoring them, assessing their knowledge, tracking their progress and helping them during collaboration with other students. All of the aforementioned systems may be valuable in their individual capacity; however, most of them operate independently from each other.

InSync will be an integrated learning system that will integrate all the above mentioned elements. It will not regard tutoring, assessment, tracking of progress and collaboration as separate systems, but will connect them using a team of synchronized AI agents that will have an up to date knowledge about the user's current level of skills.

1.1 Purpose of this Document

This document presents the research, planning, and design work behind InSync, a learning platform that brings tutoring, assessment, progress tracking, and peer collaboration together into one connected process. It explains the background of the problem that InSync is trying to solve, the study that was carried out to learn about existing tools and research in this area, and the approach that the team intends to follow while building InSync.

InSync is built around a team of five AI agents that share information through a common learner context instead of working alone. When a learner shows difficulty in one activity, such as a quiz, InSync can use that information to adjust the tutoring content, change the difficulty of assessments, and show new recommendations to teachers, all while keeping teachers informed and in control. The main questions that InSync tries to answer are whether a set of coordinated AI agents that share an understanding of a learner can give more useful and timely support than a set of separate, disconnected AI tools, and whether this coordinated approach can be practically designed and evaluated within the scope of a final year project.

This report presents the background research, the problem that InSync is addressing, the requirements of the system, and the design and approach that are planned for building InSync, along with the reasoning behind the decisions that were made along the way.

1.2 Intended Audience

The intended audience for this project includes students and teachers who will use the InSync platform, as well as the FYP panel and project advisor evaluating the project. Students will interact with the Tutor, Quiz, and Discussion Agents for personalized learning support, assessment, and collaborative learning. Teachers will use the dashboard to review student progress, to review the recommendations generated by the Teacher Assistant Agent, and to approve or edit the study groups proposed by the Teacher Assistant Agent.

1.3 Definitions, Acronyms, and Abbreviations

This section lists the important definitions, acronyms, and abbreviations used in this document so that the report can be read without referring to outside sources.

- **AI:** Artificial Intelligence. The field concerned with building systems that can perform tasks that normally require human intelligence.
- **LLM:** Large Language Model. A model trained on large amounts of text that can understand and generate natural language.
- **Agent:** A software component that uses a large language model to carry out one specific responsibility, such as tutoring or quiz generation.
- **Shared Learner Context:** A common record of each learner, containing quiz results, weak concepts, and recurring mistakes, which all five agents read and update.
- **Teacher in the Loop:** A design approach in which the system only suggests actions, and the teacher reviews and approves them before they are applied.
- **SDG:** Sustainable Development Goal. A goal set by the United Nations. This project relates to SDG 4, Quality Education.
- **FYP:** Final Year Project.
- **API:** Application Programming Interface. A set of rules that allows different software systems to communicate with each other.
- **REST:** Representational State Transfer. An architectural style in which a client communicates with a server through standard web requests.
- **ORM:** Object Relational Mapper. A tool that lets a program work with database tables as program objects.

- ER: Entity Relationship. A diagram that shows the tables of a database and how they are related.
- Scrum: An agile development method in which work is divided into short sprints.
- FastAPI: A Python web framework used to build the backend of InSync.
- LangGraph: A framework used to coordinate the agents through a shared state.
- Supabase: A platform that provides the managed PostgreSQL database, authentication and file storage used by InSync.
- pgvector: A PostgreSQL extension used to store vector embeddings for retrieval.
- Embedding: A numerical representation of text that is used to find learning material related to a question.

1.4 Conclusion

This report describes a framework for a coherent system for tutoring, assessment, progress monitoring, and peer discussion, which are interconnected processes dependent on each other. The report discusses the rationale behind choosing this particular problem scope, the intended limits of the project and the decisions regarding the research and design of this solution. The main goal of this project is to enable support of learners based on their recent activity as well as to facilitate better insights for teachers regarding how to provide necessary help. Teacher review remains central to decisions based on the system's recommendations. The following chapter explains the project vision, the people it serves, and the practical limits within which the platform will be developed.

Chapter 2 Project Vision

An understanding of the project problem area will be given in this chapter alongside the project statement and elaboration, the project objectives and the project scope.

2.1 Problem Domain Overview

Artificial intelligence has started to be used in education for purposes of tutoring, testing, monitoring progress, and collaboration with other peers. Currently, several digital services offer all these four features separately, giving students access to tools that can offer personalized tutoring, test them, monitor their progress and assist them to collaborate with their peers.

The problem with currently existing services is that these four features exist separately and do not create an integrated process. For example, when a student faces difficulties understanding something while taking a test, no information is automatically shared with the tutoring tool that could change the level of complexity of further teaching. In a classroom with many students, both teachers and students need to find out these problems and create the connections between them.

This problem is what InSync attempts to solve. Instead of creating yet another standalone tutoring system or another standalone assessment system, the idea behind the project is to create a platform where these four learning functions are supported by a team of five artificial intelligence agents that work together. That means that any information gained by one of the components, such as a learning problem faced by the student, could immediately affect another part of the system, such as the next tutoring material or recommendation made to the teacher.

2.2 Problem Statement

Personalized tutoring, adaptive assessment, progress tracking, and peer collaboration exist in digital learning environments; however, they typically work independently of each other. Whenever a learner faces difficulty in an activity, there is no automatic transmission of such data to the learning aids designed to help them in future learning processes. In this case, the learner will be receiving personalized material that does not take into account the latest difficulties he or she is facing, and teachers will not have timely knowledge regarding which learners need help and why they need it.

It is impossible for teachers to monitor and change anything concerning each learner in a large class. InSync overcomes this problem through the incorporation of the four processes of tutoring, assessment, progress tracking, and peer collaboration into a single continuous communicative process. This process

is based on an AI team. The data regarding a difficulty a learner faces in an activity can be used to customize future tutoring material, set the difficulty level of the next assessment, and make teacher recommendations regarding certain actions to take.

Through solving this issue, InSync aims at improving the timing of educational assistance, the coordination of activities, as well as making the assistance more personalized, rather than spreading the efforts through non-integrating platforms.

2.3 Problem Elaboration

The system design suggested here solves the identified problem through a team of five interacting artificial intelligence agents, each designed for a particular part of the learning process, together with a teacher dashboard. The agents share the same representation of the learner. The design includes the following elements:

- Tutor Agent, which adjusts the level and complexity of explanations according to the learner's needs;
- Quiz Agent, which creates quizzes and adjusts the difficulty of questions based on learner performance;
- Progress Agent, which analyses performance data and identifies weak concepts, trends and recurring mistakes;
- Teacher Assistant Agent, which turns the analysis of the Progress Agent into recommendations for the teacher, and forms study groups on the teacher's request, both for the teacher to review;
- Discussion Agent, which supports group discussion by giving topic-specific hints without revealing answers;
- Teacher dashboard, which keeps teachers updated about the system's recommendations and proposed groups.

These agents work through a shared learner context, which guarantees that any information produced by any particular process whether it is tutoring, testing, monitoring, or collaboration changes the joint representation of the learner and then affects the rest of the components. For example, a poor quiz score in relation to a certain concept means that this concept is updated in the shared learner context. As a result, the Tutor Agent offers additional tutoring for that concept, the Teacher Assistant Agent takes the weakness into account when proposing study groups, and the dashboard highlights the concept for further instruction.

In compliance with the scope of the project, learner profiling in this system is based solely on the information collected from within the platform, such as test scores and learner history. The collaborative workspace offers text discussion and artificial intelligence-driven hints for small group problem solving instead of offering general communications. The exact number and specialization of agents can be adjusted during development.

2.4 Goals and Objectives

The main goal of InSync is to develop a multi-agent AI platform that supports personalized and collaborative learning by coordinating different AI agents.

The objectives of the project are :

- To develop a learner context and coordination mechanism that allows the agents to use relevant learner information when providing learning support.
- To provide personalized tutoring and adaptive assessment based on each learner's performance and learning needs.
- To track learner progress using quiz results and learning activity and identify important learning patterns.
- To provide teachers with progress insights and recommendations through a dashboard, while keeping the teacher involved in reviewing these recommendations.
- To support collaborative learning by proposing small student groups based on their strengths and weaknesses, for the teacher to approve or edit, and by providing topic-related support during discussions.
- To identify and design a reusable agent coordination structure that is separate from subject-specific content, so it can be extended to other subjects in the future.

2.5 Project Scope

The scope of this project is to design and develop a web-based multi-agent AI learning platform called "InSync" which provides personalized and collaborative learning for the Programming Fundamentals subject. The system includes five AI agents: Tutor, Quiz, Progress, Teacher Assistant and Discussion. In this system, the teacher starts a learning session where learners learn the selected topic. The Quiz Agent generates topic-based quizzes and controls the difficulty level of future practice questions based on learner performance. The Tutor Agent answers concept-related questions if the teacher is not available.

The Progress Agent monitors the learner's performance and identifies repeated errors. The Teacher Assistant Agent turns this analysis into recommendations that the teacher reviews through the dashboard. When the teacher requests study groups, the Teacher Assistant Agent forms small groups based on the learners' strengths and weaknesses, and the teacher approves or edits them. The Discussion Agent then gives hints on the topic to be discussed during group work.

The system will have FastAPI to implement the backend services, and LangGraph will be implemented to coordinate the interaction between the agents. Large language model APIs will be used to generate tutoring explanations, quiz questions, and discussion hints. Supabase will be used to store learner, quiz, session, progress, and discussion data.

Main deliverables of this project include the development of the web-based platform including five agents for Programming Fundamentals subject, the teacher dashboard to monitor the learner progress and recommendations, and documentation of the system design and implementation and evaluation.

The system will be tested using sample learning sessions and generated data within the platform for Programming Fundamentals. The scope does not include integration with the university or third-party information systems, mobile application development, student monitoring based on emotions and sensors, any fully autonomous activities without teacher confirmation, and cross-subject testing.

2.6 Sustainable Development Goal (SDG)

This project mainly relates to Sustainable Development Goal 4: Quality Education, which focuses on providing inclusive and equitable quality education and promoting learning opportunities for everyone. It also relates to Target 4.5, which focuses on reducing gaps in education and ensuring equal access to quality education for learners. In a typical classroom, a teacher can't keep track of how well each student understands a topic, which can make it difficult to notice learning gaps early.

InSync tries to help with this through its five AI agents: the Quiz Agent adjusts the difficulty of questions based on student performance, the Progress Agent looks for performance patterns, and the Teacher Assistant Agent turns these patterns into recommendations for the teacher to review. Through these features, the platform aims to provide students with learning support based on their individual needs while helping teachers identify areas requiring attention. Figure 2.1.

2.7 Constraints

InSync will be implemented as a final year project; thus, it is necessary to keep implementation and evaluation feasible given the time constraints, computing capabilities, and participant availability. The



Figure 2.1: This figure represents the targeted SDG of our FYP (Quality Education)

first version of InSync will be tested only on Programming Fundamentals subject. Although the coordination process will be developed in such a way which will ensure that the framework remains flexible enough to deal with various subjects, the project should not claim the general applicability of the system to other subjects.

Tutoring explanations, quiz questions and discussion prompts will be partly generated using external services providing a large language model. External LLM services can cause issues related to additional cost, delayed response time, unavailability or output inaccuracies. To avoid such issues the development of InSync requires setting clear limits on what agents may generate, a way to review important outputs, and a means of handling service failures without losing learner records. Since the chosen topic domain requires suitable learning materials and quizzes, such materials need to be developed prior to evaluation of LLM generated content.

Personalization will rely on collecting data regarding how a learner interacts with InSync specifically including his scores on various quizzes. Learners with limited interaction data available may be provided with less accurate recommendations during their early use of InSync. Indicators of progress should demonstrate only performance inside the platform and not serve as an ultimate measure of a student's skills. Only those people and systems that require access to personal information about the learners should have it and teachers should have the ability to review, edit or dismiss recommendations before consequential action is taken.

The project excludes mobile application development, integration with external academic systems, sensor or emotion monitoring, and evaluation across multiple subjects. These boundaries allow the team to

test the central question: whether coordinated agents can make learning support and teacher information more responsive within a manageable pilot for Programming Fundamentals.

2.8 Business Opportunity

Educators use separate tools for explanation, practice, progress review, and group work and InSync addresses this practical gap. Switching between those tools can make it difficult to see how a learner's difficulty in one activity should affect the next. The use of such a platform would help the teachers identify the particular learning requirements of the learner and take action before the situation loses relevance.

The potential users include schools and universities that want to support learners with different levels of prior knowledge in Programming Fundamentals. In an institutional setting, a teacher could select a topic, review the progress dashboard, and decide whether an agent's suggested intervention is appropriate. Learners could receive targeted practice and take part in small group discussion within the same learning session. The importance of this lies in maintaining the same process, and not just using an AI chatbot.

A future deployment could be offered to educational institutions as a hosted service, subject to evidence that it is useful, affordable, and manageable for teachers. Decisions about pricing, procurement, support, and the first target market require user testing and cost analysis beyond this project's scope. For the final year project, the immediate opportunity is to build and evaluate a credible pilot for Programming Fundamentals: one that demonstrates the connected workflow, records its operating costs, and gathers feedback on whether teachers and learners would choose to use it.

2.9 Stakeholders Description/ User Characteristics

The stakeholders for the system would be the learners and teachers.

2.9.1 Stakeholders Summary

The primary stakeholders are learners and teachers. InSync provides learners with the opportunity to learn Programming Fundamentals, take adaptive quiz questions, request explanation and engage in text discussion forums. Learners could vary in terms of their age, knowledge base, self-confidence as well as familiarity with digital learning applications. The system interface should be designed in such a way that the next step and recommendation would be evident for a learner, without necessarily knowing about the work of the underlying agents.

The teacher controls the learning process, and he/she should get a brief overview of the performance in

the Programming Fundamentals, existing problems and suggestions, with enough information to make a decision regarding the reasonableness of the recommendations. The dashboard will be of great assistance in helping users to make decisions without checking all interactions.

Educational institutions can also benefit from using it. They will be interested in the following things: compatibility with the learners' needs, usability of access to the information, maintenance costs, and amount of time that teachers have to spend working with the system. There are other important stakeholders such as the project supervisor and final year project panel. They will make sure that the problem was identified by the team, the system was implemented, and tested with clear criteria. The development team will have to do the following: implement the agents, coordinate their work, sustain the pilot, and be honest about limitations.

2.9.2 Key High-Level Goals and Problems of Stakeholders

The students require explanations and activities in line with what they have learned recently in Programming Fundamentals. The challenge that learners face is that the help given could remain generic even when a quiz or discussion shows their specific challenge. InSync intends to use these cues in determining the next steps of explanation or activity and giving hints that will aid the group in their logical thinking.

The educators need to identify the gaps in learning early, know the reasoning behind a recommended action, and maintain control over the decision making in the classroom. It is not easy to go through every learner's activity manually, especially in large numbers. The Progress Agent will identify recurrent patterns, and the Teacher Assistant Agent will present them on the dashboard as recommendations while the teacher keeps control.

Educational organizations require a tool for learning that can integrate with their teaching system and operate properly. The concerns raised by such organizations are the reliability of the generated content, appropriate handling of the data of the learners, accessibility, and cost-effectiveness. While a pilot cannot solve all the issues involved with deploying such tools, it can provide evidence about usability, technical performance, and the amount of teacher oversight required.

The project team and evaluators need a demonstrable result within the final year project's limits. Their central concern is whether the five agents actually share useful learner information and whether that coordination produces a coherent learning workflow. Evaluation should therefore examine agent handovers and teacher review as well as the quality of individual responses.

Chapter 3 Literature Review / Related Work

This chapter reviews research and products related to AI-based tutoring and multi-agent learning systems. The review identifies what existing work covers, what it leaves out, and what lessons InSync takes from it.

3.1 Definitions, Acronyms, and Abbreviations

This chapter makes use of several acronyms for ease of reference. To ensure a clear understanding of the terminology used throughout the literature review, a list of these acronyms and their meanings is given below:

- **AI:** Artificial Intelligence. The broad field concerned with building systems that can perform tasks normally requiring human intelligence.
- **LLM:** Large Language Model. A language model trained on large amounts of text, able to generate and understand natural language and hold conversations.
- **ITS:** Intelligent Tutoring System. A software system that provides personalized instruction or feedback to learners without requiring a human teacher.
- **MAS:** Multi Agent System. A system made up of several interacting AI agents that coordinate with one another to complete tasks.
- **API (Application Programming Interface):** a set of specifications that allows communication between different software systems.

3.2 Detailed Literature Review

This section reviews the work of researchers on AI-based tutoring and multi-agent learning systems that is relevant to InSync.

3.2.1 MultiTutor: Collaborative LLM Agents for Multimodal Student Support

MultiTutor [1] is a multi-agent tutoring framework that aims at providing tutoring support according to the requirements of individual learners. Instead of having one AI agent doing all the work, different learning activities are assigned to different agents in the framework. The learning activities include creating explanatory text, visualization, learning resources, practice problems, and simulation. The framework also detects knowledge gaps and uses this information to support learning.

3.2.1.1 Summary

The presented framework has been developed with the purpose of examining possibilities of utilization of multiple AI agents to assist learners. MultiTutor includes multiple expert agents with specific responsibilities assigned to each of them for creation of learning content of different types. Internet searching and code generation have been applied for creation of multimodal outputs in form of images and animations. Authors performed comparison of created material to baseline models using such metrics as cognitive load, readability, depth and diversity.

3.2.1.2 Critical Analysis of the Research Item

One of the main strengths of MultiTutor is the ability to delegate various tutoring tasks to particular agents, thus providing multiple kinds of learning assistance to a student. Knowledge gap detection is quite an important aspect in case of personalized learning as it allows to direct assistance to certain areas of knowledge acquisition. At the same time, MultiTutor is quite limited in its scope and deals only with tutoring, knowledge gap detection and generation of educational content. MultiTutor does not address the problem of various classroom activities such as monitoring of long-term learning process, recommendation of teachers, discussion of groups formation and so forth. Evaluation of MultiTutor is focused on quality and properties of generated material.

3.2.1.3 Relationship to the Proposed Research Work

MultiTutor is similar to InSync in that both use multiple specialized AI agents to provide customized educational support. InSync uses a similar approach with its Tutor and Quiz Agents that use learner performance to provide explanations and practice but InSync also has a Progress Agent, a Teacher Assistant Agent and a Discussion Agent. The Progress Agent and the Teacher Assistant Agent together help the teacher detect frequent performance problems, and the Discussion Agent supports small-group collaborative learning. Therefore, MultiTutor is a useful reference point for the multi-agent tutoring aspect of InSync.

3.2.2 FACET: Multi-Agent AI Supporting Teachers in Scaling Differentiated Learning for Diverse Students

FACET [2] is the teacher-centered framework involving multi-agents AI intended for supporting differentiated learning in classrooms where different learning levels, motivations, and needs of students occur. The framework involves four specialized agents responsible for the learner simulation, diagnosis, material generation, and evaluation. The teacher-in-the-loop paradigm is applied to keep the teacher engaged

into the instructional process without leaving all responsibilities for artificial systems.

3.2.2.1 Summary

The aim of FACET is to help teachers to deliver differentiated learning support for the students with various needs. The framework was designed based on consultations with educational experts such as school principals and teachers. Thirty school principals were involved in the workshops helping to establish the system requirements whereas seventy in-service K-12 teachers evaluated the quality of the generated materials. A mixed-method evaluation method was applied and it demonstrated high perceived value of the system for inclusive differentiation.

3.2.2.2 Critical Analysis of the Research Item

One of the major strengths of FACET is the engagement of the teacher in the learning process. This point is crucial since AI-generated materials and recommendations should be checked by the teacher before their delivery to the students. Another strong point of the framework is the use of specialized agents since each of them will be responsible for its own task. However, FACET is mainly oriented on providing support for the teachers in terms of differentiated instruction and provision of learning materials but not tutoring, adaptive quizzes, progress tracking, and collaboration. The evaluation is mainly conducted based on the quality of generated materials and teachers' perceptions but not on the entire learning trajectory of each student.

3.2.2.3 Relationship to the Proposed Research Work

FACET is similar to InSync in the way that it uses several agents while keeping the teacher engaged into the learning process. InSync implements this idea with the help of the Teacher Assistant Agent and a teacher dashboard, making it possible for the teacher to approve or dismiss any recommendation and to approve or edit any proposed study group. At the same time, InSync extends the approach to student-facing activities by combining tutoring, quizzes, progress tracking, and collaborative discussions. FACET therefore provides a useful reference for the teacher-in-the-loop approach and the use of specialized agents in InSync.

3.2.3 Mapping Student-AI Interaction Dynamics in Multi-Agent Learning Environments: Supporting Personalized Learning and Reducing Performance Gaps

Hao et al. [3] study something that is often missed in multi-agent learning research: how students actually use multiple AI agents during learning and whether their interactions are related to their learning.

Instead of testing only a short interaction, the researchers studied students using MAIC, an online multi-agent learning platform where AI agents have different roles, such as teacher and peer. The study was conducted over a six-module course.

3.2.3.1 Summary

The study comprised 305 university students, who were subjected to analysis of 19,365 interaction data entries obtained as a result of their communication with AI agents. Moreover, researchers gathered pre- and post-test results and measured students' motivation and technology acceptance. Analysis of the data showed that there are two main types of engagement: co-construction of knowledge and co-regulation. In the first case, students and AI agents constructed mutual understanding via dialogic interaction. Co-regulation involved using agents for regulation and management of the learning process. Results of the research show that students with low prior knowledge engaged more in co-construction of knowledge and had better learning outcomes as well as motivation after completion of the course. At the same time, students with high prior knowledge engaged more in co-regulation but achieved fewer improvements in learning outcomes. It means that students with various levels of prior knowledge can have different ways of interacting with the same multi-agent learning environment.

3.2.3.2 Critical Analysis of the Research Item

One of the strong points of the study is the fact that it analyzes interaction of students with multiple AI agents throughout a whole course of six modules rather than their interaction for a short period of time. The study also takes into account not only learning outcomes but also non-cognitive aspects like motivation and technology acceptance. The analysis of interaction patterns of students with different prior knowledge level is also beneficial since it shows that the use of AI can be different among various learners. However, the study is mostly focused on the analysis of the interaction patterns of students with AI agents rather than the whole learning workflow, which includes adaptive quizzes, tracking of progress, recommendations of teachers, etc. The fact that the research was carried out in a single course with six modules limits its generalizability to other disciplines and learning environments.

3.2.3.3 Relationship to the Proposed Research Work

The study provides support for an important concept of InSync: students with different amounts of prior knowledge may need different types of learning support. The finding that students with lower prior knowledge showed greater learning gains and post-course motivation while relying more on co-construction of knowledge is relevant to the Quiz Agent in InSync, which modifies subsequent practice based on student performance rather than giving every student the same level of questions. The study

is also relevant to the Discussion Agent because it shows that the way students interact with learning support matters, not just whether AI support is available. InSync uses student performance information to form small groups with complementary strengths and weaknesses and provides topic-specific hints to support their discussions. The main difference is that this research analyzes student-AI interaction patterns, while InSync uses learner information to support coordination among its different agents and provide information for teacher review through the Progress Agent.

3.2.4 Artificial Intelligence Agents for Personalized Adaptive Learning

Tebourbi et al. [4] propose AIA-PAL, a framework for adaptive learning that combines specialized agents with large language models. The study is relevant to InSync because it treats personalization as a coordinated process involving learner activity, tutoring support, and teacher oversight.

3.2.4.1 Summary

AIA-PAL uses agents for tutor, practical-support and teacher-oriented roles. It is built on top of Lang-Graph for decision making and CrewAI for agent collaboration and it aims to provide real-time monitoring to adapt support as the learner progresses. The framework also uses retrieval-augmented generation with material provided by teachers and structured supervision by humans to reduce the risk of inaccurate or inappropriate responses. Preliminary gains in scaffolding and engagement are reported by the authors, although the published summary does not establish how well those gains would persist across subjects or longer teaching periods.

3.2.4.2 Critical Analysis of the Research Item

AIA-PAL's most important contribution is its attempt to connect adaptation with educational safeguards. The content provided by the teacher is a clear source of the generated responses, and the division of roles of the agents gives a clear account of the way the system selects support. This is helpful for a project where a wrong explanation or quiz item can impact later activities. But the evidence needs to be read carefully. The reported results are preliminary and the framework architecture is more detailed than the evidence available for long-term learning outcomes. It also concentrates on personalized support and not on integrating peer discussion and teacher feedback on progress recommendations into a single workflow.

3.2.4.3 Relationship to the Proposed Research Work

InSync can take advantage of the separation of agent responsibilities and focus on grounding content created by AIA-PAL. Its Tutor, Quiz, Progress, Teacher Assistant and Discussion Agents will share a

learner context, so that evidence from one activity can inform the next. InSync, unlike AIA-PAL, will specifically test the links between assessment, tutoring, progress reporting and small group discussion. The comparison also poses a practical question of evaluation: does the coordinated workflow provide any added value over individually useful AI responses?

3.2.5 A Generative Artificial Intelligence Enhanced Multiagent Approach to Empowering Collaborative Problem Solving Across Different Learning Domains

Zheng, Shi, and Gao [5] examine whether generative AI agents can improve collaborative problem solving. Their study is particularly relevant to InSync's Discussion Agent because it evaluates support given during group activity rather than treating collaboration as a simple messaging feature.

3.2.5.1 Summary

The researchers conducted two empirical studies with a total of 234 college students. They compared collaborative work supported by a generative AI multiagent approach with work supported by a chatbot and with traditional collaboration without that support across different learning tasks. Data were collected through pretests, posttests, records of the collaborative process, and interviews. For the multiagent groups, the authors found stronger results for learning achievement, knowledge elaboration, and collaborative problem-solving performance and skills.

3.2.5.2 Critical Analysis of the Research Item

The study's comparison groups make its findings more informative than an account of a system used in isolation. The combination of performance data and process data also adds to our understanding of what went on while the participants collaborated. With InSync, it is important to distinguish between assisting a group in generating ideas and providing answers. The participants in this study are college students performing the tasks set for the study. It would be erroneous to apply the findings from this study to other types of learners without further research. This study addresses only collaborative problem solving. It does not address the relationship between group assistance, learner history, adaptive testing, and the teacher dashboard.

3.2.5.3 Relationship to the Proposed Research Work

This research supports designing the Discussion Agent around prompts that stimulate explanation and exchange among learners. InSync will use topic-level performance to form small groups and provide text hints during their work. The study suggests evaluating the quality of the discussion itself, including

whether learners explain their reasoning to one another. InSync extends this line of work by connecting discussion activity with tutoring, assessment, and progress information in a single platform.

3.2.6 Multiple Agent Designs in Conversational Intelligent Tutoring Systems

Lippert et al. [6] review designs in which more than one conversational agent participates in an intelligent tutoring environment. The paper offers a useful design perspective on why agents should have distinct educational roles and how their interactions can shape a learner's experience.

3.2.6.1 Summary

The authors describe systems in which agents interact with learners and sometimes with one another to model explanations, strategies, reflection, and social interaction. They draw on examples used in educational and experimental settings across several subjects and skills. Their central contribution is a set of design patterns for assigning pedagogical roles to multiple agents, rather than an evaluation of one new platform.

3.2.6.2 Critical Analysis of the Research Item

It makes it clear that introducing agents alone does not qualify as an instructional design, where each activity and relationship must have an educational objective. The diversity of examples used will aid the decision about whether an agent should instruct, challenge, demonstrate, or cause reflection on the part of the learner.

Since this paper precedes present large language model designs, it deals with conversational agents, which are usually illustrated as graphical characters. Hence, the examples provided cannot be directly applied to the technical design of InSync. It is also true that as a design review, it will not offer one experiment with all the features of InSync, namely, tutoring, quizzing, progress monitoring, and grouping of students.

3.2.6.3 Relationship to the Proposed Research Work

InSync applies the principle of distinct pedagogical roles through five agents, but the important design question is how information passes between them. A quiz result should identify a topic that needs support; the Tutor Agent should be able to respond to that need; and the Teacher Assistant Agent should show the teacher the evidence behind its recommendation. The review helps justify explicit roles and interactions, while InSync's pilot must test whether those roles work together in practice.

3.2.7 Multi Agent Configurations for Social Learning with LLMs

Kumar, Mu, Vincentius, and Anderson [7] consider the possibility that using more than one AI agent as opposed to a single AI tutor may facilitate better learning outcomes. In spite of the fact that the current AI systems in education mostly rely on a one-to-one tutoring system, the researchers consider the possibility that using several AI agents with different behavior patterns will bring the same effect as peer-to-peer learning in the case of human tutors. To test this hypothesis in different learning areas, two controlled experiments are carried out.

3.2.7.1 Summary

In the first experiment, a sample of 315 participants solved SAT-level math problems using the experimental setup whereby an AI tutor and error-making AI peer(s) were either present or absent, with the latter making either conceptual or arithmetic mistakes. In the second experiment, 247 participants drafted argumentative and creative essays using one of three setups, namely, without any AI, with a single LLM, or with two role-separated LLMs (GPT 5.2 and Claude Opus 4.6). In both experiments, the researchers found that multi-agents arrangements can provide some meaningful advantages; however, such advantages were not always consistent and predictable. More precisely, in the second essay drafting experiment, both single-agent and two-agents setups improved the quality of the essays; but only the latter reduced the tendency of convergence in the idea-level content previously demonstrated by the single-agent setup; namely, while the essays written by a single AI agent showed similarity in linguistic and conceptual content, essays written by two different AI agents remained more diverse in terms of ideas.

3.2.7.2 Critical Analysis of the Research Item

One advantage of the current study is the well-controlled comparison study in two different learning environments: mathematical problem-solving and writing an essay, using relatively small participant groups. Moreover, the current study uses contemporary and realistic large language models (LLMs), rather than simplifications and simulations that may affect the result's reliability.

One disadvantage of this research work is the lack of consistency in terms of benefits of multi-agent implementation in two studies. The inconsistency makes it difficult to determine the exact property of the multi-agent system (for instance, agent's roles, number of agents, or type of mistakes done by the agents). The current study analyzes the generic peer configuration of agents, and not the agents who have a definite educational role (tutor, assessor, or progress tracker) like it is suggested in InSync. Besides, in this case, we cannot say that the advantages will last for a whole term, since the study was short and

conducted during a laboratory session.

3.2.7.3 Relationship to the Proposed Research Work

These research results directly relate to the design of the InSync Discussion Agent and the collaborative environment. Indeed, it turns out that just adding more AI agents to an instructional session does not automatically lead to better performance; rather, the process of role specialization is needed for any positive results to emerge. The avoidance of homogeneity of the ideas generated by a single AI agent for the group is of primary importance in this context. Therefore, the InSync design of promoting small group problem solving by generating individual AI hints is justified by these research results.

3.2.8 A Multi Agent Ecosystem for Programming Instruction

The Human AI Collaborative Learning Ecosystem study [8] presents a multi agent based programming learning system, referred to as MA PL, built for programming instruction in higher education. It uses four specialized AI agents that each support a different stage of the learning process, from understanding concepts to completing tasks and collaborating in teams. The study compares this system against a conventional teaching approach to measure its effect on learning outcomes, computational thinking, and student behavior.

3.2.8.1 Summary

MA PL is comprised of four agents: the Knowledge Guide Agent, the Task Support Agent, the Cognitive Assistant Agent, and the Collaboration Facilitator Agent. The researchers carried out a comparative analysis in university programming classes, where MA PL (experimental group) was compared to traditional methods of programming learning (control group). Differences in learning performance and computational thinking skills between the two groups were measured, and, furthermore, modifications in behavior of the learners in programming with multi-agent systems were observed and analyzed. One of the findings was that, owing to the very responsive multi-agent system, learners tended to use interactions with AI agents rather than the process of discussing the ideas with each other. This kind of behavior, which is based on an interaction mode that does not involve any resistance at all, may lead to increased personal efficiency but also may reduce peer discussions.

3.2.8.2 Critical Analysis of the Research Item

As far as the structure is concerned, the present study resembles InSync in terms of considering an authentic multi-agent system where each agent fulfills its own pedagogical role, rather than designing a multi-agent system in the context of some experimental laboratory assignment. As a result, both

theoretical value and practical relevance can be observed here, as well as the real possibility of comparing results in terms of behavioral changes and learning success rather than just opinions of the participants.

At the same time, one of the limitations of the current research is that it is focused only on programming classes, while it remains unclear how far the findings apply to other subjects to which the coordination framework could later be extended. Furthermore, the finding that a highly sensitive multi-agent system may make students prefer interaction with agents rather than with peers should be considered as a warning rather than a triumph.

3.2.8.3 Relationship to the Proposed Research Work

This research is directly applicable to the design of the Discussion Agent in InSync. The finding that a highly responsive multi-agent system may inhibit peer interaction is a design warning for the developers of InSync, because the Discussion Agent is intended to support peer discussion, not replace it. The Discussion Agent is therefore designed to help students engage in discussion and negotiation with each other, with its AI-generated hints serving as an entry point into group discussion and not as a replacement for it.

3.2.9 Khanmigo, an Existing AI Tutoring Product from Khan Academy

Khanmigo is a tutoring and instructional assistance tool based on AI created by Khan Academy that was launched in 2023 in partnership with OpenAI and built using GPT-4 [9]. Rather than giving the answers to students' questions, Khanmigo poses them with guiding questions and helps them in their thought process. It works on top of Khan Academy's preexisting lessons, video content, and exercises and is used by students, teachers, and parents for a variety of subjects such as math, science, programming, and the humanities [10].

3.2.9.1 Summary

For learners, the tutoring provided by Khanmigo is customized according to the pace of the learner with prompts towards a solution and feedback but not the actual solution itself. It further helps them write and engage in debates along with offering real-time feedback on coding. For teachers, it prepares lessons and standards-based material along with rubrics and provides reports on learner activity and progress [11]. For parents, it provides guided homework support for their children [10]. Khan Academy reports that 2.0 million students, educators, and parents used Khanmigo during the 2024–2025 school year [12].

3.2.9.2 Critical Analysis of the Research Item

One of the key strengths of Khanmigo is its current existence as an actual product backed up by a rich library of content already created and used, unlike many other systems described in research papers and prototypes. The system uses an inquiry-based approach in tutoring which does not involve directly providing the answers, just like InSync wants its Tutor agent to do. Moreover, the available teacher reports show that AI tutoring can be combined with clear usage and progress information for teachers [11]. A corresponding weakness of the system is that it was mainly created with the focus on the work of one learner with one AI tutor. It involves no cooperation between multiple learners and there is no special feature that would help in organizing such cooperation or working together in groups. Also, the tutoring, preparation for assessment tasks and reporting features of the system are tied to the content and experience of Khan Academy only.

3.2.9.3 Relationship to the Proposed Research Work

Through its use at a significant scale in practice, Khanmigo shows that two of InSync's core elements work separately: (1) guidance-based AI tutoring that does not give students the answers, and (2) reporting that keeps teachers informed about progress [11, 12]. This supports the relevance of both the tutoring function and the teacher dashboard in InSync. At the same time, Khanmigo has no peer collaboration tools and, as far as the reviewed sources show, no coordination between its tutoring, quiz and reporting features. This is the problem that InSync intends to solve.

3.2.10 AgentTutor: Empowering Personalized Learning with Multi-Turn Interactive Teaching

The study by Liu et al. [13] draws attention to the problem that most LLM-driven systems respond to only one question at a time, so they cannot easily track a learner's progress or adapt to changes. Liu et al. therefore ask whether a multi-turn tutoring system that continuously evaluates the learner and remembers its findings would deliver more personalization. They run a series of evaluations, including a benchmark of programming tasks.

3.2.10.1 Summary

Liu et al. [13] introduce AgentTutor, a multi-turn interactive intelligent education system. The authors state that many LLM-based education systems only answer learners' questions one at a time and cannot assess the learner's cognitive level or change the teaching strategy as feedback arrives. In AgentTutor, LLM-based generative agents work with a learner profile environment in which learners set their goals,

preferences and study materials. The system has five modules: curriculum decomposition, learner assessment, dynamic strategy, teaching reflection, and a knowledge and experience memory. The curriculum is split into sub-goals according to Bloom’s Taxonomy, the learner’s proficiency on each sub-goal is assessed, and the teaching experience is kept in memory for future use.

3.2.10.2 Critical Analysis of the Research Item

The main strength of the proposed AgentTutor approach [13] is that it treats tutoring as a continuous process: it assesses the learner, updates a proficiency score for each sub-goal, and keeps a long-term learner record. This matches InSync’s idea of tracking a student’s development instead of answering questions separately. The authors also mention the limitations of the approach. The evaluation relies on simulated learners and programming benchmarks, and there is little validation with real learners. Personalization depends only on the learner’s cognitive level and progress score. The system also follows a one-learner-one-tutor design, with no teacher view and no peer interaction.

3.2.10.3 Relationship to the Proposed Research Work

The AgentTutor system is consistent with the choice made by InSync to store a student’s progress from tutoring sessions. Similarly to AgentTutor, InSync keeps a simple record of each tutoring exchange: the concept, the explanation level, repeated confusion and the check question result. The Progress Agent reads this record and passes its analysis to the Teacher Assistant Agent. InSync also adds a Discussion Agent for group work, which AgentTutor does not have. The limits of simulated-learner evaluation also support testing InSync with real students.

3.2.11 EducationQ: Evaluating LLMs’ Teaching Capabilities Through Multi-Agent Dialogue

Shi, Liang, and Xu [14] argue that current benchmarks for language models focus on knowledge and reasoning, not on whether the models can teach. To address this, the authors measure the ability of a language model to teach through a simulated interaction between a teacher, a student and an evaluator, each played by an agent.

3.2.11.1 Summary

Shi et al. [14] introduce EducationQ, a multi-agent dialogue environment designed to assess the ability of language models to teach. It uses a dialogue between a teacher agent, a student agent and an evaluator agent, in which the teacher carries out informal formative assessment by checking the student’s knowledge and giving feedback. Using a carefully selected pool of 1,498 questions of different subjects and

difficulty levels, the authors tested 14 state-of-the-art models with a pre-test and post-test design. The results contradict the common belief that larger or commercial models teach better: the most effective models were some smaller open-source ones.

3.2.11.2 Critical Analysis of the Research Item

The strength of the EducationQ framework [14] is that it evaluates teaching directly based on the changes in learning from a pre-test to a post-test, in addition to an external evaluator, rather than depending on general knowledge benchmarks. Its limitation for InSync is that it evaluates teaching ability and relies on a simulated student agent. It shows what a model can do when teaching, but does not evaluate a tutoring system used in a real classroom.

3.2.11.3 Relationship to the Proposed Research Work

EducationQ helps in checking how well the InSync Tutor Agent teaches and how much students learn, not only whether its answers are right. It also suggests that the model chosen for the Tutor Agent should be tested on how it teaches, for example by asking questions and giving feedback. Its evaluator agent also supports the idea of reviewing the output of an agent, which InSync does through output validation for Tutor Agent explanations.

3.2.12 LLM Agents for Education: Advances and Applications

Chu et al. [15] take a broad view of LLM agents in education instead of testing one particular system. Their paper reviews recent research on the subject, classifies LLM agents by the roles they play, and describes their underlying technology.

3.2.12.1 Summary

Chu et al. [15] provide a systematic survey of LLM agents in education, published in Findings of EMNLP 2025. Their survey categorizes the agents into pedagogical agents that automate difficult teaching activities to aid teachers and learners (such as generating feedback and designing curriculum), and domain-specific educational agents in fields like science, language learning, and professional development. The survey provides an analysis of datasets, benchmark tests, and algorithmic framework used in LLM-based educational agents and addresses the deployment challenges, which include privacy, bias and fairness, hallucinations, over-reliance, and integration.

3.2.12.2 Critical Analysis of the Research Item

As a survey [15], its strength lies in its breadth. It offers a map of the field, a helpful vocabulary for describing agent roles, and a well-organized list of deployment risks. Because it is a review, it does not run any experiments or implement any workflows. Its coverage remains general so it cannot confirm whether a specific mix of tutoring, progress tracking, teacher support and peer discussion actually works in classroom settings.

3.2.12.3 Relationship to the Proposed Research Work

The survey places InSync in a recognized category: agents that support both students and teachers. It also reinforces the idea that privacy, hallucination and over-reliance must be treated as design concerns from the start. It points to the gap that InSync addresses: combining several agent roles in one classroom workflow.

3.2.13 ALIGNAgent: Adaptive Learner Intelligence for Gap Identification and Next-Step Guidance

Tokoli, Jaimes, and Dina [16] observe that personalized learning systems usually address only one of three tasks: knowledge tracing, diagnosis or resource recommendation.

3.2.13.1 Summary

The work by Tokoli et al. [16] introduces ALIGNAgent, which integrates knowledge estimation, skill gap detection and resource recommendations in a unified agent architecture. It is observed by the authors that existing personalized learning systems focus on either knowledge tracing, diagnosis, or recommendation, but not all three at once. In ALIGNAgent, quiz results, gradebook information and learner preferences are used to compute estimates of topic proficiency. A Skill Gap Agent uses diagnostic reasoning at the concept level to detect skills gaps, and resource recommendation is performed for further action. On real data collected from two undergraduate computer science classes, the authors report 0.87 to 0.90 precision and F1 of 0.84 to 0.87 for proficiency estimation.

3.2.13.2 Critical Analysis of the Research Item

A clear strength of ALIGNAgent [16] is that the estimates are checked against exam results in real courses and that the system turns learner data into clear next steps. The work is an arXiv preprint that looks at two computer science courses. It uses quiz and gradebook data that a classroom already has. It focuses on individual learner gaps, with no live tutoring conversation and no support for students

working together.

3.2.13.3 Relationship to the Proposed Research Work

ALIGNAgent is similar to InSync's Progress Agent, because both take learner activity and build a picture of where a student is strong or weak. ALIGNAgent also recommends next resources. In InSync this is done by the Teacher Assistant Agent, so that the teacher can review the advice. The work supports the idea of sharing this picture so that teachers and students can act on it. It also gives a model for checking progress estimates against later results, which in InSync would use the quiz results recorded inside the platform.

3.2.14 LecturaAgents: A Multi-Agent Framework for Adaptive Personalized AI-Assisted Learning and Embodied Teaching

In their paper, Sesay et al. [17] address how artificial intelligence may be used not only to tailor lectures for individual students but also to deliver them in a way that suits teaching, using speech and teaching actions. The approach suggested involves a group of agents specializing in certain aspects of lecture preparation and delivery with a professorial agent taking the lead. It was implemented and evaluated for courses at school, undergraduate and graduate level, and a student experiment shows some effects on learning.

3.2.14.1 Summary

Sesay et al. [17] introduce LecturaAgents, a multi-agent system for creating and delivering personalized lectures from start to finish. A ProfessorAgent leads a group of agents that handle research, planning, evaluation and delivery of teaching content, and the content changes according to learner profiles. The ProfessorAgent also performs teaching actions such as highlighting and underlining, and a Teaching Action-Speech Alignment algorithm matches these actions to speech. The framework was tested on courses at school, undergraduate and graduate level. Expert educators checked the materials and teaching actions. A student efficacy study shows early signs of better learning results.

3.2.14.2 Critical Analysis of the Research Item

The strong points of the research [17] include clear coordination, where an agent controls specialists; the consideration of the way content is delivered and not only produced; and its analysis at different educational levels with the help of experts. The efficacy evidence concerning the effect on the students is described by the authors as preliminary, and the system concentrates on lecture generation and delivery. Peer collaboration and teacher review of the progress of each student's progress are not considered.

3.2.14.3 Relationship to the Proposed Research Work

LecturaAgents is consistent with InSync in dividing the work between specialized agents and adapting content to learner profiles. It differs in coordination. LecturaAgents uses a lead ProfessorAgent that manages the others, while InSync has no lead agent and uses a LangGraph router with a shared learner context. It also differs in purpose, because it focuses on lectures, while InSync focuses on tutoring, quizzes, progress analysis, teacher recommendations and group discussion.

3.2.15 Impact of AI-Agent-Supported Collaborative Learning on University Programming Courses

Wang et al. [18] examine whether adding an AI agent to student groups in university-level programming classes can support learning beyond what traditional computer-supported collaborative learning (CSCL) offers. Specifically, the research question is whether an AI agent placed inside a student group benefits the group's learning. To answer it, they conduct a six-week quasi-experiment that compares groups supported by the AI agent with groups that use traditional CSCL without an AI agent.

3.2.15.1 Summary

Wang et al. [18] test an AI agent that supports student groups in university programming courses. They used a quasi-experimental design in which 45 students were randomly assigned to an AI-agent-supported group or a traditional computer-supported collaborative learning group. The study took place while students were training for ACM programming competitions. It lasted six weeks, with sessions of 70 minutes. Students in the AI-supported groups showed better learning performance, more interest in learning and more confidence in their own abilities (self-efficacy). They also reported lower mental effort (cognitive load). The researchers explain these results with cognitive load theory and self-efficacy theory.

3.2.15.2 Critical Analysis of the Research Item

A strength of this study [18] is that it does not merely measure students' grades but also considers interest, self-efficacy, cognitive load, and a control group. The study supports the effectiveness of AI support inside student groups. It does not compare this with AI support for individual students. However, it covered one programming context over six weeks, and the AI helped only when students asked for it. It therefore says nothing about AI that also acts on its own, or about reports for teachers.

3.2.15.3 Relationship to the Proposed Research Work

This study supports the idea that AI support inside student groups can help learning, which is the purpose of the InSync Discussion Agent. It also suggests that the evaluation of InSync should measure interest, self-efficacy and cognitive load in addition to performance. In the study the AI helped only when students asked for it. The InSync hint button works in the same way, but the Discussion Agent can also give a limited number of hints on its own (Section 5.9), and this study does not test that. The study also does not cover group formation, which InSync handles with the Teacher Assistant Agent and teacher approval.

3.3 Literature Review Summary Table

This section summarizes the reviewed research works in one table.

Table 3.1: Summary of Literature Review

Research work	Main contribution	Limitation most relevant to InSync	Design lesson for InSync
MultiTutor [1]	Uses specialized agents to create varied tutoring resources and identify knowledge gaps.	Concentrates on tutoring outputs rather than a full classroom workflow.	Connect personalized explanations and practice with later progress information.
FACET [2]	Uses specialized agents and teacher involvement to support differentiated learning.	Primarily evaluates teacher-oriented material generation and perceptions.	Keep teachers able to review and change suggested actions.
Hao et al. [3]	Relates patterns of student interaction with multiple agents to learning outcomes.	Examines one course setting and does not implement the proposed combined workflow.	Consider how learners use agent support, not only whether agents produce responses.

Continued on next page

Table 3.1: Summary of Literature Review (continued)

Research work	Main contribution	Limitation most relevant to InSync	Design lesson for InSync
AIA-PAL [4]	Proposes coordinated adaptive support grounded in teacher-provided material.	Reports preliminary findings and gives limited evidence about long-term outcomes.	Define agent responsibilities and check the accuracy of generated content.
Zheng et al. [5]	Compares multiagent group support with chatbot and unsupported collaboration.	Uses selected tasks with college students; transfer to other settings remains uncertain.	Write hints that prompt learner discussion, and evaluate the discussion process.
Lippert et al. [6]	Reviews educational roles and interaction patterns for multiple conversational agents.	Is a design review of earlier tutoring systems, not a test of a current LLM platform.	Give each agent a clear educational purpose and a defined interaction with the others.
Kumar et al. [7]	Tests how configurations of AI tutors and peers affect mathematics and writing tasks.	Benefits vary by task and configuration in short experimental settings.	Test whether each additional agent role improves the learning activity it supports.
MA-PL [8]	Studies specialized agents in programming learning and observes changes in learner behavior.	Findings come from programming education; responsive AI may draw attention away from peers.	Use the Discussion Agent to support conversation between learners.
Khanmigo [9]	Demonstrates guided AI tutoring and teacher-facing support in an operational product.	Does not provide the proposed integrated student grouping and five-agent workflow.	Combine guided help with useful information for teachers while testing the added value of coordination.

Continued on next page

Table 3.1: Summary of Literature Review (continued)

Research work	Main contribution	Limitation most relevant to InSync	Design lesson for InSync
AgentTutor [13]	Multi-turn tutoring system with five modules (curriculum decomposition, learner assessment, dynamic strategy, teaching reflection, memory) and a learner profile environment.	Is evaluated mainly with simulated learners and programming benchmarks, so validation with real learners is limited.	Keep assessing the learner and store a long-term learner record to adapt support.
EducationQ [14]	Uses teacher, student and evaluator agents to measure the teaching ability of 14 LLMs with pre-tests and post-tests on 1,498 questions.	Evaluates models with a simulated student, not a tutoring system used in a real classroom.	Evaluate the Tutor Agent on learning change and teaching quality, not only correctness.
Chu et al. [15]	Surveys LLM agents in education, grouped into pedagogical and domain-specific agents, with their deployment challenges.	Is a review, so it does not implement or test any specific multi-agent classroom workflow.	Treat privacy, hallucination and over-reliance as design concerns from the start.
ALIGNAgent [16]	Turns quiz and gradebook data into topic-level proficiency estimates, identifies skill gaps and recommends resources in one multi-agent framework.	Is a preprint covering two computer science courses, and focuses on individual gaps, with no live tutoring or peer collaboration.	Check progress estimates against later quiz results and pass them on for action.
LecturaAgents [17]	Uses a hierarchical agent framework led by a professor agent for personalized lecture generation and embodied delivery.	Reports preliminary student efficacy evidence, and focuses on lecture delivery, not progress tracking or peer discussion.	Coordinate agents with clear roles and adapt content to learner profiles.

Continued on next page

Table 3.1: Summary of Literature Review (continued)

Research work	Main contribution	Limitation most relevant to InSync	Design lesson for InSync
Wang et al. [18]	Compares AI-agent-supported collaborative learning in a university programming course with a traditional collaborative learning control group in a quasi-experiment.	Covers one programming context over six weeks, and the AI acts only on explicit student requests.	Measure interest, self-efficacy and cognitive load as well as performance.

3.4 Conclusion

From the literature review, we can see that AI agents are able to facilitate several aspects of the learning process, such as tutoring, assessment, progress monitoring, teacher aid, and teamwork. Also, we can conclude that assigning a specific role to an agent enhances its performance. Nevertheless, almost all studies pay attention to just one or two of these aspects, mostly using simulations or short-term experiments and hardly connecting them into one learning process. Moreover, the studies highlight the importance of using AI to assist the teachers and students in their interactions, not to substitute them. Besides, accuracy, privacy, and over-reliance issues should be considered. Taking all these factors into account, InSync combines all mentioned agents in one system. They cooperate on the basis of each learner's performance data, and the teacher controls the recommendation process.

Chapter 4 Software Requirement Specifications

This chapter presents the software requirements of InSync, a multi-agent AI platform designed to support personalized and collaborative learning. It describes the main features of the platform, functional and non-functional requirements, quality attributes, assumptions, use cases, hardware and software requirements, graphical user interface, database design, and project risks. The requirements focus on the interactions between students, teachers, and AI agents: Tutor Agent, Teacher Assistant Agent, Quiz Agent, Progress Agent, and Discussion Agent.

4.1 List of Features

The following features will be available in InSync to support students learning, help teachers manage and assess their classes, and enable users to interact with the AI agents and with each other.

- The Tutor Agent helps learners understand concepts, answer their questions, and adjust the explanation pace and depth depending on the level of a particular learner's knowledge.
- Adaptive assessments are quizzes that adjust themselves according to the level of a particular learner's performance; therefore, all learners are assessed according to their abilities.
- The shared learner context is the background profile of a learner that changes as tutoring, quizzes, and group work take place.
- Individualized progress and feedback allow each learner to see his or her performance, progress, strengths, and areas that may require improvement.
- Teachers can start a learning session and select the topic that students will study and practice.
- The Progress agent tracks each student's learning progress based on their performance in quizzes. It stores this progress data and makes this information available to the Teacher Assistant Agent.
- The Teacher Assistant agent provides progress-based recommendations for the teacher to review through the dashboard, where the teacher can approve or dismiss the recommendation.
- The Teacher Assistant agent forms small groups on the teacher's request, using learner performance and complementary strengths and weaknesses. The teacher can approve or edit the proposed groups before they are made available to students.
- Students can use a text-based workspace to discuss a topic and solve problems together after being placed in a group.
- The Discussion Agent provides topic-specific hints during group work to help students continue

their discussion without giving them the answer.

4.2 Functional Requirements

The functional requirements below describe the services and actions that InSync must provide to its users. They are grouped by the two user groups of the system: students and teachers.

4.2.1 Student Functional Requirements

- The system shall allow students to log in to their accounts and view their personal dashboards.
- The system will provide a student with quizzes approved by the teacher, in which the difficulty of the questions adjusts automatically according to the student's performance.
- The system will automatically update the learner profile of a student after the completion of tutoring sessions and quizzes.
- The system will allow a student to view the performance and progress he made during the period of time, including their strengths, weaknesses and suggestions for further study.
- The system will provide a student with the ability to initiate a tutoring session with the AI tutor on any supported topic.
- The system will allow a student to enter a collaborative workspace along with his group members and will provide AI-generated hints to the group while solving problems.

4.2.2 Teacher Functional Requirements

- The system will provide a teacher with the ability to log in to his/her account and view the class dashboard showing all enrolled students.
- The system shall allow teachers to upload learning material, start learning sessions, and select the topic to be covered.
- The system will allow a teacher to view each student's quiz performance and progress over a period of time.
- The system will present a teacher with performance patterns and recommendations generated from learner's data.
- The system will allow a teacher to approve or dismiss recommendations provided by the Teacher Assistant agent.

- The system will allow a teacher to end a learning session, which will trigger the Quiz Agent to generate a draft quiz on the topic covered, for the teacher to review and approve before it is made available to students.
- The system will allow teachers to request small study group formation and review, approve, or edit the groups proposed by the Teacher Assistant agent.

4.3 Quality Attributes

The following quality attributes describe how well InSync should perform its functions. They were chosen because they matter most for an AI-based learning platform.

4.3.1 Usability

It is imperative that InSync is intuitive and user-friendly for both students and teachers, given that neither is likely to possess any background knowledge in the use of AI-based learning systems. The interface needs to be intuitive with clear labels, AI explanations should be in plain language terms, and a new user should be able to start a tutoring session, take a quiz, or check the class dashboard without the need for a user manual. Ease of use can be determined through asking a few students and teachers to perform a series of tasks such as starting a tutoring session and reviewing a recommendation, measuring the time taken to do so and where users experience difficulty.

4.3.2 Reliability

InSync should operate consistently and accurately, even in cases where an individual AI component operates poorly or fails to produce an appropriate output. If the tutoring component fails to respond, there should be a clear indication of the problem instead of the whole application crashing and becoming non-responsive, and other functions like reviewing progress history should be accessible. This quality can be verified by purposely testing the reliability of the system using a slow or unresponsive AI service.

4.3.3 Performance Efficiency

For InSync, performance efficiency will be important for providing a smooth experience for students and teachers. The platform should display feedback within a reasonable time during activities such as quiz generation, AI-based learning support, and loading the teacher dashboard. Learner progress and recommendations should be displayed without unnecessary delay. The response time of these activities will be checked during system testing.

4.3.4 Maintainability

InSync will focus on easy maintainability by using a modular structure for its agents and backend. Each component will have a clear responsibility so that changes or fixes can be made without affecting the whole system. This structure will also make it easier to update individual components and add improvements during the development of the platform.

4.3.5 Flexibility

InSync would remain scalable and extensible in relation to new subject areas, new types of learning activities, additional AI agents and collaborative functionality, which will not necessarily require extensive changes to the entire architecture. Instead, the InSync platform would implement modularity in the architecture in which every AI agent and component of learning would perform specific tasks according to specified boundaries. Thus, any added specialized learning agent or type of activity could fit into existing architectures easily.

4.4 Non-Functional Requirements

The non-functional requirements below set the limits and conditions under which InSync must operate, such as security, privacy, availability, compatibility, accessibility and the safe use of AI.

4.4.1 Security

It is necessary for InSync to secure user accounts and personal data via login security and to limit the access that can be granted to a particular user depending on their roles as students or teachers. For instance, a student is not supposed to have access to another student's private performance data or teacher's class preferences. Passwords need to be kept safe while all communications between the application and the outside AI system need to be conducted without compromising user information.

4.4.2 Data Privacy

InSync gathers and stores the learners' data including test results and interaction history. The information is also provided to an external AI system which generates the customized content. Therefore, InSync needs to disclose to the users what data is collected by the system, what it is used for, and why it is collected. It is important that only such information as is necessary to get the needed answer is provided to the AI system. The learners and teachers should also be clearly informed about it.

4.4.3 Availability

InSync should be available whenever students and teachers are using the platform. If an AI service or system component becomes unavailable, the system should show a clear message instead of stopping the entire platform.

4.4.4 Compatibility

InSync should work properly on commonly used modern web browsers and on laptop and desktop devices. Supported browsers should have the main features of the platform without major differences in functionality.

4.4.5 Accessibility

InSync should provide an intuitive user interface for people with various accessibility needs. It means that text should be readable, there should be meaningful labels, navigation should be uniform across the entire interface, contrast should be sufficient and feedback understandable. No crucial information should depend solely on colors. Additionally, interactive objects should stand out clearly to ensure usability for everybody. Finally, group collaboration in InSync is text based, so users do not need to rely on audio.

4.4.6 AI Transparency and Safety

InSync needs to provide distinctions between AI recommendations, feedback, and suggestions and those offered by teachers or students. Whenever a difficulty of learning or a recommendation towards teacher involvement occurs, there needs to be relevant information accompanying these events. Information like scores achieved in quizzes, repetitive challenges during tasks, or participation in collaborative activities should provide context to an AI suggestion. There should be a distinction between AI recommendations as an aid and a conclusive decision. Teachers should have complete authority over critical operations including the allocation of assessments, creation or modification of learning groups as well as intervention during a student's learning process. To avoid the possibility of harmful or illegal activities performed through the system the validation of AI output should occur whenever necessary and the AI agents restricted to performing their respective educational functions. Students should not be using AI agents to carry out activities outside of educational purposes nor should they obtain any private information owned by any user. If there isn't enough evidence to formulate conclusions then it must be made clear that no conclusive statements have been reached.

4.5 Assumptions

The following assumptions are taken for the development and evaluation of InSync.

- The students and teachers have an established internet connection and access to an electronic device with an up-to-date web browser.
- The system employs an external large language model via the API, and it is accessible and cost-effective for the duration of the project.
- The learners' profiles are created solely based on the information that appears within the platform, including the quizzes and interactions.
- The teachers give correct information regarding the class and learning materials, and they become the ultimate judges for any suggestion of the system.
- The students and teachers consent to the collecting and processing of their learning data within the platform.
- The system operates only in English.
- The prototype works with a small range of topics and a small range of users, which is enough to test the concept of the project.
- The answers provided by the AI can be incorrect; therefore, the results must be checked by the teachers.

4.6 Use Cases

This section describes sixteen use cases of InSync, showing how students and teachers interact with the system to achieve their goals.

4.6.1 Log In to the System

Table 4.1: Log In to the System Use Case

Name	Log In to the System		
Actors	Student, Teacher		
Summary	This use case describes how a registered user logs into InSync using their account credentials to access the features available for their role.		
Pre-Conditions	The user has registered an account in the system.		
Post-Conditions	The user is logged in and redirected to the dashboard that matches their role.		
Special Requirements	The system must confirm the user’s role, either student or teacher, before deciding which dashboard to open.		
Basic Flow			
Actor Action		System Response	
1	The user enters their email and password and selects login.	2	The system checks the entered credentials against the stored account records.
3	The user waits for the system to respond.	4	The system confirms that the credentials are correct and identifies the user’s role.
5	The user views their dashboard.	6	The system displays role-specific data, such as ongoing tutoring sessions for a student.
Alternative Flow			
1	The user enters incorrect email or password.	2-A	The system displays an error message and allows the user to try again or use a password reset option, without logging the user in.

4.6.2 Attend an AI Tutoring Session

Table 4.2: Attend an AI Tutoring Session Use Case

Name		Attend an AI Tutoring Session	
Actors		Student	
Summary		This use case describes how a student starts and interacts with a one-to-one AI tutoring session on a topic they are studying.	
Pre-Conditions		The student is logged in and selects a subject or topic to study.	
Post-Conditions		The tutoring session is recorded and the student’s learner profile is updated with the outcome of the session.	
Special Requirements		The tutoring agent must adjust its explanations based on how well the student is following along, rather than repeating the same explanation.	
Basic Flow			
Actor Action		System Response	
1	The student selects a topic and starts a tutoring session.	2	The system opens a tutoring interface, and the tutoring agent begins explaining the topic or asking an opening question.
3	The student asks questions or answers the agent’s questions.	4	The tutoring agent responds with explanations, examples or follow-up questions suited to the student’s answers.
5	The student ends the session.	6	The system saves a summary of the session and updates the student’s learner profile.
Alternative Flow			
3	The student appears to be struggling with the material.	4-A	The tutoring agent switches to simpler explanations or breaks the topic into smaller steps instead of continuing at the same level.

4.6.3 Attempt an Adaptive Quiz

Table 4.3: Attempt an Adaptive Quiz Use Case

Name		Attempt an Adaptive Quiz	
Actors		Student	
Summary		This use case describes how a student attempts a quiz whose questions and difficulty adjust based on their answers.	
Pre-Conditions		The student is logged in and a quiz approved by the teacher has been made available to them.	
Post-Conditions		The student’s quiz results are saved, and the learner profile and progress records are updated.	
Special Requirements		The system must select the next question’s difficulty based on whether the student’s previous answer was correct.	
Basic Flow			
Actor Action		System Response	
1	The student starts the quiz.	2	The system presents the first question at a starting difficulty level suited to the student’s profile.
3	The student answers each question.	4	The system checks the answer and selects the next question’s difficulty based on the result.
5	The student completes the quiz.	6	The system calculates the final score and updates the student’s progress record.
Alternative Flow			
3	The student answers several questions on the same concept incorrectly.	4-A	The system flags that concept as a weak area and shares this with the tutor agent so that it reaches teacher dashboard, instead of only recording the score.

4.6.4 View Personal Progress

Table 4.4: View Personal Progress Use Case

Name	View Personal Progress		
Actors	Student		
Summary	This use case describes how a student views a summary of their own learning progress over time.		
Pre-Conditions	The student is logged in and has completed at least one tutoring or quiz session.		
Post-Conditions	None. This use case only displays information already stored in the learner profile.		
Special Requirements	Progress must be presented in a way that is easy to understand rather than raw data.		
Basic Flow			
Actor Action		System Response	
1	The student opens the progress section of their dashboard.	2	The system retrieves the student’s stored performance data from their learner profile.
3	The student views their progress over time.	4	The system displays a summary of the student’s strengths, weaknesses and completed activities.
Alternative Flow			
3	The student has not completed enough activity for a meaningful progress summary.	4-A	The system displays a message encouraging them to complete more tutoring sessions or quizzes instead of showing an empty or misleading summary.

4.6.5 View Feedback and Study Recommendations

Table 4.5: View Feedback and Study Recommendations Use Case

Name		View Feedback and Study Recommendations	
Actors		Student	
Summary		This use case describes how a student views feedback and study recommendations from their recent tutoring sessions and quiz results	
Pre-Conditions		The student is logged in and has recent activity, such as a completed quiz, for the system to base recommendations on.	
Post-Conditions		None. This use case only shows information already generated by the system.	
Special Requirements		The feedback and recommendations shown must be clear, relevant, and easy for the student to act on.	
Basic Flow			
Actor Action		System Response	
1	The student opens the feedback section of their dashboard.	2	The system retrieves the student’s most recent activity from the shared learner context.
3	The student views the feedback provided.	4	The system displays specific feedback, such as concepts that need more practice, based on this recent activity.
Alternative Flow			
3	No clear weak area is identified from the student’s recent activity.	4-A	The system displays general study suggestions based on the student’s overall subject progress instead of a specific recommendation.

4.6.6 Join a Collaborative Workspace

Table 4.6: Join a Collaborative Workspace Use Case

Name	Join a Collaborative Workspace		
Actors	Student		
Summary	This use case describes how a student joins the collaborative workspace assigned for a learning topic and interacts with other students in the group.		
Pre-Conditions	The student is logged in and has been assigned to a group.		
Post-Conditions	The student enters the assigned group workspace and can participate in the group discussion.		
Special Requirements	The group is formed by the Teacher Assistant Agent using relevant learner performance information, such as complementary strengths and weaknesses, and is approved by the teacher before students can access it.		
Basic Flow			
Actor Action		System Response	
1	The student opens the collaborative learning section.	2	The system gets the assigned group for the student.
3	The student selects the assigned group.	4	The system opens the group workspace screen.
5	The student enters the workspace.	6	The system gives the student access to the group discussion.
Alternative Flow			
3	A group has not been formed for that topic.	4-A	The system displays a message showing that a group has not been assigned to the student.

4.6.7 Discussion in a Group Chat

Table 4.7: Discussion in a Group Chat Use Case

Name		Discussion in a Group Chat	
Actors		Student	
Summary		The student can discuss the selected topic with group members and request a hint from the Discussion Agent when needed.	
Pre-Conditions		The student is logged in and has joined a group.	
Post-Conditions		The discussion messages are saved in the system.	
Special Requirements		None	
Basic Flow			
Actor Action		System Response	
1	The student opens the assigned group.	2	The system displays the group chat and group members.
3	The student sends a message about the selected topic.	4	The system displays the message to the group members.
5	The student discusses the topic with the group.	6	The system keeps displaying the discussion to the group.
7	The student requests a hint.	8	The agent gives a relevant hint.
Alternative Flow			
7	If students do not request a hint.	8-A	The students continue their discussion without hints.The Discussion Agent may still post a hint on its own if no message has been posted for three minutes

4.6.8 Receive AI Guided Hints in a Group

Table 4.8: Receive AI Guided Hints in a Group Use Case

Name		Receive AI Guided Hints in a Group	
Actors		Student	
Summary		This use case describes how a student can get a hint from the Discussion Agent while discussing a topic with the group.	
Pre-Conditions		The student is logged in and has joined a group discussion.	
Post-Conditions		The hint is displayed in the group discussion.	
Special Requirements		The hint should be related to the selected topic, not the exact answer.	
Basic Flow			
Actor Action		System Response	
1	The student opens the group discussion.	2	The system displays the group chat screen.
3	The student requests a hint.	4	The agent gives a hint related to the topic in the chat.
5	The student reads the hint and continues their discussion.	6	The system shows the hint in the chat.
Alternative Flow			
3	The student does not request a hint.	4-A	The group discussion continues without a hint. The Discussion Agent may still post a hint on its own if no message has been posted for few minutes

4.6.9 View Class Dashboard

Table 4.9: View Class Dashboard Use Case

Name		View Class Dashboard	
Actors		Teacher	
Summary		The teacher can view the performance and progress of students from the class dashboard.	
Pre-Conditions		The teacher is logged in and student data is available.	
Post-Conditions		The performance of students of the selected class is displayed.	
Special Requirements		None	
Basic Flow			
Actor Action		System Response	
1	The teacher opens the class dashboard.	2	The system displays the class performance data.
3	The teacher selects a student or topic.	4	The system fetches the data and displays it on the screen.
5	The teacher reviews the performance.	6	The system keeps displaying the performance data on the screen.
Alternative Flow			
3	The teacher selects a student or topic with no record.	4-A	The system displays a message that performance data is not available.

4.6.10 Review an AI Recommendation

Table 4.10: Review an AI Recommendation Use Case

Name		Review an AI Recommendation	
Actors		Teacher	
Summary		This use case describes how a teacher can review recommendations generated by the agent based on student performance.	
Pre-Conditions		The teacher is logged in and a recommendation has been sent.	
Post-Conditions		The recommendation is approved or dismissed by the teacher.	
Special Requirements		The recommendation should be based on student performance.	
Basic Flow			
Actor Action		System Response	
1	The teacher navigates to the dashboard and selects recommendations.	2	The system displays the generated recommendations.
3	The teacher reviews the recommendation.	4	The system provides options to approve or dismiss it.
5	The teacher selects an option.	6	The system performs the action selected by the teacher.
Alternative Flow			
1	The teacher finds no recommendations.	2-A	The system displays a message that there are no recommendations to review.

4.6.11 Upload Learning Material

Table 4.11: Upload Learning Material Use Case

Name		Upload Learning Material	
Actors		Teacher	
Summary		The teacher uploads learning materials such as lecture notes, slides, PDFs, or reference documents for a class.	
Pre-Conditions		1. The teacher is logged in. 2. The teacher has access to the selected class.	
Post-Conditions		The uploaded material is stored and associated with the selected class and becomes available to authorized students.	
Special Requirements		The system must support specified file formats and file-size limits. Only authorized class members can access the material.	
Basic Flow			
Actor Action		System Response	
1	The teacher opens the Learning Material section and selects the upload option.	2	The system displays the file upload interface.
3	The teacher selects the learning material and enters relevant information such as title or topic.	4	The system validates the file type, size, and entered information.
5	The teacher confirms the upload.	6	The system uploads, processes, and associates the material with the selected class.
Alternative Flow			
3	The teacher selects an unsupported or oversized file.	4-A	The system rejects the file and displays an appropriate error message.
5	The upload or processing fails.	6-A	The system informs the teacher and provides an option to retry.

4.6.12 Create and Assign a Quiz

Table 4.12: Create and Assign a Quiz Use Case

Name		Create and Assign a Quiz	
Actors		Teacher	
Summary		The teacher creates a quiz manually or uses AI assistance to generate questions and assigns the approved quiz to students or study groups.	
Pre-Conditions		1. The teacher is logged in. 2. A class exists. 3. Students are enrolled in the class.	
Post-Conditions		The approved quiz is stored and assigned to the selected students or groups.	
Special Requirements		AI-generated questions must be reviewed and approved by the teacher before assignment.	
Basic Flow			
Actor Action		System Response	
1	The teacher selects Create Quiz and enters the quiz topic, difficulty, and question settings.	2	The system displays and records the quiz configuration.
3	The teacher manually creates questions or requests AI-generated questions.	4	The system stores manual questions or generates draft questions using the Quiz Agent.
5	The teacher reviews the questions, selects students/groups, and assigns the quiz.	6	The system saves and publishes the quiz for the selected students/groups.
Alternative Flow			
1	Required quiz information is missing.	2-A	The system highlights the missing information and prevents assignment until corrected.
3	AI question generation fails or produces unsuitable questions.	4-A	The system allows the teacher to retry, edit, remove, or manually create questions.

4.6.13 Create and Manage Study Groups

Table 4.13: Create and Manage Study Groups Use Case

Name		Create and Manage Study Groups	
Actors		Teacher	
Summary		The teacher creates study groups and assigns students to groups for collaborative learning activities. The Teacher Assistant Agent can assist by proposing suitable group arrangements, which the teacher approves or edits.	
Pre-Conditions		1. The teacher is logged in. 2. The class exists. 3. Students are enrolled in the class.	
Post-Conditions		Study groups and their memberships are saved, and students can access their assigned collaborative workspace.	
Special Requirements		The teacher must have final control over group membership. AI-generated grouping recommendations must be editable.	
Basic Flow			
Actor Action		System Response	
1	The teacher opens Study Groups and selects Create Groups.	2	The system displays enrolled students and group creation options.
3	The teacher manually selects students or requests a grouping recommendation from teacher assistant agent.	4	The system creates the selected groups or provides suggested group arrangements.
5	The teacher reviews the groups, edits them if needed, and approves them.	6	The system saves the groups and provides students access to their shared group workspace.
Alternative Flow			
3	Invalid or duplicate group membership occurs.	4-A	The system displays a warning and asks the teacher to correct the group configuration.
5	The teacher rejects the AI-recommended grouping.	6-A	The system allows the teacher to modify the recommendation or manually create groups.

4.6.14 Create and Manage Class

Table 4.14: Create and Manage Class Use Case

Name		Create and Manage Class	
Actors		Teacher	
Summary		The teacher creates and manages a virtual class where students can access learning materials, quizzes, study groups, collaborative activities, and AI-supported features.	
Pre-Conditions		The teacher is registered and logged into the system.	
Post-Conditions		A class is created or its information and settings are successfully updated.	
Special Requirements		Each class must have a unique identifier or join code, and only authorized teachers can modify the class.	
Basic Flow			
Actor Action		System Response	
1	The teacher selects Create Class and enters the required class information.	2	The system validates the provided information.
3	The teacher confirms class creation.	4	The system creates the class and generates a unique class identifier or join code.
5	The teacher later modifies class information or settings.	6	The system validates and saves the updated class information.
Alternative Flow			
1	Required class information is missing or invalid.	2-A	The system displays validation errors and prevents creation until corrected.
5	The teacher chooses to archive a class.	6-A	The system requests confirmation and archives the class while retaining its existing records.

4.6.15 Start a Learning Session

Table 4.15: Start a Learning Session Use Case

Name	Start a Learning Session		
Actors	Teacher		
Summary	This use case describes how the teacher starts a learning session by selecting the topic that students will study.		
Pre-Conditions	The teacher is logged in and has access to the class.		
Post-Conditions	A session is created for the selected topic and students can join the session.		
Special Requirements	The selected topic should be available to the AI agents for providing topic-related learning support.		
Basic Flow			
Actor Action		System Response	
1	The teacher opens the learning session section.	2	The system displays the available topics.
3	The teacher selects a topic and clicks start.	4	The system creates the session and it becomes active.
Alternative Flow			
3	The teacher does not select a topic.	4-A	The system prompts the teacher to select a topic before starting a session.

4.7 Hardware and Software Requirements

This section lists the hardware and software needed to develop, test and run InSync, including the technology stack used to build the system.

4.7.1 Hardware Requirements

The following hardware components are needed for the development of the project:

- An internet-capable computer will be required for the development and testing of the project.

4.7.2 Software Requirements

The InSync platform requires a set of software technologies to support the web interface, backend services, AI-agent coordination, data storage, retrieval, and real-time collaboration. The selected technolo-

gies are intended to keep the system modular, maintainable, and suitable for AI-assisted educational workflows.

Table 4.16: Software Technologies Used in InSync

Software / Technology	Purpose in InSync
React	Used to build the student and teacher web interfaces, including dashboards, learning workspaces, quizzes, progress views, and collaborative group workspaces.
TypeScript	Used with React to provide type safety and improve maintainability of the frontend code.
FastAPI	Used as the main backend framework for API development, authentication-related services, class and quiz management, AI integration, and real-time functionality.
Python	Main programming language used for the FastAPI backend, AI agents, progress analysis, recommendation logic, and retrieval pipeline.
LangGraph	Used to coordinate the Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, and Discussion Agent through a shared workflow state.
PostgreSQL	Used as the main relational database for users, classes, topics, quiz attempts, learner information, groups, recommendations, and related application data.
Supabase	Used as the managed PostgreSQL hosting platform and may also provide storage and related backend services.
pgvector	PostgreSQL extension used to store vector embeddings of teacher-provided learning material for semantic retrieval.
SQLAlchemy	Used as the Object Relational Mapper (ORM) between the FastAPI backend and PostgreSQL.
Pydantic	Used for request, response, and AI-output validation in the FastAPI backend.
Redis	Used for temporary real-time state such as user presence, controlled-editing locks, caching, and other short-lived collaborative data.
WebSockets	Used through FastAPI for real-time communication in group chat, collaborative workspaces, presence updates, and live teacher observation.
Supabase Storage / Compatible Object Storage	Used to store uploaded learning materials such as PDF documents, slides, and other course files.
External LLM API	Used by AI agents to generate explanations, hints, recommendations, and other language-based outputs.
Embedding API / Model	Used to convert learning-material chunks and learner queries into vector representations for retrieval.
GitHub	Used for version control, shared development, issue tracking, pull requests, and code review.
Docker	Used to containerize the application components and provide a consistent development and deployment environment.
Visual Studio Code	Used as the primary development environment for frontend, backend, database, and AI-agent development.

The minimum software environment therefore consists of a modern web browser for accessing the application, a React and TypeScript frontend, a FastAPI-based Python backend, PostgreSQL for persistent storage, and LangGraph for AI-agent coordination. Additional technologies such as Redis, pgvector, WebSockets, and object storage support the real-time collaboration, retrieval, and persistence requirements of the system.

4.8 Graphical User Interface

This section presents the main InSync user interfaces, including the authentication screens and the interfaces provided to students and teachers.

4.8.1 Authentication Screens

The following figures show the InSync authentication interfaces.

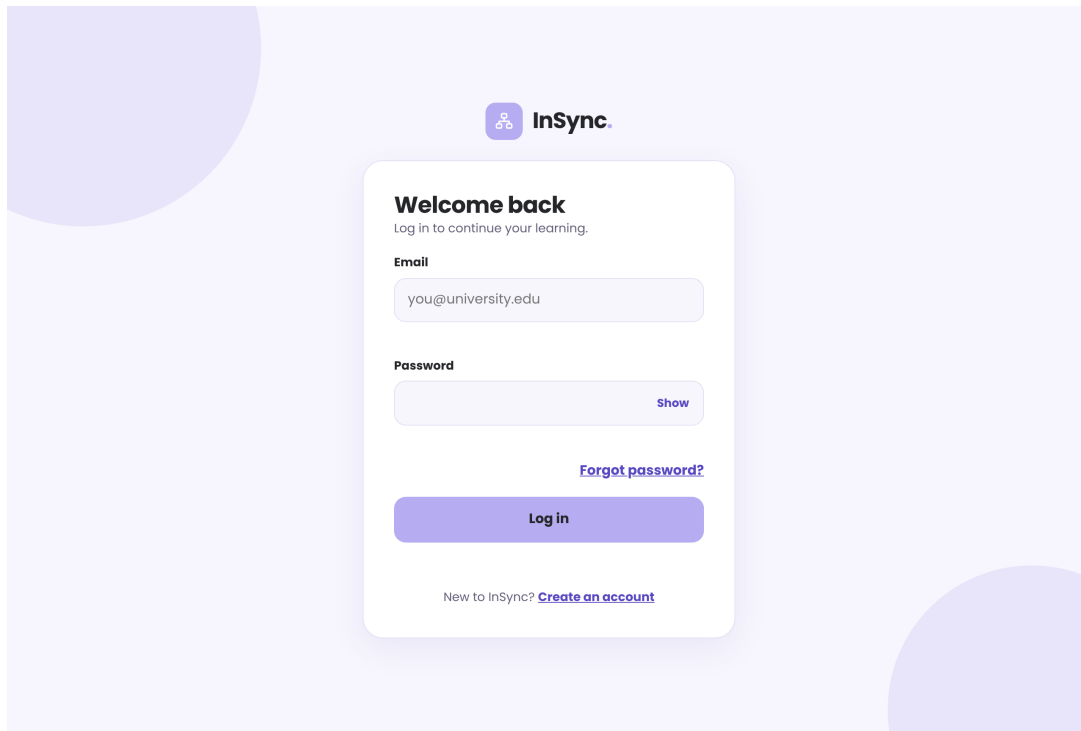


Figure 4.1: Login Screen

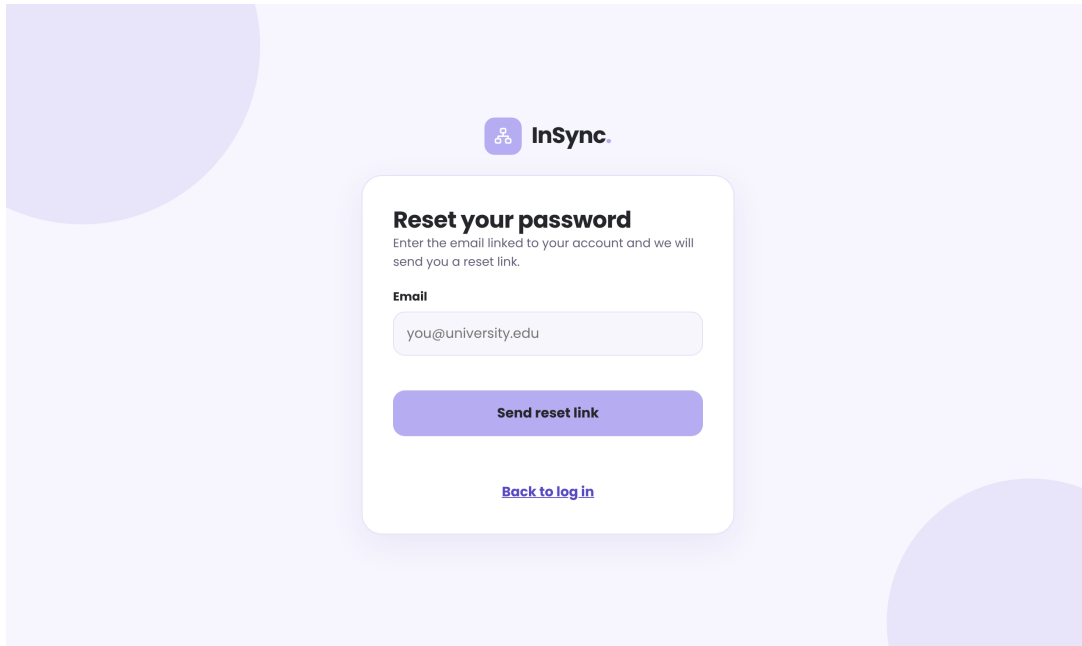
The image shows a 'Forgot Password' screen for the InSync platform. At the top, there is the InSync logo, which consists of a purple square with a white icon of three people, followed by the text 'InSync'. Below the logo, the heading 'Reset your password' is displayed in bold. Underneath the heading, a subtext reads: 'Enter the email linked to your account and we will send you a reset link.' There is a text input field labeled 'Email' containing the text 'you@university.edu'. Below the input field is a purple button with the text 'Send reset link'. At the bottom of the form, there is a link that says 'Back to log in' in blue text. The entire form is centered on a light purple background with large, faint circular shapes in the corners.

Figure 4.2: Forgot Password Screen

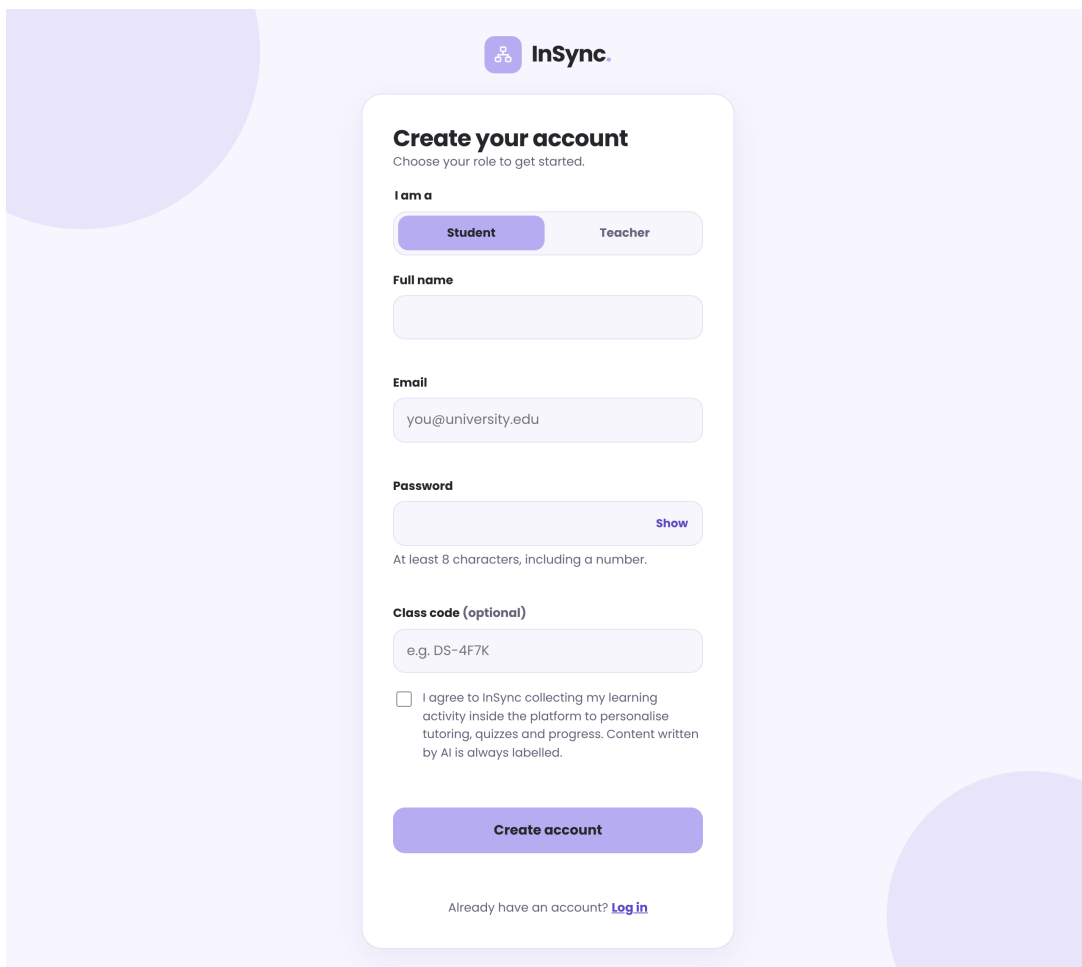
The image shows a 'Sign Up' screen for the InSync platform. At the top, there is the InSync logo, which consists of a purple square with a white icon of three people, followed by the text 'InSync'. Below the logo, the heading 'Create your account' is displayed in bold. Underneath the heading, a subtext reads: 'Choose your role to get started.' There is a section labeled 'I am a' with two buttons: 'Student' (which is highlighted in purple) and 'Teacher'. Below this is a text input field labeled 'Full name'. There is a text input field labeled 'Email' containing the text 'you@university.edu'. Below the email field is a text input field labeled 'Password' with a 'Show' link to its right. Below the password field, a subtext reads: 'At least 8 characters, including a number.' There is a text input field labeled 'Class code (optional)' containing the text 'e.g. DS-4F7K'. Below the class code field, there is a checkbox with the text: 'I agree to InSync collecting my learning activity inside the platform to personalise tutoring, quizzes and progress. Content written by AI is always labelled.' At the bottom of the form is a purple button with the text 'Create account'. Below the button, there is a link that says 'Already have an account? Log in' in blue text. The entire form is centered on a light purple background with large, faint circular shapes in the corners.

Figure 4.3: Sign Up Screen

4.8.2 Student Interface

The following figures present the main interfaces available to students.

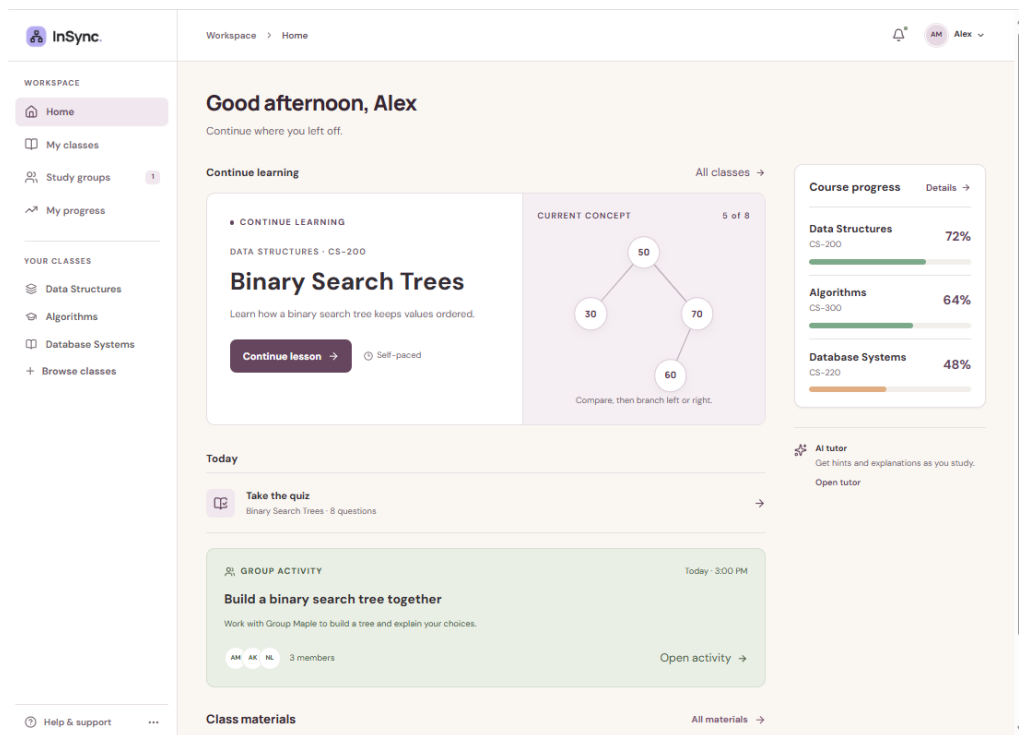


Figure 4.4: Student Dashboard

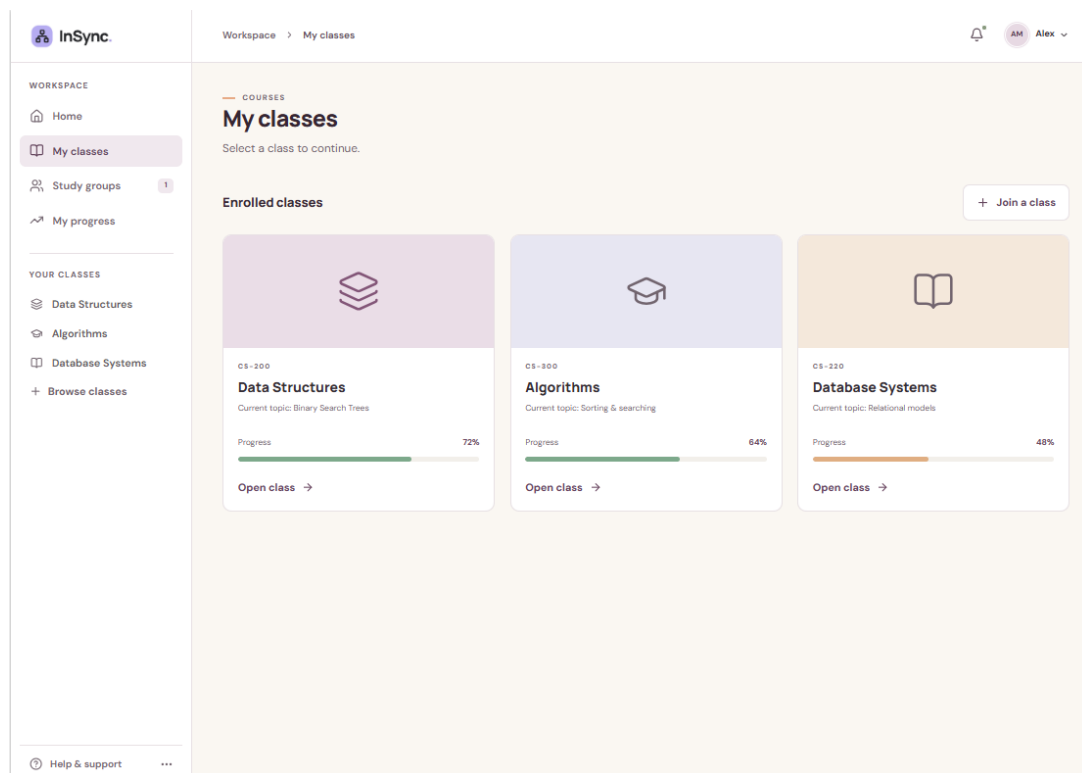


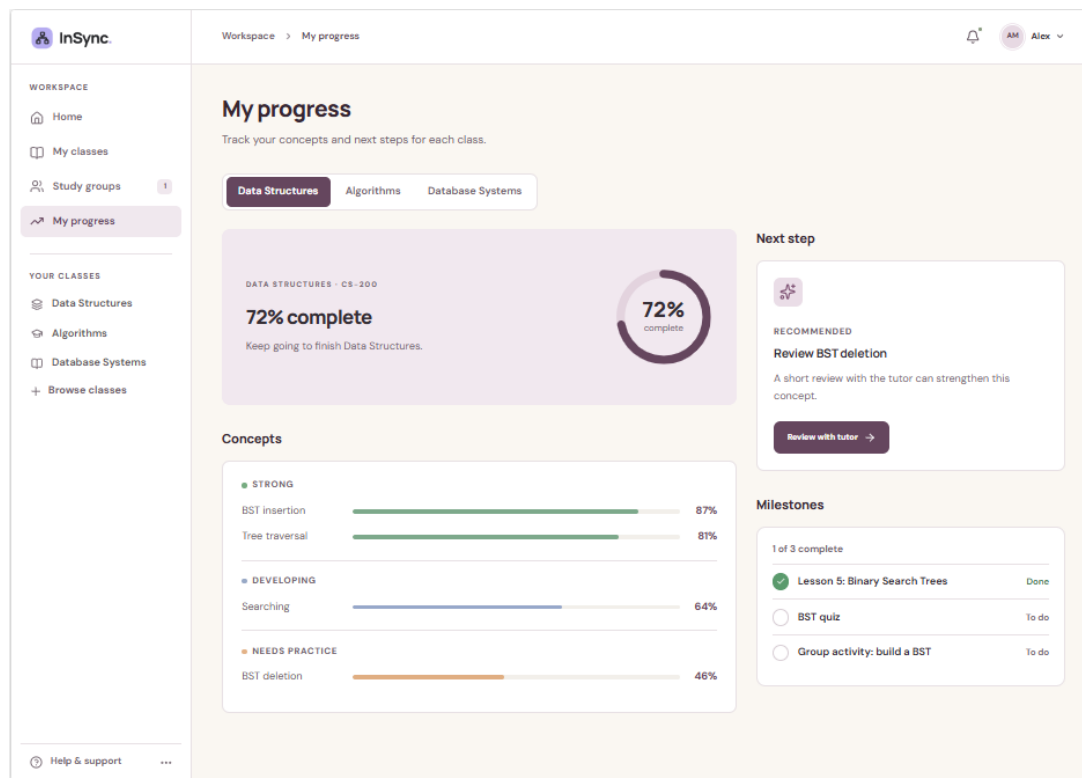
Figure 4.5: Student Views his Classes

The screenshot shows the InSync student interface for the 'Data Structures' class. The left sidebar contains navigation links for 'Workspace' (Home, My classes, Study groups, My progress) and 'Your classes' (Data Structures, Algorithms, Database Systems, Browse classes). The main content area is titled 'Data Structures' and includes a progress bar at 72%. Below the title, there are tabs for 'Overview', 'Learning', 'Materials', 'Quizzes', and 'Groups'. The 'Current topic' section features a 'Binary Search Trees' topic card with a diagram of a binary tree (root 50, left child 30, right child 70, and 70's left child 60) and a 'Continue learning' button. The 'Materials' section lists three items: 'Lecture 05 - Binary Search Trees' (PDF, 24 pages), 'BST worked examples' (PDF, 8 pages), and 'Practice questions' (PDF, 12 pages). The 'Today' section shows a 'Review the lesson' button, a 'Quiz' button (8 questions), and a 'Group activity' button (Today at 3:00 PM).

Figure 4.6: Student Views Data Structure Class

The screenshot shows the InSync adaptive quiz screen for the 'Data Structures / Binary Search Trees' topic. The left sidebar is the same as in Figure 4.6. The main content area is titled 'DATA STRUCTURES / BINARY SEARCH TREES' and shows '1 OF 8' questions. The quiz question is 'Where should 60 be inserted in this tree?' and asks to 'Choose the correct answer.' The diagram shows a binary tree with root 50, left child 30, and right child 70. Below the diagram are four radio button options: 'Left of 30' (A), 'Right of 30' (B), 'Left of 70' (C), and 'Right of 70' (D). There is a 'Hint' button and a 'Submit' button. At the bottom, it says 'Your answers shape your recommendations.'

Figure 4.7: Adaptive Quiz Screen

**Figure 4.8: Student Views his Progress**

InSync

Settings

Manage your profile, alerts and how InSync looks.

AK

Ayesha Khan
ayesha@university.edu

Student

Your classes
Data Structures Algorithms

Log out

Profile

Full name

Ayesha Khan

Email

ayesha@university.edu

Join another class
Enter a class code

Save profile

Notifications

New quiz assigned

☒

Group activity starting

☒

Teacher uploaded new material

☒

Email me a weekly progress summary

☐

Appearance and accessibility

Theme

Match my device

Text size

Default

Privacy

InSync only uses activity from inside the platform. It does not collect webcam, emotion or sensor data.

Use my learning activity to personalise tutoring and quizzes

☒

Change password

Current password

New password

At least 8 characters, including a number.

Update password

Figure 4.9: Student Portal Settings Page

4.8.3 Teacher Interface

The following figures present the main interfaces available to teachers.

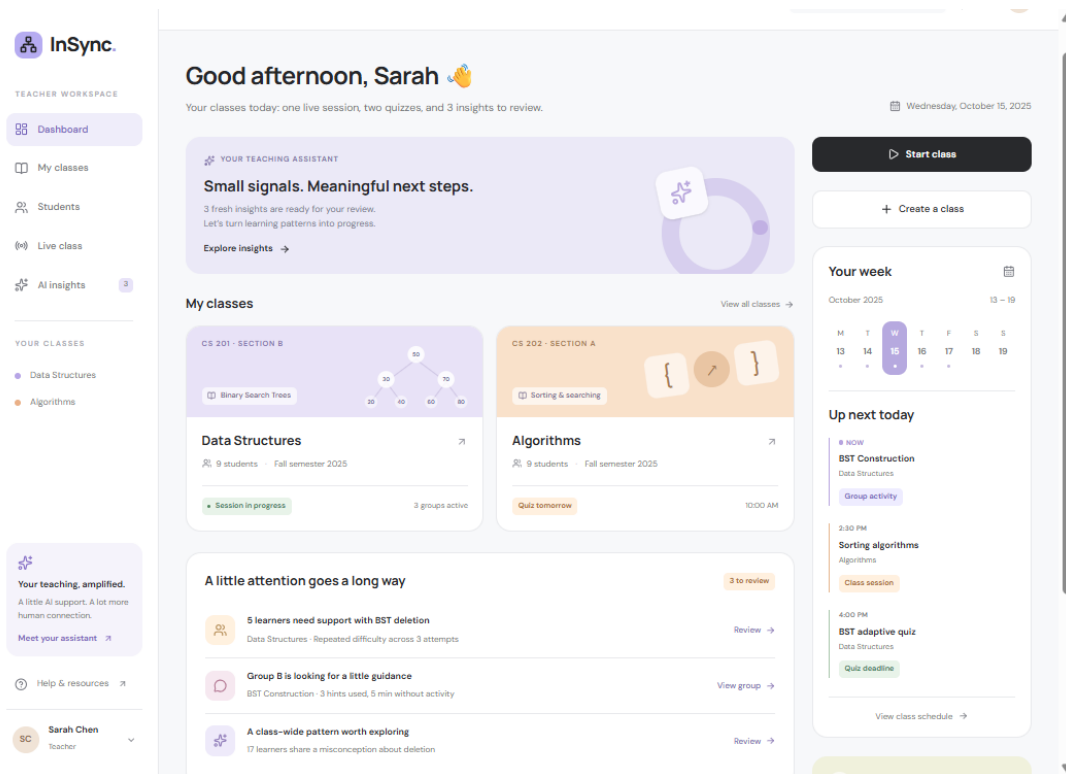


Figure 4.10: Teacher Dashboard

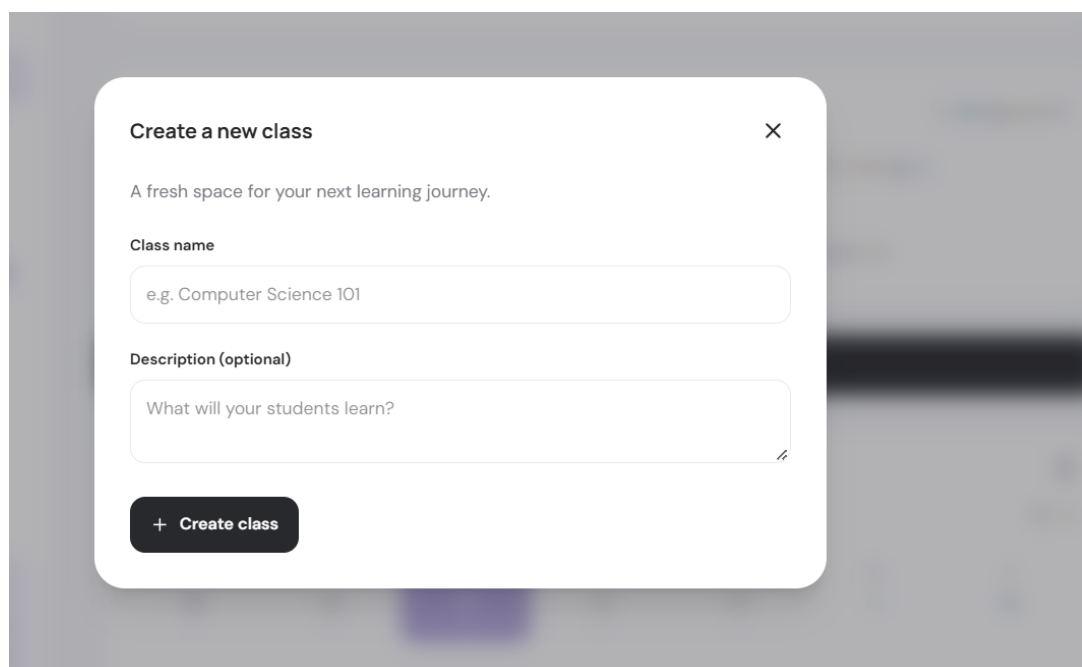


Figure 4.11: Create Class Dialog

The screenshot displays the InSync Teacher Workspace interface. On the left is a sidebar with navigation options: Dashboard, My classes (selected), Students, Live class, and AI insights (with a notification badge). Below these are 'YOUR CLASSES' listed as Data Structures and Algorithms. At the bottom of the sidebar, there's a section for 'Your teaching, amplified.' with a link to 'Meet your assistant' and a 'Help & resources' link. The user profile 'Sarah Chen' is shown at the bottom left.

The main content area is titled 'My classes' with the subtitle 'Every learning journey, in one place.' and a '+ Create a class' button. It features two class cards:

- CS 201 - SECTION B**: Focuses on 'Binary Search Trees'. It includes a diagram of a binary tree with root 50, left child 30, and right child 70. Node 30 has children 20 and 40; node 70 has children 60 and 80. Below the diagram, it says 'Data Structures', '9 students · Fall semester 2025', 'Session in progress' (in a green box), and '3 groups active'.
- CS 202 - SECTION A**: Focuses on 'Sorting & searching'. It includes icons for curly braces, a pencil, and a magnifying glass. Below, it says 'Algorithms', '9 students · Fall semester 2025', 'Quiz tomorrow' (in an orange box), and '1000 AM'.

At the bottom of the main area, there is a footer note: 'Made for the moments that make learning click.' and a status indicator 'All systems in sync'.

Figure 4.12: Teacher Views Classes

The screenshot displays the InSync Teacher Workspace interface for a class named "Data Structures". The left sidebar shows navigation options: Dashboard, My classes (selected), Students, Live class, AI insights (3), and YOUR CLASSES (Data Structures, Algorithms). The main content area shows the class overview for "Data Structures" (CS 201 - Section B - 9 students - Fall semester 2025). The current learning topic is "Binary Search Trees" (Week 6: Explore, practice, collaborate), with a status of "Learning in progress". A diagram of a binary search tree is shown. The "The learning cycle" section lists: Learning materials uploaded, Topic taught, Adaptive quiz, and Collaborative activity (Live). Below this are four summary cards: Average topic accuracy (68%, +6% this week), Learners needing support (3, A little help goes a long way), Active study groups (3, Collaborating right now), and AI recommendations (3, Ready for your review). A section titled "A little attention goes a long way" (3 to review) lists three items: 5 learners need support with BST deletion, Group B is looking for a little guidance, and A class-wide pattern worth exploring.

Workspace > My classes

Search anything...

SC

TEACHER WORKSPACE

Dashboard

My classes

Students

Live class

AI insights 3

YOUR CLASSES

Data Structures

Algorithms

My classes

Data Structures

CS 201 - Section B - 9 students - Fall semester 2025

Observe live activity

Overview Materials Activities Quizzes Groups Students Insights

CURRENT LEARNING TOPIC

Binary Search Trees

Week 6 - Explore, practice, collaborate

Learning in progress

The learning cycle

- Learning materials uploaded
- Topic taught
- Adaptive quiz
- Collaborative activity **Live**

Average topic accuracy

68%

+6% this week

Learners needing support

3

A little help goes a long way

Active study groups

3

Collaborating right now

AI recommendations

3

Ready for your review

Your teaching, amplified.

A little AI support. A lot more human connection.

Meet your assistant

Help & resources

Sarah Chen Teacher

A little attention goes a long way

3 to review

- 5 learners need support with BST deletion
Data Structures - Repeated difficulty across 3 attempts
Review
- Group B is looking for a little guidance
BST Construction - 3 hints used, 5 min without activity
View group
- A class-wide pattern worth exploring
17 learners share a misconception about deletion
Review

Figure 4.13: Data Structure Class Overview

The screenshot shows the InSync Teacher Workspace interface. On the left is a sidebar with the InSync logo and navigation links: Dashboard, My classes (selected), Students, Live class, and AI insights (with a notification badge). Below these are 'YOUR CLASSES' (Data Structures, Algorithms) and a section titled 'Your teaching, amplified.' with a message about AI support and a link to 'Meet your assistant'. At the bottom of the sidebar is a user profile for Sarah Chen, Teacher.

The main content area is titled 'Data Structures' and shows 'CS 201 - Section B - 9 students - Fall semester 2025'. It has tabs for Overview, Materials (selected), Activities, Quizzes, Groups, Students, and Insights. A button 'Observe live activity' is in the top right. The 'Materials' tab displays a list of learning materials under the heading 'Learning materials' with an 'Upload material' button. The materials are:

- BST Lecture.pdf (Binary Search Trees · Learning resource) - Ready
- BST Examples.pdf (Binary Search Trees · Learning resource) - Ready
- Practice Problems.pdf (Binary Search Trees · Learning resource) - Ready

At the bottom of the main area, there is a status bar that says 'Made for the moments that make learning click.' and 'All systems in sync'.

Figure 4.14: Teacher uploads Learning Material

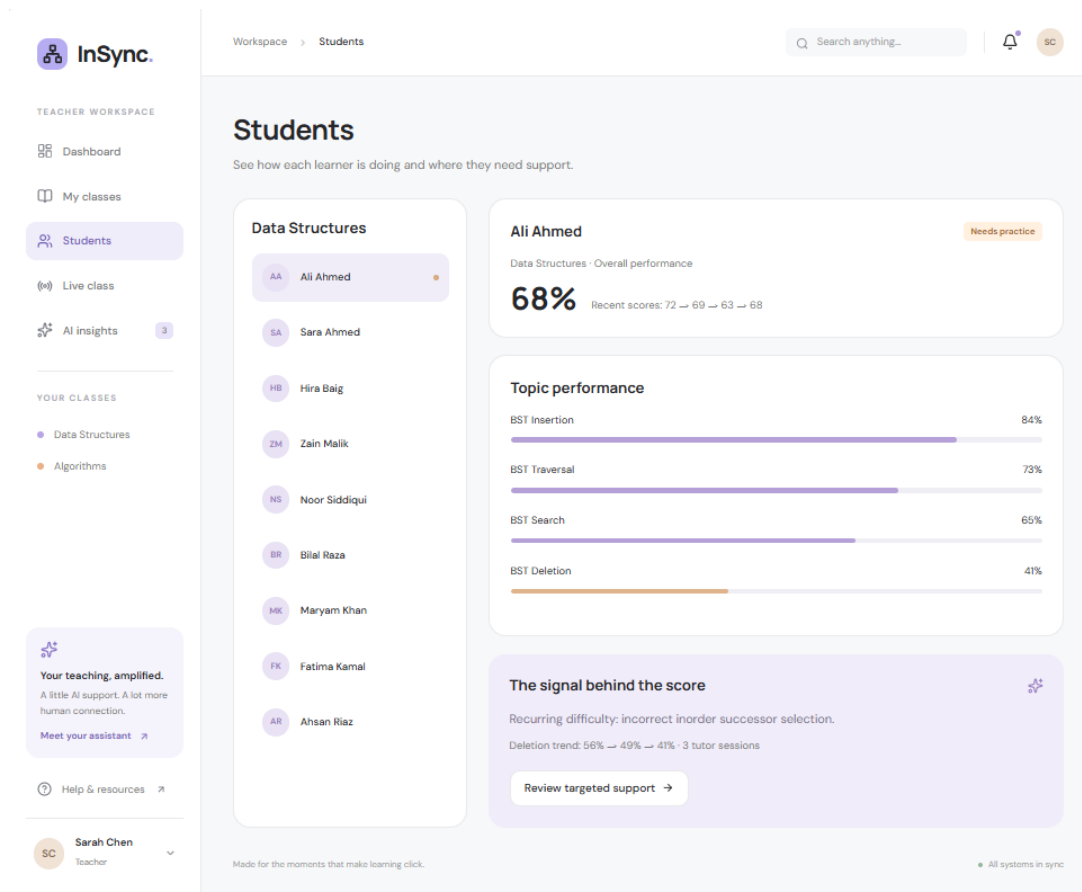


Figure 4.15: Students Performance Overview

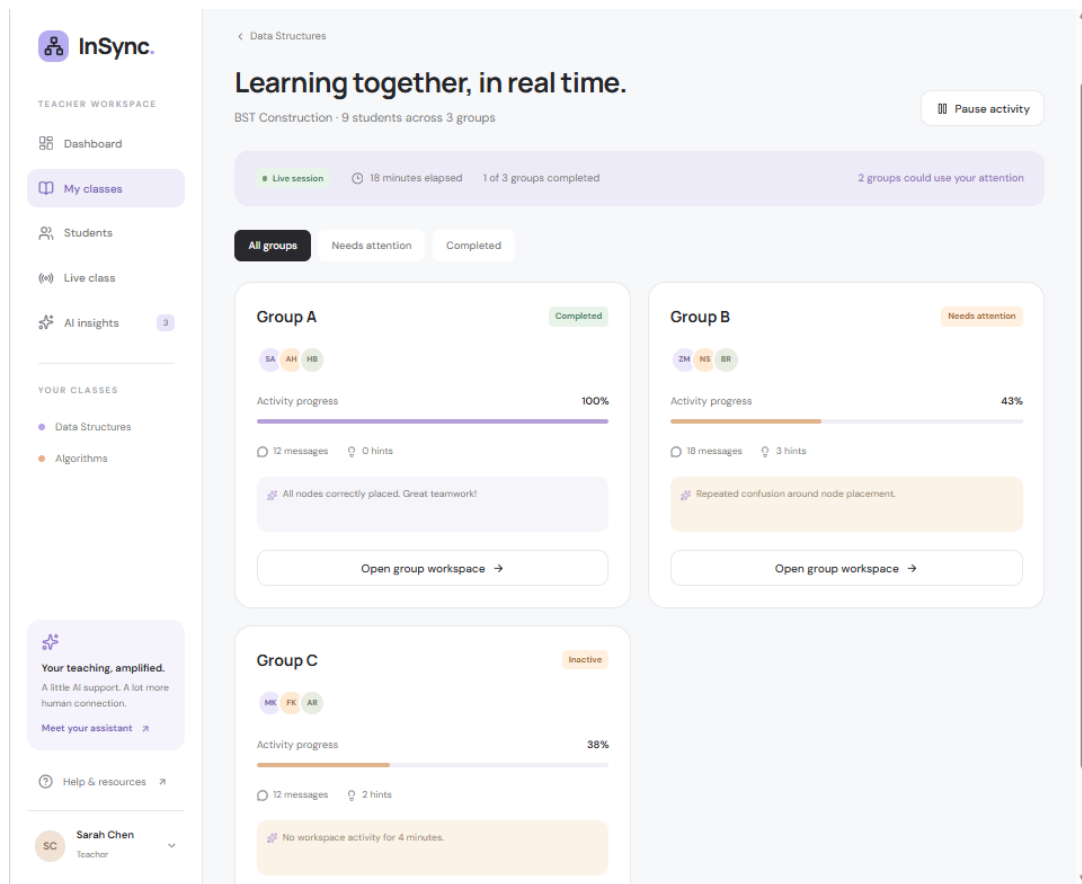


Figure 4.16: Group Monitoring Screen

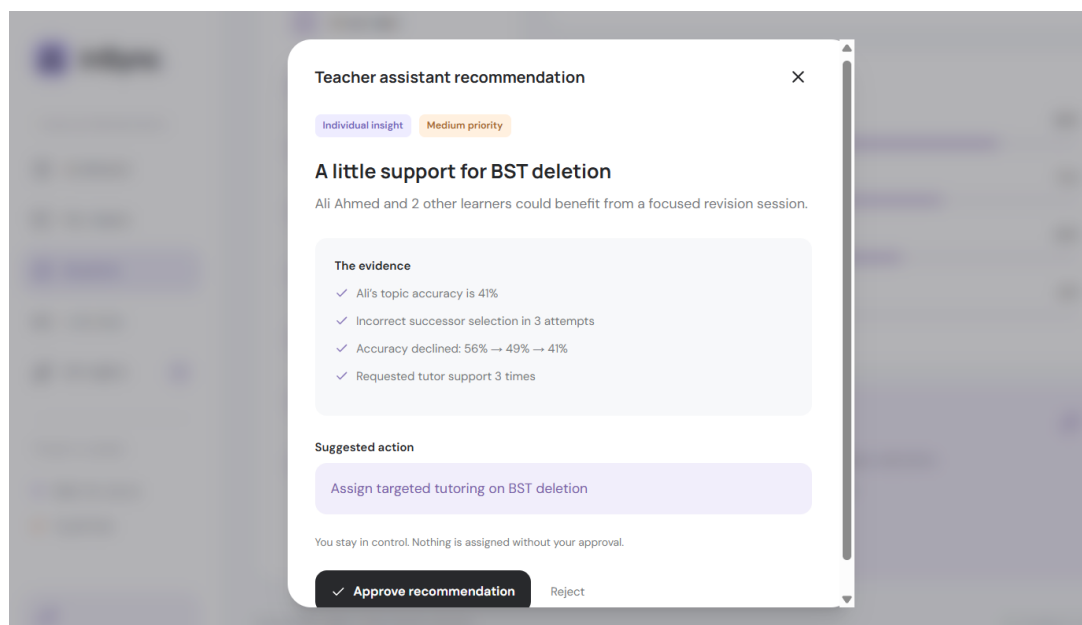


Figure 4.17: AI Insights and Recommendations

4.9.2 Data Dictionary

This table contains all the entities along with their attributes in the below table:

Table 4.17: Data dictionary of the InSync database

Entity	Attributes	Data Type	Nullable	Description
USER	user_id	uuid	No	Primary key, linked to login account
	full_name	text	No	User's full name
	email	text	No	Unique login email
	role	enum(student, teacher)	No	Account type
	created_at	timestampz	No	Account creation time
CLASS	class_id	uuid	No	Primary key
	teacher_id	uuid	No	Owner teacher (FK to USER)
	name	text	No	Class name
	join_code	text	No	Unique code for students to join
	status	enum(active, archived)	No	Class state
	created_at	timestampz	No	Creation time
CLASS_MEMBER	class_id	uuid	No	Class joined (PK, FK to CLASS)
	student_id	uuid	No	Student joining (PK, FK to USER)
	status	enum (pending, enrolled)	No	Join request state
	joined_at	timestampz	Yes	Set once enrolled
TOPIC	topic_id	uuid	No	Primary key
	class_id	uuid	No	Parent class (FK to CLASS)
	name	text	No	Topic name
	description	text	Yes	Short topic description
CONCEPT	concept_id	uuid	No	Primary key
	topic_id	uuid	No	Parent topic (FK to TOPIC)
	name	text	No	Concept name
LEARNING_MATERIAL	material_id	uuid	No	Primary key
	topic_id	uuid	No	Related topic (FK to TOPIC)

Continued on next page

Entity	Attributes	Data Type	Nullable	Description
	title	text	No	Material title
	file_path	text	No	Storage path of the file
	status	enum(processing, ready, failed)	No	Processing state
	uploaded_at	timestampz	No	Upload time
MATERIAL_CHUNK	chunk_id	uuid	No	Primary key
	material_id	uuid	No	Source material (FK to LEARNING_MATERIAL)
	chunk_index	integer	No	Position of chunk in material
	chunk_text	text	No	Text of the chunk
	embedding	vector(1536)	No	Vector used for retrieval
LEARNING_SESSION	session_id	uuid	No	Primary key
	topic_id	uuid	No	Topic taught (FK to TOPIC)
	status	enum (active, ended)	No	Session state
	started_at	timestampz	No	Start time
	ended_at	timestampz	Yes	Set when session ends
QUIZ	quiz_id	uuid	No	Primary key
	session_id	uuid	No	Related session (FK to LEARNING_SESSION)
	generated_by	enum (ai, teacher)	No	Quiz source
	status	enum (draft, approved)	No	Approval state
	created_at	timestampz	No	Creation time
	approved_at	timestampz	Yes	Set when teacher approves
QUESTION	question_id	uuid	No	Primary key
	quiz_id	uuid	No	Parent quiz (FK to QUIZ)
	concept_id	uuid	No	Concept tested (FK to CONCEPT)
	question_text	text	No	Question wording
	options	jsonb	No	List of answer choices
	correct_answer	text	No	Correct option, hidden from students
	difficulty	enum (easy, medium, hard)	No	Question difficulty

Continued on next page

Entity	Attributes	Data Type	Nullable	Description
QUIZ_ ATTEMPT	attempt_id	uuid	No	Primary key
	quiz_id	uuid	No	Quiz taken (FK to QUIZ)
	student_id	uuid	No	Student attempting (FK to USER)
	started_at	timestamptz	No	Start time
	completed_at	timestamptz	Yes	Empty while in progress
	raw_accuracy	numeric(5,4)	Yes	Share of correct answers (0 to 1)
	weighted_score	numeric(5,4)	Yes	Difficulty-weighted score (0 to 1)
	final_difficulty	enum (easy, medium, hard)	Yes	Difficulty set for next quiz
ATTEMPT_ ANSWER	answer_id	uuid	No	Primary key
	attempt_id	uuid	No	Parent attempt (FK to QUIZ_ ATTEMPT)
	question_id	uuid	No	Question answered (FK to QUESTION)
	selected_answer	text	No	Option chosen by student
	is_correct	boolean	No	Whether the answer was right
	mistake_type	text	Yes	Error type, wrong answers only
LEARNER_ CONTEXT	context_id	uuid	No	Primary key
	student_id	uuid	No	Learner, one per student (FK to USER)
	quiz_difficulty	enum (easy, medium, hard)	No	Current quiz level
	explanation_level	smallint	No	Tutor explanation level (1 to 3)
	tutoring_summary	text	Yes	Short summary of tutoring so far
	updated_at	timestamptz	No	Last update time
CONCEPT_ MASTERY	mastery_id	uuid	No	Primary key

Continued on next page

Entity	Attributes	Data Type	Nullable	Description
	context_id	uuid	No	Learner (FK to LEARNER_CONTEXT)
	concept_id	uuid	No	Concept tracked (FK to CONCEPT)
	correct_count	integer	No	Correct answers on concept
	attempted_count	integer	No	Total answers on concept
	mastery	enum (strong, developing, weak)	No	Mastery level
	weak_flag	boolean	No	True when mastery is weak (auto-set)
TUTORING_INTERACTION	interaction_id	uuid	No	Primary key
	context_id	uuid	No	Learner (FK to LEARNER_CONTEXT)
	concept_id	uuid	No	Concept discussed (FK to CONCEPT)
	explanation_level	smallint	No	Level used (1 to 3)
	repeated_confusion	boolean	No	Student confused again on concept
	check_correct	boolean	Yes	Check question result, if asked
	summary	text	No	Short summary of exchange
	created_at	timestampz	No	Interaction time
PROGRESS_ANALYSIS	analysis_id	uuid	No	Primary key
	context_id	uuid	No	Learner (FK to LEARNER_CONTEXT)
	average_accuracy	numeric(5,4)	No	Average accuracy (0 to 1)
	trend	enum (improving, declining, stable)	No	Performance direction
	summary	text	No	Short written analysis
	created_at	timestampz	No	Analysis time
RECOMMENDATION	recommendation_id	uuid	No	Primary key
	analysis_id	uuid	No	Source analysis (FK to PROGRESS_ANALYSIS)
	concept_id	uuid	Yes	Related concept (FK to CONCEPT)

Continued on next page

Entity	Attributes	Data Type	Nullable	Description
	action_type	text	No	Suggested action
	evidence	text	No	Data supporting the suggestion
	priority	enum (high, medium, low)	No	Urgency
	status	enum (pending, approved, dismissed)	No	Teacher decision
	created_at	timestampz	No	Creation time
	reviewed_at	timestampz	Yes	Set when teacher decides
STUDY_GROUP	group_id	uuid	No	Primary key
	session_id	uuid	No	Related session (FK to LEARNING_SESSION)
	name	text	No	Group name
	strategy	enum(balanced, strength_based)	No	Grouping method used
	status	enum (proposed, approved)	No	Approval state
	approved_at	timestampz	Yes	Set when teacher approves
GROUP_MEMBER	group_id	uuid	No	Group joined (PK, FK to STUDY_GROUP)
	student_id	uuid	No	Member (PK, FK to USER)
DISCUSSION_PROBLEM	problem_id	uuid	No	Primary key
	session_id	uuid	No	Related session (FK to LEARNING_SESSION)
	problem_text	text	No	Problem given to groups
	hidden_solution	text	No	Reference solution, never shown to students
GROUP_MESSAGE	message_id	uuid	No	Primary key
	group_id	uuid	No	Group chat (FK to STUDY_GROUP)
	sender_id	uuid	Yes	Student sender; empty for AI hints
	problem_id	uuid	Yes	Problem discussed (FK to DISCUSSION_PROBLEM)

Continued on next page

Entity	Attributes	Data Type	Nullable	Description
	message_type	enum (student, ai_hint)	No	Student message or AI hint
	content	text	No	Message text
	hint_level	smallint	Yes	Hint level, AI hints only
	hint_trigger	text	Yes	Reason hint was given, AI hints only
	created_at	timestampz	No	Sent time

4.10 Risk Analysis

This section identifies the main risks that could affect the development and success of InSync.

4.10.1 AI gives incorrect or misleading explanations

Due to the fact that InSync uses a large language model to create tutoring explanations, quiz answers and suggestions, there is a possibility that AI may provide an explanation, answer or suggestion which is not correct or is misleading. The probability of this risk is considered to be medium because large language models may provide incorrect information despite of precise instructions especially in case of advanced or rare topics. The impact of such risk is considered to be medium to high as incorrect information provided to the student will negatively influence his or her learning process.

To minimize such risk, the list of subjects and topics covered by the Tutor agent will be limited to the ones which have been tested in the course of development, the feature for users to notify about wrong or misleading information will be implemented and regularly analyzed.

4.10.2 Dependence on a third-party language model service

The AI service can be considered external to the InSync system because InSync utilizes an API to use the AI service to enable its tutoring, assessment, and feedback functionalities. Downtime, pricing changes, and limits on the number of requests sent to the external AI service can make key features of InSync not functional or unaffordable. The likelihood of this risk occurring is low to medium, because of the reliability of established AI services, but rate limits and price changes are realistic concerns for a student project with a limited budget. The impact of this risk is rated high because almost all of the key functionality of InSync depend on this service, therefore, downtime or rate limit will affect the entire application.

The following risk mitigation strategies will be employed: API usage will be monitored during development to operate under free or inexpensive usage limits, design the system so that when a request to

the AI service fails the user sees an understandable error message. The application shall not crash on AI failure, and the AI provider shall be replaceable without code changes. Keep the system flexible and easy to update.

4.10.3 Leak or misuse of student data

InSync also stores data about learners, including quiz scores, interaction records, and progress records, some of which is sent to an external AI tool that generates personalized content. There is a possibility that this data may be leaked due to a breach in the security protocol or mishandled in a manner that leads to misuse of the data. The likelihood associated with this risk is rated as low if proper security measures are used during development of the system while the likelihood is considered to be high if the user authentication and data handling processes are not designed well. The impact associated with this risk is high because student data is private in nature and any misuse or leakage would create an ethical issue for the research project.

The risk can be mitigated through a proper authorization process for accessing student data, limiting the sending of unnecessary personal information to the external AI system and hashing passwords and other personal information.

4.10.4 Unsafe or inappropriate AI-generated content

This threat occurs as students interact with AI agents, hence there is a probability that an agent may give an output that is not appropriate and does not add to the learning experience in any way in response to a student's question that goes beyond the intended scope of the Tutor Agent. The probability of this threat occurring is low to moderate due to the existence of safety precautions offered by the majority of AI providers, although some probability remains, particularly due to student input being open-ended. The impact of the threat is low to moderate due to inappropriate content being a great issue to teachers using an educational tool.

These threats will be mitigated through the use of the safety and content filtering tools provided by the AI provider, limiting the output of the tutoring agent to the subject matter only, and offering teachers a way to monitor for inappropriate content.

4.10.5 Agents fail to coordinate or share learner context correctly

Since InSync uses different AI agents that need to share relevant learner information with each other, there is a possibility that an agent may not receive the required information correctly or that the information may not be updated properly. The likelihood of this risk is considered to be medium because

communication between multiple agents can have errors during development. The impact is also considered medium because incorrect or missing learner information can affect the tutoring, quiz, progress, or group activities provided by the system.

To reduce this risk, the learner information shared between agents will be kept in a clear and consistent format. The communication between agents will also be tested with different learning scenarios to make sure that the required information is passed and updated correctly.

4.10.6 Scope growth and schedule delays

Since InSync has five AI agents and a number of features for both students and teachers, it is possible that the project could take longer than the originally planned time. The probability of this risk is considered medium since the team has a limited development timeframe and there are many components that have to be finished. The effect of this risk is seen as high since delays in development could decrease the time left for testing and might influence the completion of the final system.

This risk can be lowered by adhering to the features specified in the project scope and by avoiding the inclusion of any unnecessary features during development; tasks will also be divided among the team members and regularly checked in order to enable any delays to be identified and dealt with early.

4.10.7 Students rely on AI and interact less with peers

InSync offers AI-based learning support, it is possible that students might rely too much on the AI and fail to discuss the topic with their group members. However, the likelihood of this risk is judged to be low to medium since students can use the Tutor Agent for individual assistance and the platform also includes group learning activities. The impact is seen as medium because excessive dependence on the AI could decrease the amount of interaction among students when they are engaged in collaborative learning.

The risk can be lowered by having the Discussion Agent give hints rather than providing direct answers. The students will continue to discuss the topic with their group, the AI will only offer extra assistance when this is required.

4.10.8 Difficulty finding real users for testing

If enough students and teachers are not available during the testing phase, then the testing of InSync will be difficult. The likelihood of this risk is considered to be medium because students and teachers may have limited time to participate in testing. The impact of this risk is considered medium because having fewer real users may limit the feedback available about the system and its usability.

This risk will be reduced by planning the testing activities in advance and contacting available students and teachers for participation. If enough users are not available, sample student data and learning sessions will be used to test the main features of the system before conducting user testing when participants become available.

4.10.9 Slow responses under heavy use

There may be multiple students accessing InSync simultaneously. Increased loads will occur due to AI requests, real-time collaboration, updating of the shared workspaces, and operations in the database. The likelihood of this threat happening is considered moderate especially during live collaborative sessions in which a number of activities take place. The effect would be considered high since delayed AI answers or poor synchronization of shared workspaces might interfere with collaboration and negatively affect the overall learning experience. Some mitigation measures against such a threat include optimizing database operation and API calls, implementing asynchronous processing of lengthy AI processes, caching where applicable and optimized real-time communication.

4.10.10 Inaccurate learner profiles lead to poor personalization

The AI agents employed by InSync gather all available information, including quiz outcomes, attempts at completing assignments, collaborations and previous learner interactions in order to construct an accurate representation of their progress in the learning process. However, a lack or poor interpretation of this information may lead to inaccurate representation of learners and thus inappropriate hints, suggestions and insights for teachers. The probability of occurrence of this threat is medium since AI may sometimes make incorrect assumptions, especially if little information is gathered about a learner. However, the consequences are high in severity since wrong personalizations would adversely affect learner support and teacher insights. This risk can be managed by creating each learner profile using different pieces of information instead of just one result. It is important to present AI conclusions as suggestions. These should always come with strong evidence. Teachers should keep control over important educational decisions.

4.10.11 Team members unavailable or workload imbalance

Members of the team may experience periods of absence due to the amount of work at university being too heavy, illness, personal issues or commitments to some other projects. Additionally, unequal division of labor might lead to an extremely high load of certain team members. There is a medium chance of occurrence in this regard since team members would still have other projects going simultaneously during the entire development phase. The impact is also medium because any absence or overload can

cause delay either in developing, integration testing or meeting project deadlines. These risks can be minimized by assigning specific roles to team members, documenting key information about projects and redistributing tasks whenever required.

4.10.12 Loss of code or data due to missing backups

The source code of the project, its configurations, learning materials, database records, or any critical data may be accidentally deleted, corrupted, or become inaccessible due to various hardware or software issues. Based on version control and the planned usage of backup solutions during development, the likelihood of data loss is expected to be low. However, should such circumstances happen, it would have a significant effect as this would cause considerable delays and recreate the already finished work. Source code of the project would be stored and version-controlled using Git and committed to a remote repository on a frequent basis, while key project and database data would be saved periodically.

Chapter 5 Proposed Approach and Methodology

InSync uses a coordinated multi-agent learning methodology to support personalized tutoring, adaptive assessment, progress tracking, teacher decision-making, and collaborative learning. Rather than assigning all responsibilities to a single AI component, the system uses five specialized agents: the Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, and Discussion Agent. LangGraph coordinates these agents through a shared system state so that information produced during one learning activity can support later activities without requiring direct communication between agents. The teacher remains involved in important educational decisions, particularly when AI-generated recommendations may affect a learner's future activities. The following sections describe the overall multi-agent learning approach, tutoring procedure, adaptive quiz mechanism, performance analysis, teacher recommendation methodology, collaborative group formation, discussion hints, agent coordination, and the complete end-to-end learning workflow.

5.1 Overall Multi-Agent Learning Approach

The InSync system adopts a coordinated multi-agent approach and consists of five specialized AI agents working together within a shared learner context throughout all the learning process stages. It starts with the teacher initiating a new learning session, choosing the topic, and starting the tutoring process. Learners attend the session, receive additional tutoring if necessary, and perform the topic-based quizzes. Information about the learners collected through these actions is used to update the learner context and facilitate future decisions. The Progress Agent analyses the performance and typical mistakes of the learners, and the Teacher Assistant Agent converts this analysis into recommendations for the teacher, who can approve or dismiss each recommendation. When the teacher requests study groups, the Teacher Assistant Agent forms balanced or strength-based groups using the performance data, and the teacher approves or edits them. The Discussion Agent then supports the collaborative discussion by providing topic-specific hints generated by the AI. All of this is enabled through the LangGraph coordination mechanism that allows the information obtained in one learning activity to be used by the other agents and the teacher's decisions to be included in the process.

AI Agents and Their Responsibilities:

- The Tutor Agent explains the current topic using the learning material and the learner's context.
- The Quiz Agent creates topic-specific quizzes, evaluates answers, and adjusts quiz difficulty.
- The Progress Agent tracks learner progress and identifies patterns and repeated mistakes.

- The Teacher Assistant Agent interprets the findings of the Progress Agent and delivers recommendations to the teacher supported by evidence. It also forms study groups from learner performance when the teacher requests them, and presents the groups to the teacher for approval or editing.
- The Discussion Agent gives hints on the current topic during group discussions without revealing the answer.

The proposed solution utilizes a multi-agent system, decision-making heuristics, and an agent coordination protocol. LangGraph acts as a platform that integrates the five agents through the shared state which includes all necessary information about the particular learner. Heuristics are used to make decisions like adaptive quiz difficulty and balanced or strength-based grouping. Agent coordination protocol is used to exchange the information and results between the agents and integrate the teacher's decision into the process.

5.2 Learner Context Management

Learner context is the shared state that contains all the information needed by the InSync agents during the learning process. Each learner has an individual context that encompasses their current learning state and information collected from previous interactions. Instead of facilitating direct interaction between the agents, LangGraph enables them to access information from this shared state that is needed by each one of them and update only those fields they are responsible for. Learner context is established as soon as a learner begins his/her learning task. In the beginning, the learner's ID, the topic being studied and the starting difficulty level of the quiz is recorded. The learners who do not have any prior history with the quizzes get a Medium starting difficulty level. Further information is acquired gradually based on various interactions of the learners.

The shared learner context may contain information such as the learner identifier, current topic, quiz difficulty, explanation level, quiz history, raw accuracy, weighted score, topic-level performance, weak concepts, recurring mistakes, tutoring history, recent tutoring summary, group identifier, hint history, Progress Agent analysis, Teacher Assistant recommendations, and teacher decisions.

Different agents use different parts of this shared information. The Quiz Agent updates quiz-related data such as question performance, difficulty, scores, and weak concepts. The Tutor Agent records information about explanations, repeated confusion, tutoring history, and check-question performance. The Teacher Assistant Agent updates group-related information, such as the group identifier, once the teacher approves the proposed groups, and the Discussion Agent records the hints provided during collaborative activities. The Progress Agent reads the accumulated learning information and produces structured per-

formance analysis, while the Teacher Assistant Agent reads this analysis and generates teacher-facing insights and recommendations.

The learner context is updated whenever an activity produces information that may be useful to another agent. For example, after a quiz is completed, the Quiz Agent stores the learner's results and final difficulty level and identifies weak concepts. The Progress Agent can later use this information together with tutoring and collaborative-learning data to identify trends and recurring mistakes. Similarly, weak concepts identified during assessment can later influence the level or focus of explanations provided by the Tutor Agent.

To avoid inconsistent updates, each type of information is mainly maintained by the agent responsible for producing it. Where multiple agents may contribute to the same type of information, such as weak concepts or learning difficulties, new entries are added without overwriting the evidence already recorded by another agent.

During implementation, LangGraph maintains the active shared state used during agent coordination, while persistent learner information is stored in PostgreSQL. This allows relevant learner information to remain available across different learning activities and prevents important records from being lost if an individual agent request fails.

5.3 Tutor Agent Explanation Procedure

The Tutor Agent answers learner's questions about concepts during a tutoring session. Each of its explanations relies on two factors. One factor is the information contained in the shared learner context about the learner's present level and the learner's previous mistakes. The other is the learning materials provided by the teacher on the topic. The Tutor Agent uses three types of input. First is the question asked by the learner. Second is the learner context, which includes the present difficulty level, weaknesses in concepts, mistakes and the summary of the last tutoring session. Third is the learning material on the topic provided by the teacher.

The explanation process is completed in the following sequence:

- The agent checks whether the learner's query is related to the session topic by comparing it with the learning material. It labels the query as on-topic, off-topic, or unclear.
- The agent retrieves the learner's difficulty level, weak concepts, related mistakes, and summary of the last tutoring session.
- The agent extracts the three sections from the learning material that are most relevant to the learner's query.

- The agent selects an explanation level based on the criteria described below.
- The language model uses the learner's query, relevant material, selected explanation level, and recorded misconceptions to generate a short explanation, one worked example, and one check question.
- The agent validates the response to ensure it is on-topic, does not reveal answers to an ongoing quiz, and passes the provider's content filter. If validation fails, the response is generated again. If it fails a second time, a default message is shown.
- The agent saves a short summary in the learner's tutoring history, including the concept, explanation level, and whether the learner answered the check question correctly. The Progress Agent uses this information later.

Explanations can be provided in three ways. Explanation 1 is simple and easy to understand. The sentences are short, using analogies and a step-by-step procedure. This method is used if the learner is at Easy stage or the concept appears on weak concepts list. Explanation 2 gives a complete explanation with one example, and is used at Medium stage. Explanation 3 gives concise explanation, deals with corner cases and makes connections to other concepts. It is used at Hard stage.

The explanation level changes dynamically within one tutoring session. Explanation level decreases by 1 if the learner displays confusion by asking the same question twice or by giving a wrong answer to a check question. The level increases by 1 after two consecutive right answers to check questions. When the learner has a recurring misconception about the concept, then that particular misconception is explained instead of the regular explanation.

The agent responds to problem questions in the following way:

- For an off-topic query, the agent politely redirects the learner to the session topic.
- For a vague query, the agent asks one clarification question before providing an explanation.
- If no relevant information is found, the agent gives a brief response, explains that the query is unrelated to the course material, and flags it for the instructor's review.
- If the language model service fails, the agent displays a clear error message while keeping the session history accessible.

5.4 Adaptive Quiz Difficulty Heuristic

The Quiz Agent chooses the difficulty of each question according to the recent answers of the learner. This way, the Quiz Agent ensures that each learner will be practicing questions that are sufficiently

challenging for them. A Streak-based approach is chosen as a simple method that is easy to understand for teachers and to implement quickly.

Each question in the question bank has one of three difficulty levels: Easy, Medium and Hard. Easy questions check the knowledge of the definition or fact and require a single step to solve. Medium questions apply a single concept over two steps. Hard questions combine the concepts and require multiple steps. Each question in the question bank has its level marked and is tagged with the tested concept. While calculating the score, Easy questions have a weight of 1, Medium has a weight of 2 and Hard has a weight of 3.

The following rules apply for switching difficulty:

- If a learner answers two consecutive questions correctly at the current level, the learner moves up to the next level. Hard is the highest difficulty level.
- If a learner has two consecutive incorrect answers at the current level, the learner moves down to the next lower level. Easy is the lowest level.
- In all other cases, the learner remains at the current level.
- The streak count is reset whenever the difficulty level changes.
- If the learner answers two questions incorrectly for the same concept in one quiz, that concept is marked as weak and sent to the Teacher Assistant Agent and teacher dashboard.

The quiz proceeds according to the following algorithm:

- The quiz starts at the learner's current difficulty level. Learners with no quiz history start at Medium.
- The agent selects an unanswered question at the current level, preferably covering a weak or previously untested concept. If no suitable question is available, the agent generates one using the language model or selects a question from the nearest difficulty level.
- The learner's answer is recorded as correct or incorrect, and the streak count is updated.
- If the learner answers two questions incorrectly for the same concept, that concept is marked as weak.
- The next difficulty level is selected according to the rules described above.
- Steps 2 to 5 are repeated until the quiz ends.
- The agent calculates raw accuracy and weighted score. The weighted score is calculated by dividing the total weights of correctly answered questions by the total weights of all attempted

questions.

- The results and final difficulty level are stored in the learner context and provided to the Progress Agent.

In a practical example, the learner begins a six-question quiz at Medium difficulty:

- Questions 1 and 2 (Medium, Concept A) are answered correctly, so the learner moves up to Hard.
- Questions 3 and 4 (Hard, Concept B) are answered incorrectly, so the learner moves down to Medium. Concept B is marked as weak because it was answered incorrectly twice.
- Question 5 (Medium, Concept C) is answered correctly.
- Question 6 (Medium, Concept A) is answered incorrectly, so the learner remains at Medium.

Accuracy for the learner at this stage stands at 50 percent (3 out of 6). The learner's weighted score is 42.9 percent: the learner scored 6 of 14 weighted points. The final level is still Medium. As concept B is labeled weak, the Tutor Agent will explain it more simply if the learner asks for clarification.

5.5 Progress Tracking and Performance Analysis

The Progress Agent studies the behavior of the learner and tries to identify patterns in the performance of the learner throughout the process. Instead of considering only the latest quiz result of the learner, it takes into account the results obtained from various quizzes, tutoring, problematic concepts, mistakes made, and collaborative activity. The aim is to identify the patterns developing in the performance of the learner and create an analysis of it. It is important to note that the Progress Agent does not suggest any actions for education to the teacher in its analysis. Its only job is to provide an analysis of the performance of the learner.

5.5.1 Performance Data Collection

Performance information is collected whenever a learner completes an activity that provides evidence about their understanding of a topic. During an adaptive quiz, the system records the questions attempted by the learner, the concept being tested, the question difficulty, whether the learner's answer was correct or incorrect, and the weight associated with the question. The Quiz Agent also maintains the correct and incorrect answer streaks required for adaptive difficulty adjustment.

After a quiz is completed, the following information is stored:

- Quiz accuracy
- Weighted quiz score

- Final quiz difficulty
- Performance for each attempted concept
- Concepts identified as weak
- Answers provided by the learner

Tutoring interactions also provide useful evidence about learner understanding. After a Tutor Agent interaction, the system records information such as:

- The concept for which help was requested
- The explanation level used
- Whether the learner showed repeated confusion
- Whether the learner answered the follow-up check question correctly
- A short summary of the tutoring interaction

Collaborative learning may provide additional supporting evidence. The Discussion Agent records AI hints that were given during collaborative activities, together with the subject and the level of the hint as well as the reasons why it was given. Whenever relevant, workspace events may be recorded. For example, multiple incorrect attempts in the same learning task could serve as evidence for future progress analysis.

Low-level user interaction events such as raw cursor positions, mouse movements, or similar information are not considered learning evidence and thus are not recorded. The Progress Agent performs the progress analysis after major events such as completing a quiz, collecting enough new interaction evidence, or when the system requests an updated perspective on the learner's progress.

5.5.2 Performance Pattern Analysis

The Progress Agent converts the collected performance information into measurable indicators that allow learner performance to be compared across concepts and over time.

For quiz q , the learner's **raw accuracy** is calculated as:

$$A_q = \frac{C_q}{N_q}$$

where:

- C_q = number of correctly answered questions in quiz q

- N_q = total number of attempted questions in quiz q

The **weighted quiz score** is calculated using the difficulty weights:

$$\text{Weighted Score} = \frac{\sum \text{weights of correctly answered questions}}{\sum \text{weights of all attempted questions}}$$

This allows questions of different difficulty levels to contribute differently to the learner's overall quiz performance. The learner's performance is also evaluated separately for each concept. The **accuracy for concept c** is calculated as:

$$T_c = \frac{C_c}{N_c}$$

where:

- C_c = number of correctly answered questions for concept c
- N_c = total number of attempted questions for concept c

Concepts are classified using the following thresholds:

- $T_c \geq 0.70$: Strong
- $T_c \leq 0.50$: Weak
- $0.50 < T_c < 0.70$: Developing / Average

A concept for which no assessment data exists is not classified because the absence of attempts should not be interpreted as a weakness.

The learner's **average accuracy across n completed quizzes** is calculated as:

$$A = \frac{1}{n} \sum_{q=1}^n A_q$$

where:

- A_q = accuracy obtained in quiz q .
- n = total number of completed quizzes.

To determine whether the learner's performance is improving or declining, the latest result is compared with the previous comparable result:

$$I = A_{\text{latest}} - A_{\text{previous}}$$

where:

- $I > 0$ indicates improvement

- $I < 0$ indicates decline
- $I = 0$ indicates no change

For the initial implementation, a difference of 0.10 is treated as the minimum meaningful change. An increase of at least 0.10 indicates an improving trend, while a decrease of at least 0.10 indicates a declining trend. Smaller changes are treated as stable performance. These thresholds are intentionally simple so that they can be understood by teachers and implemented consistently. They may be adjusted later if evaluation of the system shows that another threshold provides more meaningful results. The output of the analysis may therefore include information about concepts where the learner performs strongly, concepts requiring additional attention, improving or declining performance, current quiz difficulty, repeated tutoring requests, recurring mistakes, and relevant collaborative-learning signals. The complete analysis is then passed to the Teacher Assistant Agent.

5.5.3 Recurring Mistake Detection

A recurring mistake is an ongoing misconception or mistake seen in the actions of the learner recently. This is different from weak concepts since weak concepts are indicative of poor performance by the learner in a certain area, while recurring mistakes are associated with particular bad practices.

When a mistake is identified, the system records information including:

- Affected topic or concept
- Normalized mistake type
- Activity in which it occurred
- Timestamp
- Supporting assessment or interaction reference

For the initial implementation, the same normalized mistake is classified as recurring when it occurs at least three times within the learner's five most recent relevant interactions.

For example, if the learner repeatedly applies the same incorrect rule across several questions, those events are grouped under the same normalized mistake type. Once the defined threshold is reached, the Progress Agent flags the mistake as recurring. Information from tutoring and collaborative learning provides supplementary evidence. If the same question appears repeatedly within the discussion, or several hints point to the same difficulty, then it can be used to make an even stronger argument about the need for additional help.

When a recurring mistake is identified, the Progress Agent records:

- The affected concept
- The mistake type
- Number of occurrences
- Most recent occurrence
- Evidence supporting the finding

The detected pattern is then included in the performance analysis passed to the Teacher Assistant Agent.

5.6 Teacher Recommendation Methodology

It is the role of the Teacher Assistant Agent to use the results of the analysis done by the Progress Agent to come up with insights and recommendations that teachers can easily understand. The fact that the roles of the two agents are distinct and separated makes it easier for the teachers to make their decisions. This process begins with the activities of learners including quizzes, learning sessions, and collaborative activities. The next step is the Progress agent that will analyze these activities and provide a structured analysis of performance. This analysis is then provided to the Teacher Assistant Agent, which interprets it and provides recommendations.

The Teacher Assistant Agent may receive information including concept-level accuracy, weak concepts, strong concepts, performance trends, recurring mistakes, recent quiz performance, tutoring interaction history, and relevant collaborative-learning activity. The agent interprets this information to determine whether the learner may benefit from additional learning support.

Possible recommendations include:

Table 5.1: Teacher Assistant Agent Recommendations

Observed Condition	Recommendation
Concept accuracy is 0.50 or below	Revise the concept using additional explanation
Concept accuracy is between 0.50 and 0.70	Provide additional practice
A recurring mistake is detected	Provide targeted tutoring focused on the misconception
Performance continues to decline after additional support	Recommend direct teacher attention

Observed Condition	Recommendation
Repeated Tutor Agent support is required for the same concept	Provide additional explanation or practice
Concept accuracy is at least 0.70 with no re-occurring difficulty	Continue to more advanced practice
A common weakness is found among several learners	Consider class-level revision or collaborative activity

The agent interprets this information and determines whether the learner may benefit from additional learning support.

Each recommendation should include the learner identifier, affected concept or topic, relevant analysis, supporting evidence, suggested learning action, and recommendation priority.

For example, a recommendation may state that a learner is showing persistent difficulty with a particular concept and suggest additional Tutor Agent support. The recommendation should also display the evidence that caused it to be generated, such as low concept accuracy, repeated mistakes, or repeated requests for tutoring assistance.

5.6.1 Teacher Review

The Teacher Assistant Agent presents each recommendation to the teacher along with the relevant analysis and supporting evidence. The teacher has two available actions: approve or dismiss the recommendation.

5.6.1.1 Approve

The teacher accepts the recommendation without modification. Once approved, the recommendation becomes an actionable instruction and is passed through the LangGraph router to the relevant agent. For example, an approved recommendation for additional explanation goes to the Tutor agent.

After the approved learning activity is completed, the resulting learner performance data is collected and analyzed again. This creates a continuous cycle in which the Progress Agent evaluates learner activity, the Teacher Assistant Agent interprets the analysis and provides a recommendation, and the teacher reviews and approves the proposed action.

5.6.1.2 Dismiss

If the teacher dismisses a recommendation, no automatic learning activity is generated from it. The recommendation remains recorded on the teacher's dashboard for reference but it does not affect the learner's current activities.

This design keeps the teacher in control of learning interventions while reducing the amount of learner data that needs to be reviewed manually. The Teacher Assistant Agent identifies relevant information and provides recommendations supported by evidence, but the final decision to conduct the proposed learning activity remains with the teacher.

5.7 Score-Based Collaborative Group Formation

The Teacher Assistant Agent forms small groups of learners, when the teacher requests them, based on the performance information stored in the learner context. The teacher approves or edits the proposed groups, and the approved groups are then handed to the Discussion Agent. The goal is to form groups where learners can help each other solve problems and understand difficult concepts. Two grouping strategies are used: balanced grouping, which distributes learners with different overall performance levels across groups, and strength-based grouping, which brings together learners whose strengths can complement each other's weaknesses.

5.7.1 Learner Performance Representation

For group formation, each learner is represented using a per-topic performance vector and an overall performance score. The per-topic vector contains the learner's accuracy for each topic for which sufficient quiz data is available.

A learner's topic scores are represented as:

$$P_i = [p_{i1}, p_{i2}, \dots, p_{in}]$$

where p_{ij} is the accuracy of learner i on topic j , calculated as:

$$p_{ij} = \frac{\text{Correct answers on topic } j}{\text{Attempted answers on topic } j}$$

The overall score across all attempted questions is calculated as:

$$O_i = \frac{\text{Total correct answers}}{\text{Total attempted answers}}$$

A topic is only included when the student has attempted questions on that topic. This prevents a topic with no available performance data from being treated as a weakness.

5.7.2 Balanced Complementarity

Balanced grouping distributes learners with different overall performance levels across the groups. This helps prevent a group from containing only high-performing or only low-performing learners.

For a group G_k , its average overall score is calculated as:

$$A_k = \frac{\sum_{i \in G_k} O_i}{|G_k|}$$

where O_i is the overall score of learner i and $|G_k|$ is the number of learners in group G_k .

To measure how evenly the groups are distributed, the difference between the highest and lowest group averages is calculated as:

$$B = \max(A_1, A_2, \dots, A_m) - \min(A_1, A_2, \dots, A_m)$$

A smaller value of B indicates that the groups have more similar average performance.

To create the groups, students are first sorted from highest to lowest according to their overall score. Each student is then placed into the group with the lowest current total score, provided that the group has not reached the selected group size. If the group is full, the next suitable group is selected. This provides a simple way to distribute learners with different performance levels across the groups.

5.7.3 Strength-Based Complementarity

Strength-based grouping focuses on matching students whose strong topics can help with another student's weak topics. A topic is considered a strength when the student's accuracy is 0.70 or higher, and a weakness when the accuracy is 0.50 or lower. Scores between 0.50 and 0.70 are treated as average and are not counted as either a strength or a weakness.

For two students, complementarity is measured by counting the topics where one student is strong and the other student is weak. A higher count indicates a greater match between their strengths and weaknesses.

For example, if Student A is strong in Programming but weak in Databases, while Student B is strong in Databases but weak in Programming, their strengths complement each other in both topics.

For groups with more than two students, the same idea is extended by adding the complementarity scores

between each pair of students in the group. A higher total indicates more opportunities for students to help each other in their weaker areas.

5.7.4 Group Formation Procedure

The group formation process requires three inputs: the list of students, the desired group size, and the chosen strategy, either balanced or strength-based. The process starts when the teacher requests groups formation from the Teacher Assistant Agent. The Teacher Assistant Agent proposes the groups, and the teacher decides whether to approve or edit them.

- The teacher requests groups, selects the group size and chooses the balanced or strength-based strategy.
- The Teacher Assistant Agent fetches each learner's performance information from the learner context, including the topic-level accuracy supplied by the Progress Agent.
- The overall score and topic-level accuracy are computed for each student.
- For balanced grouping, students are sorted by overall score and placed one by one into the group with the lowest current total score while keeping the selected group size.
- For strength-based grouping, students are matched according to complementary strong and weak topics while keeping the selected group size.
- If the number of learners does not divide evenly by the selected group size, an additional group is created for the remaining students. The selected group size is not exceeded.
- If two groups have the same selection score, the group with fewer students is chosen first.
- The proposed groups are shown to the teacher on the dashboard. The teacher can approve them, or edit them by moving learners between groups.
- Once the teacher approves the groups, they are saved and handed over to the Discussion Agent for the group activity.

Worked Example: Suppose there are six learners with overall scores of 0.90, 0.82, 0.75, 0.60, 0.48, and 0.40, and the selected group size is three. Using balanced grouping, the students are sorted from highest to lowest and then distributed so that each new student is placed into the group with the lowest current total score.

Group 1: 0.90, 0.60, 0.48

Group 2: 0.82, 0.75, 0.40

$$A_1 = \frac{0.90 + 0.60 + 0.48}{3} = 0.66$$

$$A_2 = \frac{0.82 + 0.75 + 0.40}{3} \approx 0.66$$

The two groups therefore have nearly identical average performance while including learners with different performance levels rather than forming groups containing only strong or only weak performers.

5.8 Discussion Hint Generation

The hints provided by the Discussion Agent allow the group to continue discussing the subject without revealing the answer. According to previous literature, highly responsive agents can negatively affect discussion among students [8]. Therefore, hints are designed to encourage group members to discuss their reasoning with each other rather than rely on the agent [5].

There are four ways in which a hint may be activated:

- A group member clicks the hint button.
- No message has been posted in the group chat for three minutes while the problem is still open.
- The last five messages are not related to the problem.
- The discussion reflects a misconception about the concept of the problem.

The first activation is initiated by the learners. The other three are proactive hints given by the Discussion Agent. Proactive hints are limited to one every five minutes and are never posted while the group is discussing the problem correctly.

A hint is provided based on the following procedure:

- The agent analyzes the latest messages to determine the current stage reached by the group or the misconception revealed during the discussion.
- The first hint provided by the agent for a particular problem is Level 1. Each subsequent hint for the same problem moves to the next level, up to Level 3. The hint level is reset when the group proceeds to a new problem.
- The language model generates a hint based on the current discussion. The hint must not provide the answer or direct solution steps.
- The generated hint is compared with the hidden solution to the problem. If the hint contains the

answer, it is generated again. If the hint fails the check again, a more general prompt is used, such as asking group members to explain their ideas about the next step in the solution process.

- The hint is posted in the chat window and marked as an AI hint. The Discussion Agent records the hint in the shared state, and the Progress Agent reads this record for later analysis.

There are three types of hints. A Level 1 hint encourages participants to think by asking them to clarify what the problem is about. A Level 2 hint refers to the relevant key concept or rule. A Level 3 hint provides a general structure for a step without giving its solution, such as asking participants to consider two separate cases.

Hints are always directed at the whole group and never refer to any particular participant's weakness.

5.9 Agent Coordination and Information Handoff

The five agents are coordinated using LangGraph. The agents do not communicate with each other directly. Instead, each agent is represented as a node in the graph that reads from and writes to the same shared state, which is the learner context.

The graph consists of a router node, the Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, and Discussion Agent nodes, along with a Teacher Review Node where the graph pauses until the teacher responds.

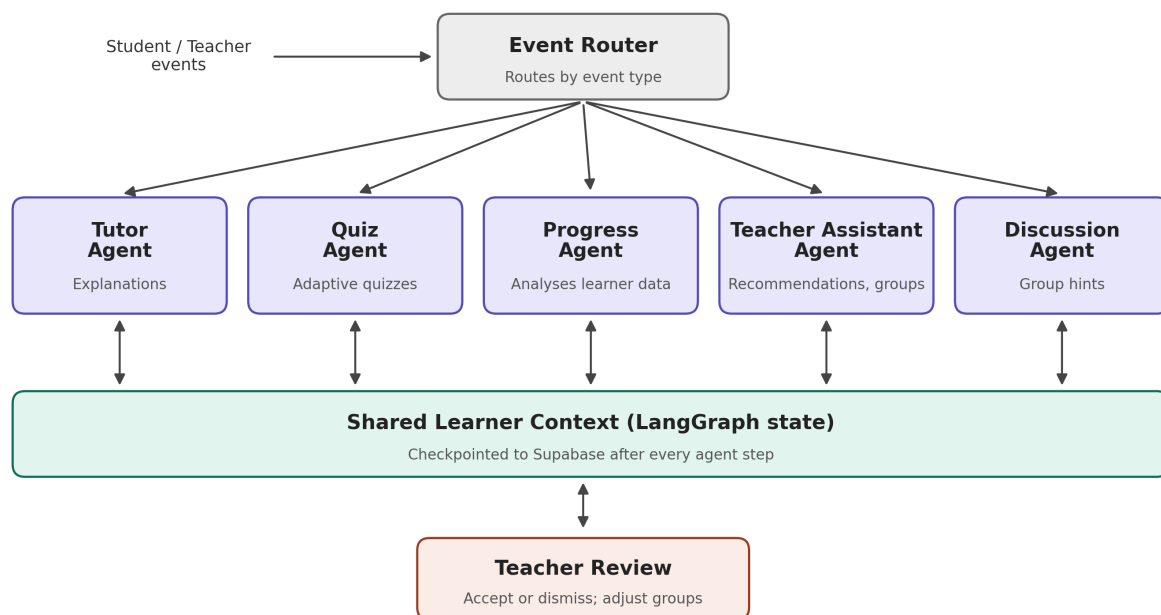


Figure 5.1: Agent Coordination and Information Handoff

The router operates according to the following rules:

- When a learner asks a tutoring question, the request is routed to the Tutor Agent.
- When a learner answers a quiz question, the answer is routed to the Quiz Agent, which evaluates it and provides the next question.
- When a quiz is completed, the Quiz Agent saves the quiz results in the shared state. The Progress Agent then analyzes the results, after which the Teacher Assistant Agent generates teacher-facing recommendations and the graph moves to teacher review.
- When the teacher ends a session, the Quiz Agent creates a draft quiz on the topic of that session, which the teacher reviews and approves before it is made available to students.
- When the teacher requests groups, the request is sent to the Teacher Assistant Agent, which forms the groups.
- When a hint is requested or a hint condition is triggered, the request is routed to the Discussion Agent.
- When the teacher approves a recommendation, the graph resumes, and the resulting action is sent to the Tutor Agent, the Quiz Agent or the Teacher Assistant Agent, depending on the action. When the teacher dismisses a recommendation, no action is taken.

Information is passed between agents through the shared state:

- Upon completion of a quiz, the Quiz Agent stores the quiz performance, answers, and difficulty levels for analysis by the Progress Agent.
- When a concept is identified as weak, the Quiz Agent records the weak concept in the shared state so that it can be used by the Tutor Agent for targeted tutoring.
- After each tutoring session, the Tutor Agent stores the tutoring history, concepts asked about, and identified instances of confusion for analysis by the Progress Agent.
- After each hint, the Discussion Agent stores the hint log for analysis by the Progress Agent.
- During group formation, the topic-level accuracy stored in the shared state is used by the Teacher Assistant Agent to form groups.
- After analyzing learner performance, the Progress Agent stores its findings in the shared state for the Teacher Assistant Agent.
- The Teacher Assistant Agent interprets the findings and stores teacher-facing insights and recommendations together with their supporting evidence.

- Once the teacher approves an action, it is handed over to the Quiz Agent or Tutor Agent, depending on which agent the action concerns. Approved groups are handed over to the Discussion Agent.

Three rules ensure consistent coordination:

- Each field in the shared state is updated by only one agent, while other agents can read it. Fields that require contributions from multiple agents, such as weak concepts, accept new entries from the responsible agents. Each entry is tagged with its source so that one agent cannot overwrite another agent's data.
- Recommendations from the Teacher Assistant Agent do not trigger any learning activity until the teacher reviews them. The teacher can approve or dismiss each recommendation. If a recommendation conflicts with an automatic action from another agent, the teacher's decision takes priority.
- The output of each agent is validated against the required format before it is saved to the shared state.

Failures are handled as follows:

- If a language model request fails or takes longer than 20 seconds, the request is retried twice. If it still fails, an error message is shown and the affected agent is marked as unavailable.
- If one agent becomes unavailable, other independent activities can continue where possible. For example, a learner can still take a quiz if the Progress Agent is unavailable, and the performance analysis can be performed later using the saved quiz data.
- If the teacher does not respond to a recommendation, the recommendation remains pending on the dashboard and no automatic action is taken.

5.10 End-to-End Learning Workflow

The previous sections have described the methodology of each individual agent. This section integrates these methods into a complete learning cycle. It shows how the teacher starts a learning session by uploading learning material, how the five agents contribute at different stages, and how learner information is updated in the shared state for use in later activities.

- The teacher logs in, uploads the learning material, and starts a learning session by selecting the topic.
- The Tutor Agent conducts the tutoring session using the uploaded learning material and the relevant learner context.

- If a learner asks a question related to the concept during or after the lecture, the Tutor Agent checks whether the question is related to the current topic. It then uses the learner's background and the uploaded learning material to generate an explanation at an appropriate difficulty level.
- When the teacher ends the learning session, the Quiz Agent creates a draft quiz on the topic that was just taught. The teacher reviews and approves the quiz before it is made available to students. The quiz begins at the learner's difficulty level.
- The learner takes the quiz. After each answer, the Quiz Agent records whether it is correct or incorrect, updates the difficulty according to the defined streak rules, and tracks repeated mistakes to identify weak concepts.
- After the quiz is completed, the quiz results, answers, difficulty information, and identified weak concepts are stored in the learner context for analysis by the Progress Agent.
- The Progress Agent analyzes the learner's performance to identify performance patterns, weak concepts, and recurring mistakes.
- The Teacher Assistant Agent uses the findings produced by the Progress Agent to generate evidence-based recommendations for the teacher. These recommendations may suggest actions such as additional practice or revision of a weak topic.
- The teacher can approve or dismiss a recommendation. An approved action is passed to the relevant agent for further learning activity.
- When collaborative learning is needed, the teacher asks the Teacher Assistant Agent for group formation.
- Learners discuss the problems within their groups. When a learner requests help or a defined discussion condition triggers a hint, the Discussion Agent provides a topic-specific hint.
- Hint information is logged in the shared state for subsequent analysis by the Progress Agent.
- The updated learner context remains available so that relevant information from one learning activity can be used by the Tutor, Quiz, Progress, and Discussion Agents in future activities.

5.11 Conclusion

The proposed methodology brings five agents together to support the learning process. The Tutor Agent provides explanations using the uploaded learning material and relevant learner information. The Quiz Agent adjusts quiz difficulty based on learner performance and identifies concepts that may require additional attention. The Progress Agent analyzes learner performance and identifies patterns, weak areas,

and recurring mistakes, while the Teacher Assistant Agent converts these findings into recommendations for the teacher and proposes study groups for the teacher to approve or edit. The Discussion Agent gives hints when the groups discuss the subject together. LangGraph coordinates these agents through the shared learner context, allowing relevant information from one learning activity to support later activities while keeping the teacher involved in important learning decisions.

Chapter 6 High-Level and Low-Level Design

The process of designing InSync was about transforming an abstract concept into a real system, where all activities, namely tutoring, assessment, progress monitoring, and group discussion, are to be done in collaboration with each other and learn from each other. The five agents share a common understanding of each student because learners and teachers use one web platform and all actions are stored in a single database. At all times, the teacher has the control over the main decisions. This chapter is devoted to the description of the structure and functioning of all the components. It is to consider all the constraints and considerations of the design, architectural decisions and alternatives, and policy regarding the development of the system.

6.1 System Overview

This chapter describes the architecture of InSync at both high and low levels. It discusses how the system is designed and the decisions made to reach such a design, as well as policies for implementing the system. This introductory section provides an overview of the system prior to discussing the design itself.

6.1.1 Functional Overview

InSync is a web-based learning platform that integrates tutoring, testing, progress tracking and peer discussions into an uninterrupted learning process. The students are able to pose questions related to the Programming Fundamentals topic being covered, do quizzes that adapt to their level of knowledge, track their own progress and collaborate on problem solving with other learners in discussion groups. The teachers are able to create classes, load learning materials, start and finish learning sessions, monitor the progress of their class and analyze the recommendations provided by the system. This functionality is delivered by five AI agents and the teacher dashboard:

- Tutor Agent provides explanations and answers to the learners' questions based on the learning material prepared by the teacher at the adequate level of knowledge.
- Quiz Agent generates topic-specific quizzes and adjusts the level of the quiz questions based on the learners' responses.
- Progress Agent evaluates the learners' performance and identifies the gaps, patterns of learning and repeated mistakes.
- Teacher Assistant Agent turns the results of the analysis performed by the Progress Agent into actionable recommendations for the teacher, displays them on the dashboard along with the ev-

idence, and delivers approved actions to the appropriate agent. It also forms study groups when the teacher requests them and presents the groups to the teacher to approve or edit.

- Discussion Agent provides hints during group discussions without providing the answers.
- Teacher Dashboard provides information about the state of the class, the recommendations made and the proposed study groups, so that the teacher can approve or dismiss the recommendations and approve or edit the groups.

Considering the scope of work in Chapter 2, the platform was developed for Programming Fundamentals course only, works in a web browser only, and provides the teacher with total control over all major decisions.

6.1.2 Basic Design Approach

InSync consists of three major elements:

- The web client, which is used by both students and teachers to log in, manage classes, take part in learning activities and view dashboards.
- The FastAPI backend, which contains the application services (login, classes, sessions, quizzes, materials and dashboard data) and the agent service where the five agents operate. The web client sends a request to the backend every time AI support is required.
- A single PostgreSQL database, hosted on Supabase, where all information about learners, classes and activities is stored.

The major assumption of the design is the fact that these agents do not operate individually. They share one and the same record about each learner, namely the shared learner context.

- The agents make any records about their findings concerning the learners in the shared learner context. For example, it may be some weak concepts detected through a quiz or questions raised in a tutoring session.
- The other agents review the same record prior to making their actions.
- As a consequence, difficulties discovered in one activity influence the explanation of the next, the questions asked in the quiz, suggestions for the teacher, as well as discussion groups creation.

Without this shared record, InSync will be just five individual tools and this is what the project wants to avoid.

6.2 Design Considerations

There were some concerns that needed to be sorted out before settling on the InSync design:

- Some are beyond the control of the team, including the external services the system relies on.
- Some involve time and money constraints and testing.
- Some include guidelines the team decided to use.

These issues will be discussed in four sections in this section: Assumptions and Dependencies, General Constraints, Goals and Guidelines, and Development Methods used.

6.2.1 Assumptions and Dependencies

6.2.1.1 Software and Services

InSync depends on the following software and services:

- Supabase: provides the database, user login and file storage for uploaded material.
- Redis and FastAPI WebSockets: provide real-time group chat, presence and editing locks
- Large language model API: generates explanations, quiz questions, hints and recommendation text.
- Embedding model: allows the Tutor Agent to find the parts of the learning material relevant to a question.
- LangGraph: coordinates the five agents.

If any of these services changes its pricing, limits or availability, the features that rely on it will be affected. For this reason, the design allows the AI provider to be replaced.

6.2.1.2 Operating environment

The system is intended for usage in modern web browsers such as Chrome, Edge and Firefox on laptops and desktops. The system will not have a mobile app, and it is supposed that the users will have a stable internet connection. The users are assumed to be:

- Students and Teachers who have basic computer skills but no prior experience with AI-based learning applications.
- English speakers.
- Teachers who provide correct learning material and act as the final judges of the system's recom-

mendations.

- Users who consent to the collection of their learning data within the platform.

6.2.1.3 Learning material

Learning materials and initial sets of questions for the Programming Fundamentals should be developed by the team prior to evaluation, as they affect the quality of tutoring and quizzes. The following can be expected to change during or after the project:

- The provider and AI model can be changed if needed due to cost or quality concerns.
- Additional subjects might be added in the future.
- The rules used inside each agent can be refined during development.

These changes have been accounted for in the design through the independence of agents and separation of subject matter from the coordination algorithm.

6.2.2 General Constraints

The development of InSync is affected by the following constraints that directly influence the implementation of the system.

- AI requests are expensive and rate limited and Supabase is operating in a free or low-cost plan, hence AI requests and storage are minimized.
- AI response takes several seconds, and hence long tasks run asynchronously with feedback to the interface.
- Learner data is confidential and partially exposed to external AI services, hence role-based access and sharing of only minimum data.
- InSync does not have any integration with the university or other third parties, and thus, learner data comes from internal platform only.
- Testing is conducted only on Programming Fundamentals and limited number of users, hence usage of sample data and test sessions for that subject only.
- AI output may not always be correct and may be inappropriate, hence agents will confine to their roles, check output, and important actions should get teacher approval.

6.2.3 Goals and Guidelines

These guidelines govern the design of InSync:

- Simple and comprehensible rule systems are preferred to complex systems, thus speeding up development and making it easier for teachers to understand the system.
- No significant action, such as implementing a recommendation or confirming a study group, takes place without the teacher's consent, because recommendations from AI may be incorrect.
- An erroneous or late reply from an AI system should not result in any data loss or unavailability of the platform.
- Coordination among agents is not dependent on the content of the subjects, thus allowing for InSync expansion to other subjects without a need to change the design.
- The agents help both the students and the teachers but never replace their decisions, because very responsive AI may decrease interactions among learners [8].

6.2.4 Development Methods

In developing the InSync software, the Scrum method will be adopted. The Scrum approach is an agile software development framework in which software is developed through repeated iterations known as sprints. The software will be split into various sprints, each lasting for two weeks. Each sprint will result in a functional piece of the software at the end of the period. At the end of each sprint, there will be short sessions that will take place between 15 to 20 minutes to reflect on the progress, identify problems and modify schedules if required. There will also be regular consultations with the project supervisor during the development process.

The Scrum approach will suit this project perfectly since it is highly flexible and allows for adjustments of existing software features or creation of new ones during development. This approach is ideal in developing artificial intelligence software due to multiple modifications that are required to achieve the desired results.

6.3 System Architecture

InSync follows a layered and modular architecture in which the user interface, application logic, AI-agent coordination, and data management are separated according to their responsibilities. The platform uses a React-based web client for student and teacher interaction, a FastAPI backend for application services and agent execution, LangGraph for coordinating the AI agents, and PostgreSQL for persistent relational and learner data. PostgreSQL is hosted through Supabase, while pgvector is used in the same database for storing and retrieving embeddings of teacher-provided learning material.

The architecture contains four logical subsystems: the Web Client, Application Backend, Agent Service,

and Data and Retrieval subsystem. Although the Application Backend and Agent Service are shown separately to clarify their responsibilities, both are implemented within the FastAPI based backend rather than as two separate services while maintaining a clear separation between ordinary application functionality and AI-agent orchestration.

The five AI agents used by InSync are the Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, and Discussion Agent. These agents are coordinated through LangGraph using a shared learner state rather than directly calling one another. External language model and embedding APIs are accessed through provider-independent wrappers.

The interfaces for students and teachers are provided by the Web Client. Students use the Web Client to access learning resources, communicate with the Tutor Agent, take quizzes, follow their progress, and engage in collaborative group activities. Teachers use it to manage their classes and learning resources, create activities, monitor learners, analyze recommendations made by the Teacher Assistant Agent, and watch how groups collaborate.

The application backend is implemented using FastAPI. It provides REST APIs for application operations such as class management, student membership, learning sessions, quizzes, learning resources, study groups, progress information, recommendations, and teacher decisions. User authentication is handled through Supabase Authentication, while FastAPI validates authenticated requests and enforces the appropriate student and teacher permissions. FastAPI WebSockets are used for real-time features such as group communication, collaborative workspace updates, and teacher monitoring.

The Agent Service is a logical subsystem in the FastAPI backend. LangGraph controls five agents via an event router and shared learner state. Those agents access language-model capabilities with an LLM client wrapper and course information via a retrieval component.

The Data and Retrieval Subsystem is responsible for managing both application data and AI-related data. Users, classes, quizzes, learner context, recommendations, groups, and other relational data are stored in PostgreSQL. Vector embeddings for retrieval-augmented generation are stored in pgvector. Learning resources are uploaded to file storage.

6.3.1 Subsystem Architecture

The overall InSync architecture can be divided into four logical subsystems: the Web Client, Application Backend, Agent Service, and Data and Retrieval subsystem. Each subsystem is responsible for a distinct part of the platform while communicating through clearly defined interfaces. The following subsections describe these components in greater detail and show how the AI and application layers work together without tightly coupling their internal implementations.

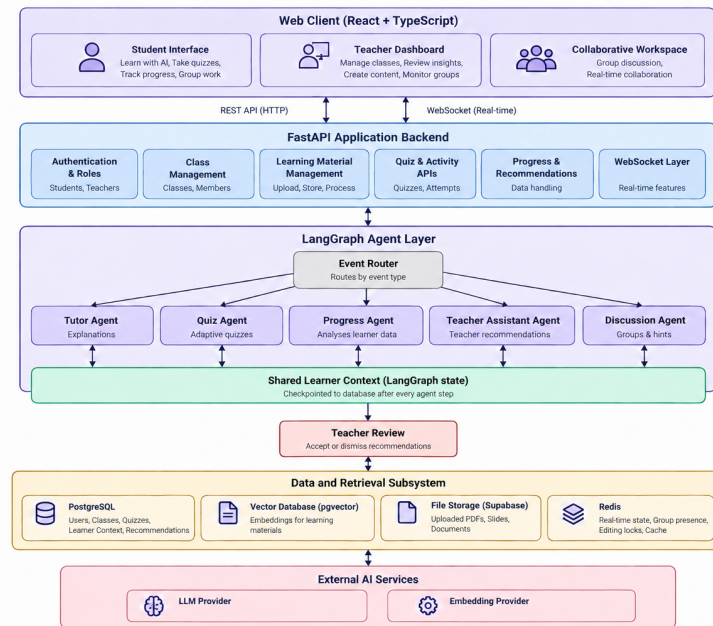


Figure 6.1: Overall architecture of the InSync system

6.3.1.1 Web Client Subsystem

The web client is the part of InSync that students and teachers access directly through a web browser. It uses the same client but displays different features based on the role of the logged in user. Student views include the tutoring interface, adaptive quiz interface, personal progress and feedback views and the group workspace for collaborative discussion. Teacher views include the Class dashboard, Recommendation review screen, Group review screen (to approve or edit proposed groups), Session management (start/end of learning session), and Material upload screen.

For most actions, such as logging in, starting a quiz or viewing progress, the client sends REST requests to the application backend. The client sends a request, the backend processes it, and the response updates the screen. The group workspace also requires real-time communication so that new chat messages, Discussion Agent hints, collaborative workspace updates, group presence information, and teacher monitoring updates can be delivered without refreshing the page. InSync uses FastAPI WebSockets for communication between the web client and backend. Redis is used to maintain temporary real-time information such as active user presence, collaboration events, and controlled-editing locks, while persistent group and message data is stored in PostgreSQL.

6.3.1.2 Application Backend Subsystem

The Application Backend is implemented using FastAPI and acts as the main server-side layer of InSync. It receives requests from the React web client, validates authenticated users and their permissions, performs application-level operations, communicates with the database, and invokes the Agent Service

when an AI-supported activity is required.

The backend provides REST endpoints for class management, class membership, topics, learning sessions, uploaded learning materials, quizzes, learner progress, recommendations, study groups, and collaborative activities. Authentication identities are obtained through Supabase Authentication, while FastAPI applies role-based authorization so that students and teachers can access only the operations permitted for their roles.

FastAPI also provides WebSocket endpoints for functionality that requires real-time communication. These connections are used for group discussion, collaborative workspace events, live activity updates, and teacher observation. Redis maintains temporary real-time information such as user presence and controlled-editing locks, whereas permanent application data is stored in PostgreSQL.

The backend communicates with the Agent Service through internal service interfaces rather than exposing individual agents directly to the client. This separation allows the web client to interact with consistent application APIs while LangGraph manages the internal AI workflow.

6.3.1.3 Agent Service Subsystem

Agent Service is responsible for controlling the AI assisted learning abilities of InSync. Agent Service operates as a module integrated into the FastAPI backend and utilizes LangGraph to control agent execution.

The following are the main elements of the Agent Service: Event Router, Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, Discussion Agent, Teacher Review interrupt, LLM Client, and Retriever.

Event Router analyzes the triggering event that initiates the agent execution and defines which agent should be executed. A learner question is sent to the Tutor Agent, a quiz answer to the Quiz Agent, and a completed quiz to the Progress Agent. A request from the teacher for study groups is sent to the Teacher Assistant Agent, and group discussion events and hint requests are sent to the Discussion Agent. In case of meaningful patterns detected by the Progress Agent, the structured analysis results are passed to the Teacher Assistant Agent, where they are translated into educational suggestions to teachers.

Teacher Review interrupt is the human in the loop interruption. Recommendations and proposed groups are held until the teacher reviews them. Recommendations can be approved or dismissed, and proposed groups can be approved or edited. Approved recommendations are returned to the router and forwarded to the appropriate agent for execution. Proposed study groups are activated after teacher approval, while edited group proposals are updated according to the teacher's changes before the collaborative activity begins.

The agents do not communicate with one another directly. Instead, they coordinate by reading from and

writing to the shared state represented by `InSyncState`. Such an approach enables the information generated by one task to be available to another without making any dependence between the agent types. For example, once the Quiz Agent has found out the weak concept, it writes it to the learner context, and later the Tutor Agent can read the weak concept to determine what level of explanation is necessary. Likewise, the Progress Agent collects the information from the quizzes, tutoring, and collaboration to inform the Teacher Assistant Agent.

6.3.1.4 Data and Retrieval Subsystem

The Data and Retrieval subsystem handles both the persistent application data and data needed for the retrieval-augmented AI response generation process. Structured data including users, classes, topics, quiz attempts, context of learners, study groups, recommendations, and teachers' decisions are kept in PostgreSQL.

Embeddings derived from learning material provided by the teacher are stored via the `pgvector` extension for PostgreSQL. Using only one database for relational data and vector embeddings will minimize the number of separate storage solutions that should be managed, protected, and configured.

All uploaded PDFs, presentations, documents, and other files related to a class are stored using Supabase Storage or some other object storage solution.

LangGraph checkpoints preserve the relevant workflow state so that an interrupted agent execution can be restored where appropriate. Redis is used for short-lived real-time information such as group presence and temporary controlled-editing locks rather than permanent learner records.

6.3.2 Material Processing Pipeline

When a teacher uploads learning material, the original file is first stored. The system then extracts textual content, divides it into smaller chunks, generates an embedding for each chunk, and stores those embeddings using `pgvector`. The original source and chunk metadata are retained so that retrieved information can be traced back to the teacher-provided material. Question asked by the student during tutoring process is converted into retrieval task. This system looks for the most appropriate chunks associated with the current course and subject, and provides only this retrieved content as factual knowledge background to the Tutor Agent.

6.4 Architectural Strategies

The following section gives an overview of the primary architectural design decisions behind InSync architecture: coordination of five agents, selection of a database system, web backend vs. agent service

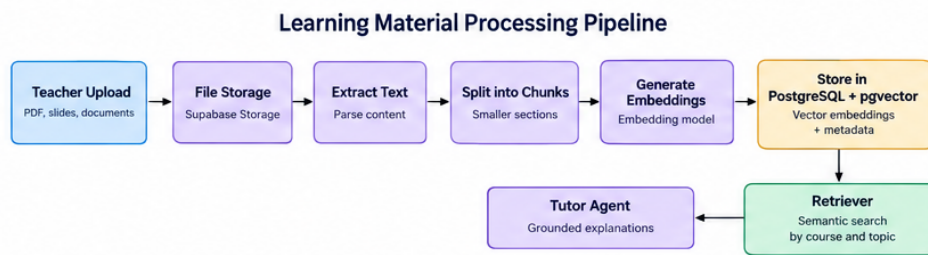


Figure 6.2: This figure represents the material processing pipeline of the system

separation, teacher-in-the-loop, grounding of answers in the teacher’s materials, error handling, real-time interaction, background processing, and responsible data handling. Every decision in this list is presented in the same manner: what was decided, what other options were under consideration, and why the particular decision was made.

6.4.1 Shared-State Agent Coordination with LangGraph

The InSync system uses the shared-state coordination approach, where the AI agents use a shared learner state for communication rather than contacting each other directly. The LangGraph is responsible for defining the agent nodes, routing rules, workflow state, and any teacher-review interruptions.

The shared state contains important information, including topic, level of difficulty of quizzes, tutoring history, performance on the topic, weak points, common mistakes, group information, hints, analysis from Progress Agent, recommendations from Teacher Assistant, and decision from teacher. Every agent accesses only necessary information and writes in its own part of the shared state.

This approach was selected since the InSync system required traceability of information flows and control over which agent executes after specific events.

Direct agent-to-agent calls was considered but discarded because of increased dependency between agents and difficulty of monitoring information flow. However, despite the consideration of CrewAI and AutoGen as possible systems for coordinating agents, the InSync system requires clear routing, constant common state, and clearly specified points of possible interruption by a teacher. Therefore, LangGraph was selected because it matches the planned learning cycle in InSync using a graph structure.

The selected solution provides an opportunity for future extensions as well. A new agent can be simply integrated into a graph as a new node related to appropriate events and state fields without re-designing all other agents.

6.4.2 Single PostgreSQL Database with pgvector

InSync uses PostgreSQL as the primary storage database and takes advantage of the pgvector extension for semantic search. That allows not only storing structured application data but also embeddings for the learning materials within one database system.

InSync has a lot of explicit relations in the relational data. A student belongs to classes, participates in quizzes, has quiz attempts and learner context, belongs to study groups, and may have multiple recommendations and teacher decisions. PostgreSQL is therefore suitable for enforcing these relationships and performing the required joins.

For the Tutor Agent, teacher-provided materials need semantic search as well. Instead of implementing a vector database separately, embeddings of such materials are stored via pgvector.

Separate vector database such as Pinecone or Chroma was explored but not implemented at first because it would bring an additional database which requires its own setup, access management, backup, and synchronization with the relational database. The MongoDB database was explored but not adopted because most of InSync's core data is strongly relational. The application enjoys clearly articulated relationships between students, classes, topics, quizzes, attempts, and recommendations.

That is why it made sense to rely on PostgreSQL with pgvector as persistence is simple and both relational queries and semantic search can be supported.

6.4.3 Separation of the Web Backend and the Agent Service

InSync uses a single FastAPI backend with two logical subsystems: the Application Backend and the Agent Service. The Application Backend is responsible for the core operations of the platform like authentication, classes, sessions, learning materials, quizzes, progress records and teacher decisions. LangGraph coordinates the five AI agents and the Agent Service handles AI-related tasks.

The two subsystems are separated at the software level since they have different responsibilities. The Agent Service is responsible for agent execution, learner state updating, retrieval and requests to language model APIs, and the Application Backend manages normal application operations and database access. This separation allows for better organization of the code without the need for deploying two distinct backend services.

There was a thought to have a single combined implementation without this logical separation, but this would make the application and AI related code more tightly coupled, and harder to maintain. In contrast, the deployment of each AI agent as a separate service was considered to be unnecessary for the proof-of-concept as it would introduce additional communication and deployment complexity. Also, having the five agents inside the Agent Service helps them coordinate through LangGraph and the shared learner state.

The primary advantage of this approach is that it enables the system to be easily deployable while clearly separating the roles of application processing and the coordination of AI agents. In a future version, if needed, the Agent Service can be broken out into its own deployment, without changing the overall responsibilities of the agent.

6.4.4 Teacher-in-the-Loop Control

The InSync follows a teacher-in-the-loop methodology. Agents can propose actions but important actions happen only after a teacher confirms them. The Teacher Assistant Agent plays a key role in this methodology. First, the Progress Agent analyzes the learner's performance and the Teacher Assistant Agent translates the results of the analysis into recommendations. The recommendations are pending on the dashboard until the teacher approves or dismisses them. Only approved recommendations will lead to a learning activity for the student. Dismissed recommendations will not cause any actions.

The same concept applies to other parts of the system. The groups proposed by the Teacher Assistant Agent will be presented to the teacher, who can approve or edit them. Questions generated by AI need to be confirmed before they are inserted in the assigned quiz. Automatic changes will be made when there is no risk involved, for instance the level of explanation or the difficulty of the next question in a quiz. Such changes will have fixed boundaries and will always be visible to the teacher.

Such an approach has been selected because, while the results generated by the AI may be inaccurate or not have enough information, the teachers themselves are responsible for all decisions which have an influence on their students' education. This decision is supported by the literature analyzed in earlier Chapters. FACET engages the teachers in evaluating the materials generated by the AI [2], Khanmigo combines AI tutoring with teacher-facing assistance [9], and AIA-PAL utilizes structured human supervision to avoid the risks of any unsuitable responses [4].

Two other approaches have been considered. Automated agents were dismissed since the possibility of errors in reaching the learners would remain and an autonomous behavior without teacher approval is out of scope of this project. Manual mode without any recommendations of AI was dismissed since it would not resolve the problem of the teachers' inability to monitor every single learner in a class.

The key drawback of the approach selected is that some actions take longer because they are waiting for the teacher's approval. In order to minimize this load on the teacher, the Teacher Assistant Agent shows all recommendations together with the supporting information, for example, quiz results and repeated mistakes, so that the teacher can decide quickly.

6.4.5 Grounded Generation through Retrieval

The Tutor Agent relies on material that has been uploaded to the Tutor Agent by the teacher in the process of retrieval-augmented generation for its educational explanations. Instead of relying only on the vast training of the language model, the Tutor Agent retrieves the course material relevant to the student's query. The retrieval process is:

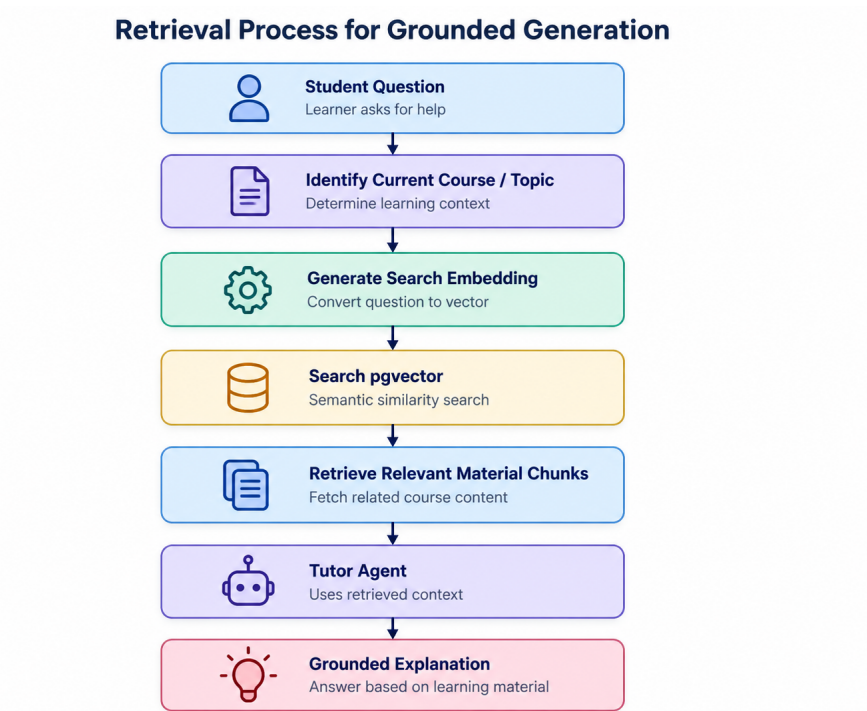


Figure 6.3: This figure represents the retrieval process of the system

The importance of this approach is that the teacher usually assumes that the student must use certain terminology, definitions, examples, or techniques from the class. Using retrieval, the Tutor Agent can be trained using the actual information provided in those materials so that the produced explanation is correct and aligned with it.

Plain prompting with no retrieval was taken into consideration but not used as it would be possible for the model to give an answer that is correct in general but does not correspond to the teacher's material. Furthermore, it would also provide less control over the factual source used by the Tutor Agent.

Fine-tuning on course materials was also considered. But this approach would need additional effort in preparation, training, evaluation, and retraining of the language model when the learning material is changed, which is not needed in the scope of this initial pilot project for Programming Fundamentals when the material can be constantly changing.

In conclusion, retrieval allows updating learning material without retraining the language model. New

learning material will be processed into chunks and embeddings and made available to the Tutor Agent through the same retrieval pipeline.

6.4.6 Error Detection and Recovery for External AI Services

As InSync depends on the use of third-party AI and embedding services, the system is vulnerable to delays, downtime, incorrect outputs, or availability issues. The system has a number of methods for protecting itself from one failing AI request taking down everything.

A time limit is applied to external AI requests. If a request fails or exceeds the configured timeout, the system retries the request up to two times. If the request continues to fail, the affected agent is marked temporarily unavailable and the learner receives a clear fallback message rather than leaving the interface in an indefinite loading state.

Structured format response by an agent is checked before writing it into the shared state/database. Validation in the FastAPI backend is possible with the help of Pydantic models.

LangGraph checkpoints ensure the safety of the state of the workflow, preventing loss of the generated data before the crash. The data of the learner remains in PostgreSQL and is not linked to any particular AI call. A provider-agnostic wrapper for LLMClient is used, since there's no need to depend on a particular provider of the language model. In case of future changing of the provider, the code of agents will remain mostly untouched.

First and foremost, crashes are isolated. In case of unavailability of the Progress Agent or Teacher Assistant Agent, a learner can use the available learning materials or features that do not require the specified agent. The same applies to the Tutor Agent – its crash should not shut down the whole FastAPI app.

6.4.7 Real-Time Communication in the Group Workspace

As group members work in the shared workspace, new chat messages, Discussion Agent hints, collaborative workspace events, and presence updates should appear without requiring the page to be refreshed. InSync uses FastAPI WebSockets to provide bidirectional real-time communication between the web client and backend. Redis supports temporary real-time information such as active user presence, collaboration events, and controlled-editing locks, while persistent group and message data is stored in PostgreSQL.

Supabase Realtime was considered because PostgreSQL is hosted through Supabase and it provides built-in database-driven updates. However, FastAPI WebSockets were selected because collaborative activities require backend-controlled authorization, agent-generated events, teacher monitoring, and controlled-editing behavior to pass through the application backend. Polling was also considered but rejected because it introduces repeated requests and delays in receiving real-time updates. Socket.IO

and Server-Sent Events were considered. Server-Sent Events are one-way and cannot carry chat input; Socket.IO adds a second protocol layer without a benefit for this scope.

6.4.8 Asynchronous Processing of Long AI Tasks

Some processes in InSync may consume more time than a few seconds. This includes transforming the uploaded learning content into chunks and embeddings, creating a quiz when the lesson session ends, and performing analysis of students' results after the quiz. Using such processes as normal synchronous calls would require the user to wait until the whole process completes, making the application seem non-responsive.

Asynchronous or background processing has been adopted by InSync in order to implement these processes. The client is able to present the user with an indication that the process is currently being performed and only the result is ready at the end of the process. The way of doing background processing will be determined during implementation.

Two approaches were evaluated. First, fully synchronous processing was discarded due to the fact that the user had to wait for the completion of lengthy AI tasks. Secondly, an entire task queue approach like Celery was also considered, however, it would have required additional components like a message broker and workers. This would add unnecessary complexity for a simple proof-of-concept involving a single Programming Fundamentals subject.

6.4.9 Privacy and Responsible Use of Learner Data

The InSync platform is based on a limited number of rules for collecting and using learner data. Only the required data for a certain task is transferred to the external AI service. Thus, the Tutor Agent can get the information about the student's current question and necessary retrieved chunks from the material provided by the teacher, but not the entire history of the student's studies. In the same way, the Teacher Assistant Agent works only with the necessary data that should be collected to create the required recommendation for the student.

Both students and teachers are informed about the data collected and its purpose and access to learner data is limited by user roles. Row-level security is used to provide access to the required records. Students can see their own learning records and necessary group information, while the teachers can see the data related to the classes they administer. The `discussion_problem` table that stores the hidden reference solutions to be used for group tasks is not visible to the client to prevent the students from seeing the solution before solving the task. The AI-generated material is marked as AI-generated wherever it is used.

The system intentionally refuses to collect some of the possible data. It does not collect emotions or

sensors' data in accordance with the scope of the project. Besides, it does not collect the learner data from outside sources, as the performance data is gathered from the activity in the InSync.

Sending a student's complete learning history to the AI service when any agent needs a context was one of the options considered in this case. However, it was refused, as it could expose much more information about the student to the external AI service than required for the task. Instead, the agents get the part of the learner context necessary for their task.

AI-generated output is not assumed to be correct by default. All tutoring responses should pass through validation while the hints generated by the Discussion Agent are verified against the problem information before showing the group.

6.5 Detailed System Interaction Sequence Diagram

The detailed system interaction sequence diagram presents the order of interactions among users and the major internal components of InSync during a learning workflow. It shows how requests move through the FastAPI backend, LangGraph coordination layer, specialized agents, and supporting data services.

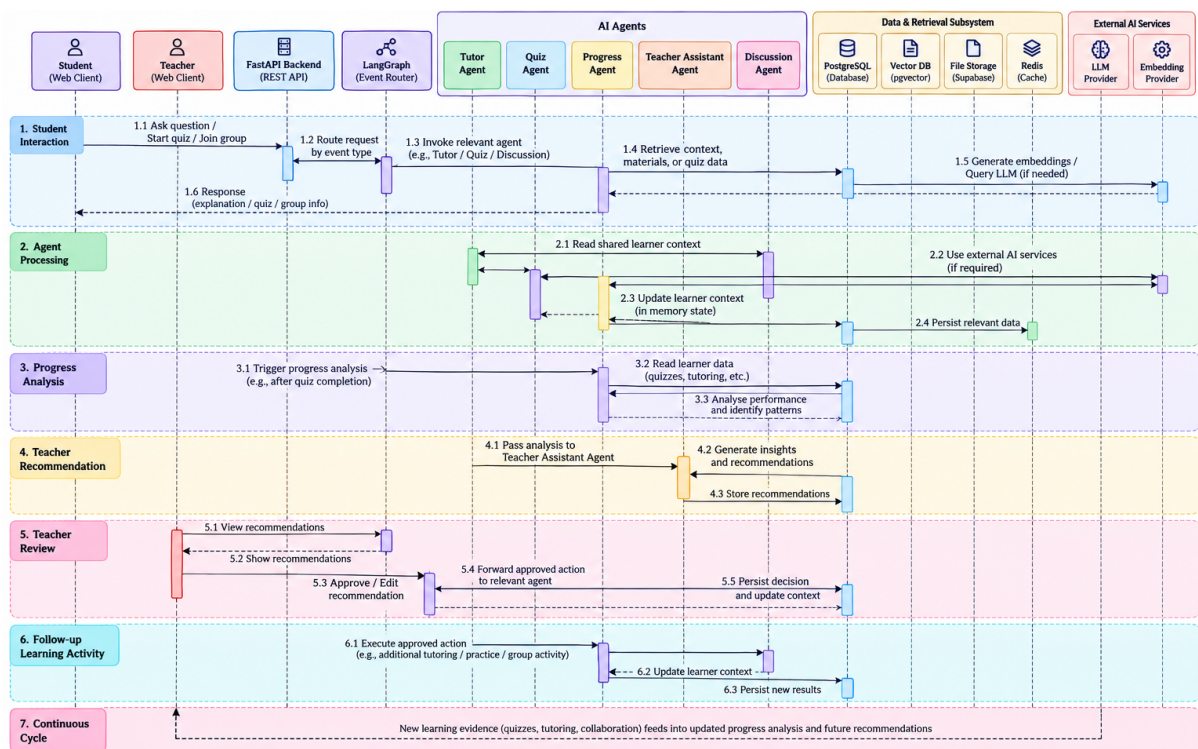


Figure 6.4: This figure represents the system sequence diagram of the system

6.6 Domain Model/Class Diagram

This section presents the structure of InSync using class diagrams. Class diagrams show the primary objects employed in InSync and their relations to one another.

6.6.1 Domain Model

A User is an account on the platform that holds common account information and the role of the user. Student and Teacher are two types of users and include the role-specific actions performed by them. A Teacher has ownership of one or multiple Class objects. Students and Classes are linked through class membership. Each Class object has multiple Topic objects inside, each having multiple Concept objects used to track individual learning weaknesses and strengths. Every Class has also LearningSession objects that are initiated and finished by the teacher.

A LearningSession may have an associated Quiz. Each Quiz contains one or more Question objects associated with concepts and difficulty levels. Student participation in a quiz is recorded through QuizAttempt, which stores the attempt and its resulting performance information.

LearnerContext represents persistent learner information that may be used during agent workflows. Related records such as concept mastery, tutoring interactions, quiz performance, and progress analyses provide additional evidence about the learner's previous activities. Relevant information from these records can be loaded into the shared agent state when required. It is complemented by other relevant information such as concept mastery, mistakes record, and progress snapshots. This way, the agents can rely on the relevant information from the previous learning activities when accomplishing their tasks.

A LearningSession may also include StudyGroup objects. Students and StudyGroups have group membership relation between them. Group messages and Discussion Agent hints allow study groups to engage in collaborative discussions. A Recommendation is created by the Teacher Assistant Agent using the progress information and provided to the teacher. A Recommendation includes relevant information that should be available to the teacher to understand the recommendation and its current review status.

Table 6.1: Domain Model Class Responsibilities

Class	Responsibility
User	Stores shared account information and the role common to every user.
Student	Represents a learner and supports student-specific actions such as joining sessions and attempting quizzes.

Class	Responsibility
Teacher	Represents a teacher and supports teacher-specific actions such as starting sessions, uploading learning material, and reviewing recommendations.
Class	Groups enrolled students under one teacher for a particular subject.
Topic	Represents a subject-matter area taught within a class.
Concept	Represents a finer-grained unit within a topic, used to track specific strengths and weaknesses.
LearningSession	Represents a learning session associated with a class and topic and managed by the teacher.
Quiz	Represents a set of questions created by the teacher or generated by the Quiz Agent after a learning session.
Question	Represents a single question associated with a difficulty level and concept.
QuizAttempt	Records a student's attempt at a quiz, including answers and performance results.
LearnerContext	Holds the learner's ongoing topic-level performance and relevant learning information used by the five agents.
StudyGroup	Represents a small group of students proposed by the Teacher Assistant Agent, approved or edited by the teacher, and assigned for collaborative learning and discussion.
GroupMessage	Represents a message posted by a student in a group workspace.
Hint	Represents a Discussion Agent hint posted in a group workspace, with its level and the reason it was given.
Recommendation	Represents a teacher-facing suggestion prepared by the Teacher Assistant Agent from Progress Agent analysis, including supporting evidence and review status.

6.6.2 Agent Layer Class Diagram

The agent layer is organized around an abstract `BaseAgent` class and a shared `InSyncState` class. The five specialized agents: Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, and Discussion Agent inherit the common behavior defined by `BaseAgent`. Supporting classes provide additional

functionality such as event routing, language-model access, retrieval of learning material, adaptive quiz difficulty, and collaborative hint generation.

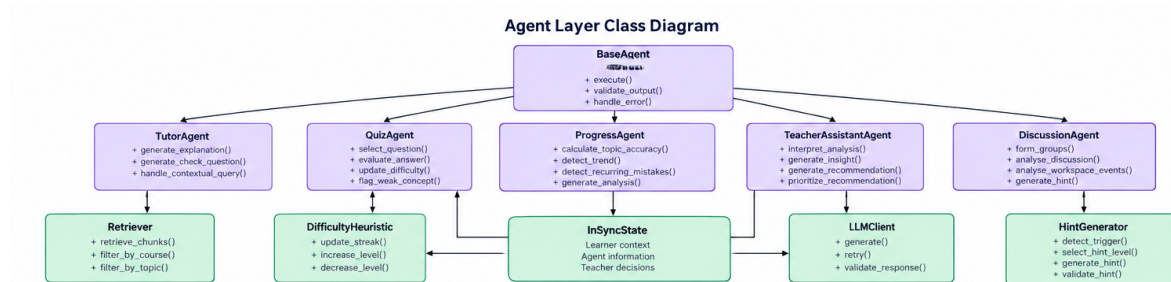


Figure 6.5: This figure represents the Agent Layer Class diagram of the system

Table 6.2: Agent Layer Class Responsibilities

Class	Responsibility
Base Agent	Defines shared behavior required by all agents, including execution, validation, and common error handling.
Tutor Agent	Provides personalized explanations, examples, and learner check questions using retrieved learning material.
Quiz Agent	Manages adaptive quiz interaction, evaluates responses, changes difficulty, and records weak concepts.
Progress Agent	Converts learner activity into performance metrics, trends, topic-level analysis, and recurring-mistake information.
TeacherAssistant Agent	Converts Progress Agent analysis into teacher-facing insights and evidence-based recommendations, and forms study groups on the teacher's request for the teacher to approve or edit.
Discussion Agent	Observes group discussion and meaningful workspace activity, and generates collaborative hints without revealing the answer.
GroupFormation Heuristic	Implements the balanced and strength-based rules used to propose study groups.
EventRouter	Determines which agent executes for a particular event and resumes workflows following teacher review.
InSyncState	Represents the shared state through which agents exchange relevant learner and workflow information.

Class	Responsibility
LLMClient	Provides a common interface to the selected external language-model provider and handles request-level validation and retry behavior.
Retriever	Searches teacher-provided learning material stored through pgvector and returns the most relevant chunks.
DifficultyHeuristic	Implements the rules that increase, decrease, or maintain adaptive quiz difficulty.
HintGenerator	Determines discussion-hint triggers and generates progressively informative hints without revealing the final answer.

The agents coordinate through the shared state represented by `InSyncState` rather than communicating with one another directly. Each agent reads the information required for its task and updates only the state information for which it is responsible. This reduces direct dependencies between the agents and helps prevent one agent from unintentionally overwriting information produced by another.

The Tutor Agent provides explanations and learning support using retrieved teacher-provided material. The Quiz Agent manages adaptive quizzes and evaluates learner responses. The Progress Agent analyzes learner performance and identifies patterns such as strengths, weaknesses, trends, and recurring mistakes. The Teacher Assistant Agent interprets the analysis produced by the Progress Agent and converts it into teacher-facing insights and recommendations. It also proposes study groups using the relevant learner-performance information, after which the teacher may approve or edit the proposed grouping. The Discussion Agent supports learners after group formation by analyzing collaborative discussion and relevant workspace activity and generating hints when required.

The Progress Agent and Teacher Assistant Agent are deliberately separated because they perform different responsibilities. The Progress Agent addresses the analytical question, “What is happening in the learner’s performance?”, while the Teacher Assistant Agent addresses “What should the teacher know, and what action may be appropriate?” and “How should the learners be grouped?” This separation keeps performance analysis independent from teacher-facing decision support and group recommendations.

Figure 6.5 presents the Agent Layer Class Diagram of `InSync`, including the inheritance relationships between `BaseAgent` and the five specialized agent classes, their associations with `InSyncState`, and their dependencies on supporting classes such as `EventRouter`, `LLMClient`, `Retriever`, `DifficultyHeuristic`, and `HintGenerator`.

6.7 Policies and Tactics

The policies and tactics below will be implemented throughout the development process of InSync. They provide guidelines on how the system should be developed and maintained without altering its architecture.

6.7.1 Coding Guidelines and Conventions

The FastAPI backend, including its application modules and Agent Service, follows PEP 8 conventions for Python code. The frontend is implemented using React and TypeScript and follows consistent TypeScript and ECMAScript coding conventions. Identifiers used in the codebase follow the naming conventions defined for the corresponding application and database components. Comments are added where necessary to explain non-obvious logic. Visual Studio Code is used as the primary development environment.

6.7.2 Source Code Organization

The source code is divided into folders depending on the purpose. The frontend/ directory contains the React and TypeScript web client. The backend/ directory contains the FastAPI application, including API routes, application services, database access, and authorization logic. Within the backend, the agents/ module contains the Tutor Agent, Quiz Agent, Progress Agent, Teacher Assistant Agent, and Discussion Agent together with the LangGraph coordination components.

Such an organization brings together related components and helps to collaborate as the team works on different parts of the system. Division between the application backend and the agent service matches the two logical subsystems, inside a single FastAPI application.

6.7.3 Version Control and Continuous Integration

GitHub is used as the shared repository for the code. Development takes place in feature branches instead of directly in the main branch, and changes are merged using pull requests to allow peer-review before the integration.

The continuous integration process will be relatively simple and perform basic checks and tests on pull requests before their integration to catch mistakes earlier without waiting until changes are committed to the main branch. Credentials such as language-model API keys, Supabase credentials, database connection strings, and other secrets are stored in environment variables and are never committed to the GitHub repository. Privileged credentials such as the Supabase service-role key are used only by server-side services and are never exposed to the web client.

6.7.4 Database Management

InSync uses PostgreSQL through Supabase as the main database. The application backend communicates with the database and does not allow any direct client-side database operation. Database structure changes are managed through migrations so that changes can be tracked and reproduced.

Access to the learner data is controlled by row-level security. Students have access only to their own records, and teachers have access to the data related to classes that they manage. Server-side database access is used in the case of some privileged database access, such as updating the shared context of the learner through agent operations.

Retrieval operations are restricted to the learner's authorized class and current topic before vector similarity search is performed. This prevents learning-material chunks belonging to unrelated classes or teachers from being returned through the Tutor Agent.

WebSocket connections for collaborative activities are authenticated before access is granted. The backend verifies that a student belongs to the requested study group or that a teacher manages the corresponding class before allowing the connection. Group identifiers received from the client are therefore not treated as sufficient authorization on their own.

6.7.5 Prompt Design and AI Output Validation

Each of the five agents uses a reusable prompt template in order to make sure that the set of instructions sent to the language model stays consistent among students and sessions. Agents are designed to return their results in a structured form, and the generated outputs are checked against the expected structure before they get stored or presented to a user. Invalid outputs are being rejected according to the agent-specific rules for validation and handling of errors.

In addition to this, the prompts for each of the agents match their role in education. So, for example, the Discussion Agent is asked to provide hints rather than full answers, allowing the use of hidden reference solutions to discussion problems, which are kept secret from students. All generated content by five agents is also filtered for any unsafe or inappropriate material.

6.7.6 User Interface

The InSync interface will be easy to use, modern and consistent without the need for any training for either students or teachers. The labels, messages and AI explanations will use clear language, and every piece of content that is generated by any agent will be marked as AI-generated. It will satisfy the accessibility guidelines: readable text, adequate contrast, meaningful labels, and absence of colour-only information presentation. Every recommendation on the teacher dashboard will present the evidence

and will provide options to approve or dismiss it.

6.7.7 Extensibility

The InSync project follows a modular design pattern, making future extension possible without significant modifications. Subject content and agent coordination are separated, so adding a new subject is done by providing the corresponding topics, concepts, learning materials, and assessment content. The five agents are designed as separate components coordinated through a shared state and LangGraph routing. This reduces direct dependencies between agent implementations. As a result, new agents or alternative model providers can be introduced with limited changes to existing components, provided that the required state interfaces and routing rules remain compatible.

6.7.8 Engineering Trade-offs

A few trade-offs have been considered during InSync development, taking into account such aspects as system performance, cost, development time and learner safety:

- Simple models (for example, quiz heuristic based on streaks) were preferred to more complex ones because they are easier to implement and explain in terms of project limitations.
- Teacher approval of certain actions makes some reactions slow, but ensures learners' safety from wrong AI decisions.
- An external language model API allows reducing development efforts and achieving good results in terms of quality at the cost of request fees and third-party dependency.

These trade-offs are documented for consistency of design decisions.

6.8 Conclusion

This chapter presented the high-level and low-level design of InSync, including the Web Client, FastAPI application backend, Agent Service, and Data and Retrieval subsystem. InSync uses five specialized agents coordinated through LangGraph and a shared state. Information recorded by one agent can therefore be made available to other agents during later stages of the learning workflow without requiring direct agent-to-agent communication. The main architectural decisions include the use of PostgreSQL with pgvector for persistent and retrieval data, retrieval-grounded tutoring based on teacher-provided learning materials, real-time collaborative functionality, modular agent coordination, and human-in-the-loop review for important AI-supported decisions.

Bibliography

- [1] E. Sun and L. Tai, “MultiTutor: Collaborative LLM agents for multimodal student support,” in *Proceedings of the Innovation and Responsibility in AI-Supported Education Workshop*, ser. Proceedings of Machine Learning Research, vol. 273. PMLR, Mar. 2025, pp. 174–190. [Online]. Available: <https://proceedings.mlr.press/v273/sun25a.html>
- [2] J. Gonnermann-Müller, J. Haase, N. Leins, M. Igel, K. Fackeldey, and S. Pokutta, “FACET: Multi-agent AI supporting teachers in scaling differentiated learning for diverse students,” in *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*, Bremen, Germany, Aug. 2026, pp. 7165–7173. [Online]. Available: <https://www.ijcai.org/proceedings/2026/797>
- [3] Z. Hao, J. Cao, R. Li, J. Yu, Z. Liu, and Y. Zhang, “Mapping student-AI interaction dynamics in multi-agent learning environments: Supporting personalized learning and reducing performance gaps,” *Computers & Education*, vol. 241, p. 105472, 2026.
- [4] H. Tebourbi, S. Nouzri, Y. Mualla, and A. Abbas-Turki, “Artificial intelligence agents for personalized adaptive learning,” *Procedia Computer Science*, vol. 265, pp. 252–259, 2025.
- [5] L. Zheng, Z. Shi, and L. Gao, “A generative artificial intelligence-enhanced multiagent approach to empowering collaborative problem solving across different learning domains,” *Computers & Education*, vol. 241, p. 105489, 2026.
- [6] A. Lippert, K. Shubeck, B. Morgan, A. Hampton, and A. Graesser, “Multiple agent designs in conversational intelligent tutoring systems,” *Technology, Knowledge and Learning*, vol. 25, no. 3, pp. 443–463, Sep. 2020.
- [7] H. Kumar, Z. K. Mu, J. Vincentius, and A. Anderson, “Beyond the AI tutor: Social learning with LLM agents,” arXiv:2604.02677, 2026. [Online]. Available: <https://arxiv.org/abs/2604.02677>
- [8] Z. Xiao, H. Wang, S. Jin, Y. Lai, Y. Cao, Y. Zhang, and C. Wang, “Human-AI collaborative learning ecosystem: Effects of multi-agent-based programming learning on

- learning outcomes, computational thinking, and behavioral patterns in higher education,” *International Journal of Human–Computer Interaction*, Jul. 2026, early access. [Online]. Available: <https://doi.org/10.1080/10447318.2026.2692021>
- [9] Khan Academy, “Harnessing GPT-4 so that all students benefit: A nonprofit approach for equal access,” [Online]. Available: <https://blog.khanacademy.org/harnessing-ai-so-that-all-students-benefit-a-nonprofit-approach-for-equal-access/>, Mar. 2023, accessed: Oct. 4, 2026.
- [10] —, “Meet Khanmigo: Khan Academy’s AI-powered teaching assistant and tutor,” [Online]. Available: <https://www.khanmigo.ai/>, accessed: Oct. 4, 2026.
- [11] —, “What reports does Khan Academy offer teachers to monitor their students’ Khanmigo use?” [Online]. Available: <https://support.khanacademy.org/hc/en-us/articles/29473549307277>, Oct. 2024, accessed: Oct. 4, 2026.
- [12] —, “Khan Academy annual report: SY24–25,” [Online]. Available: <https://annualreport.khanacademy.org/>, 2025, accessed: Oct. 4, 2026.
- [13] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, “AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems,” arXiv:2601.04219, 2026. [Online]. Available: <https://arxiv.org/abs/2601.04219>
- [14] Y. Shi, R. Liang, and Y. Xu, “EducationQ: Evaluating LLMs’ teaching capabilities through multi-agent dialogue framework,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 32 799–32 828. [Online]. Available: <https://aclanthology.org/2025.acl-long.1576/>
- [15] Z. Chu, S. Wang, J. Xie, T. Zhu, Y. Yan, J. Ye, A. Zhong, X. Hu, J. Liang, P. S. Yu, and Q. Wen, “LLM agents for education: Advances and applications,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 13 782–13 810. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.743/>
- [16] B. Tokoli, L. Jaimes, and A. S. Dina, “ALIGNAgent: Adaptive learner intelligence for gap identification and next-step guidance,” arXiv:2601.15551, 2026. [Online]. Available: <https://arxiv.org/abs/2601.15551>
- [17] J. Sesay, Y. Yu, S. Dong, Y. Shi, G. Chen, and B. F. Karlsson, “LectūraAgents: A

- multi-agent framework for adaptive personalized AI-assisted learning and embodied teaching,” arXiv:2606.16428, 2026. [Online]. Available: <https://arxiv.org/abs/2606.16428>
- [18] H. Wang, C. Wang, Z. Chen, F. Liu, C. Bao, and X. Xu, “Impact of AI-agent-supported collaborative learning on the learning outcomes of university programming courses,” *Education and Information Technologies*, vol. 30, no. 12, pp. 17 717–17 749, Aug. 2025.